

CS 6120

Natural Language Processing

Instructor: Amir Tahmasebi, PhD

Fall 2026



Parsing



Previously...

- Regular Expressions
- Tokenization
 - Word-level
 - Char-level
 - Subword-level
- Tokenization methods
 - BPE
 - Unigram
 - WordPiece
 - SentencePiece
- Normalization
 - Stemming
 - Lemmatization

Announcements

Absence Submission

Please submit prior to the session you are going to be absent from.

[Absence Submission Form – Fill out form](#)

Absence Submission Form



QUIZ – Cheat Sheet

Special characters

<code>\</code>	escape special characters
<code>.</code>	matches any character
<code>^</code>	matches beginning of string
<code>\$</code>	matches end of string
<code>[5b-d]</code>	matches any chars '5', 'b', 'c' or 'd'
<code>[^a-c6]</code>	matches any char except 'a', 'b', 'c' or '6'
<code>R S</code>	matches either regex <code>R</code> or regex <code>S</code>
<code>()</code>	creates a capture group and indicates precedence

Special sequences

<code>\A</code>	start of string
<code>\b</code>	matches empty string at word boundary (between <code>\w</code> and <code>\W</code>)
<code>\B</code>	matches empty string not at word boundary
<code>\d</code>	digit
<code>\D</code>	non-digit
<code>\s</code>	whitespace: <code>[\t\n\r\f\v]</code>
<code>\S</code>	non-whitespace
<code>\w</code>	alphanumeric: <code>[0-9a-zA-Z_]</code>
<code>\W</code>	non-alphanumeric
<code>\Z</code>	end of string
<code>\a<id></code>	matches a previously defined group

Quantifiers

<code>*</code>	0 or more (append <code>?</code> for non-greedy)
<code>+</code>	1 or more (append <code>?</code> for non-greedy)
<code>?</code>	0 or 1 (append <code>?</code> for non-greedy)
<code>{m}</code>	exactly <code>m</code> occurrences
<code>{m, n}</code>	from <code>m</code> to <code>n</code> . <code>m</code> defaults to 0, <code>n</code> to infinity
<code>{m, n}?</code>	from <code>m</code> to <code>n</code> , as few as possible

Special sequences

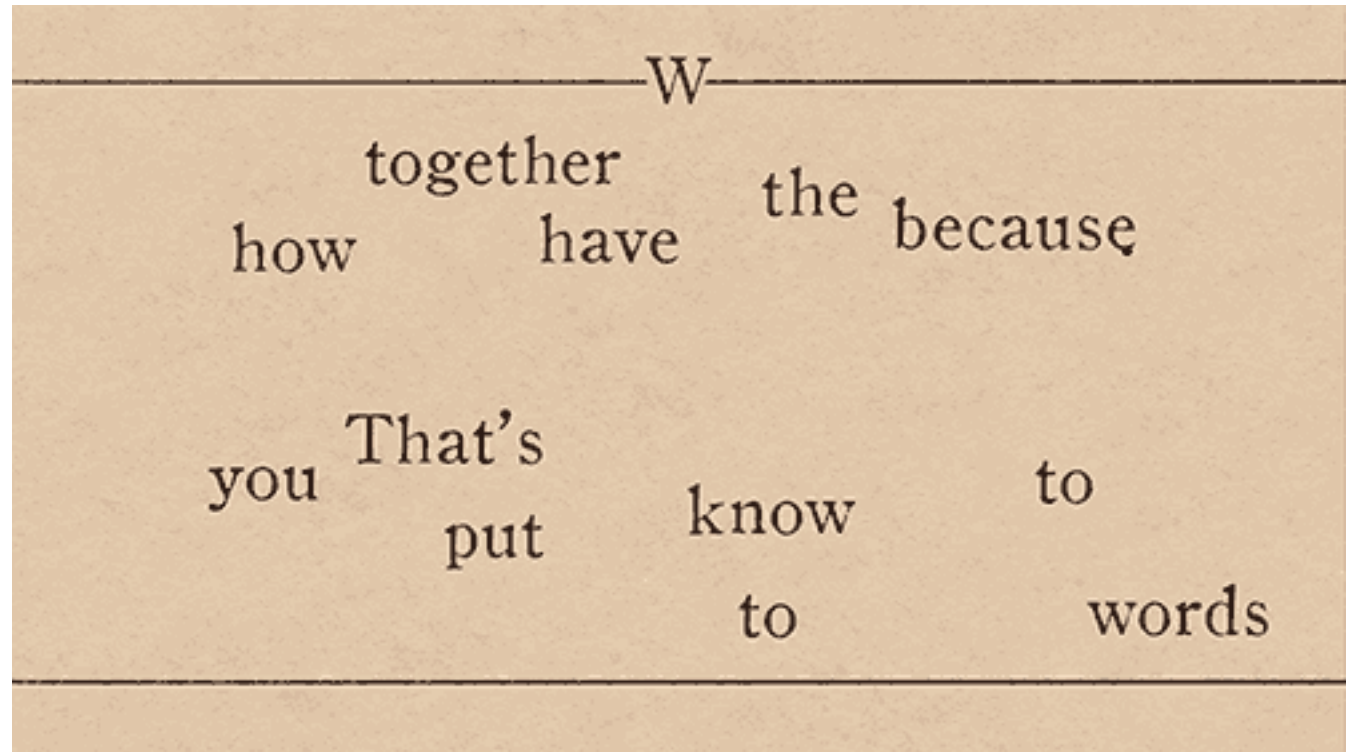
<code>(?iLmsux)</code>	matches empty string, sets re.X flags
<code>(?:...)</code>	non-capturing version of regular parentheses
<code>(?P...)</code>	matches whatever matched previously named group
<code>(?P=)</code>	digit
<code>(?#...)</code>	a comment; ignored
<code>(?=...)</code>	lookahead assertion: matches without consuming
<code>(?!...)</code>	negative lookahead assertion
<code>(?<=...)</code>	lookbehind assertion: matches if preceded
<code>(?<!=...)</code>	negative lookbehind assertion
<code>(? (id)yes no)</code>	match 'yes' if group 'id' matched, else 'no'

Syllabus

- What is Natural Language Processing?
 - History
 - Applications
- Python Primer
- Text Processing Fundamentals
 - Regular Expressions
 - Tokenization
 - Normalization
 - Libraries and Toolkits
- Parsing
 - Syntactic
 - Semantic
- Word Representation
 - TF-IDF
 - word2vec
 - Glove
- Intro to Machine Learning
 - Logistic Regression (LR)
 - Neural Networks
 - Recurrent Neural Networks (RNN)
 - Convolutional Neural Networks (CNN)
 - Transformers
- Language Models
- NLP Use Cases
 - Named Entity Recognition
 - Machine Translation
 - QA
- Large Language Models
 - chatGPT

Parsing

- Process of analyzing the structure of a text, both in terms of its grammar (syntactic parsing) and its meaning (semantic parsing).

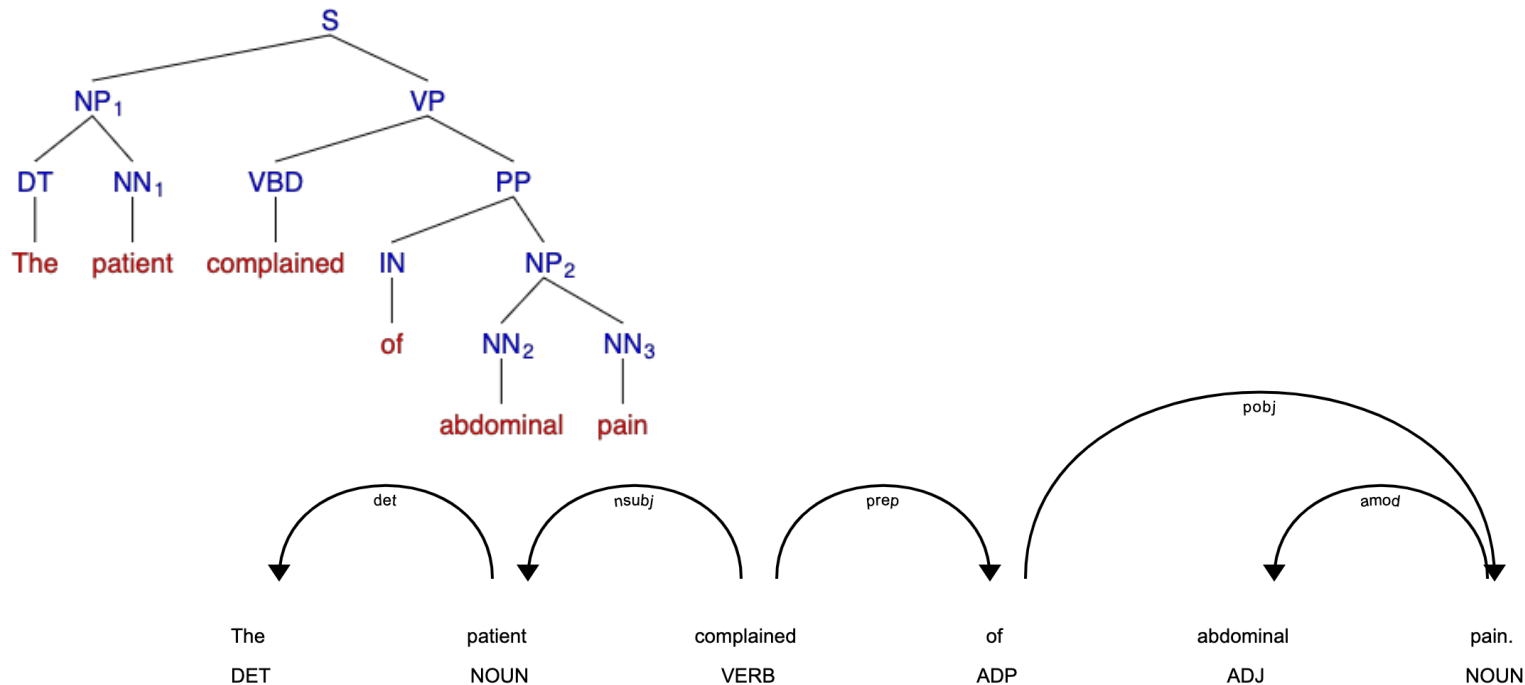


<https://www.google.com/url?sa=i&url=https%3A%2F%2Ftowardsdatascience.com%2Fnlp-extracting-the-main-topics-from-your-dataset-using-lda-in-minutes-21486f5aa925&psig=AOvWaw31kUTa37Hx8EjdINIO-3K&ust=1716255907266000&source=images&cd=vfe&opi=89978449&ved=0CBEQRxqFwoTCMC2ps-Nm4YDFQAAAAAdAAAAAB88>

Parsing

- Break down a sentence into its constituent elements and understanding their syntactic roles, relationships, as well as their semantic implications

e.g. The patient complained of abdominal pain.



```
{  
  "action": "complain",  
  "agent": "patient",  
  "complaint": {  
    "type": "pain",  
    "location": "abdominal"  
  }  
}
```

Parsing: *Why?*

Machine Translation

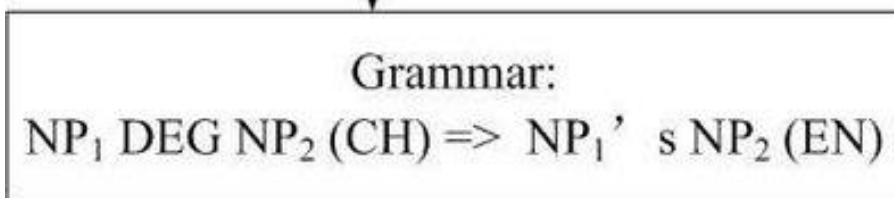
(a)

CH:

玛丽 的 父母

EN:

Mary 's parents



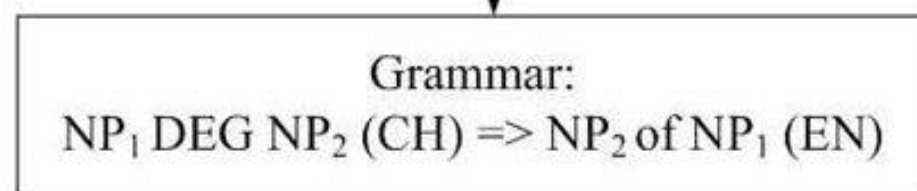
(b)

CH:

机器翻译 的 应用

EN:

the application of machine translation



Parsing: *Why?*

Text Summarization



Results

Input

In an ad hoc announcement dated 5 April 2019, **the BMW Group** reported that the EU Commission had informed it of a "Statement of Objections" in conjunction with ongoing antitrust proceedings. The EU Commission alleges that the manufacturers colluded with the aim of restricting innovation and competition with regard to certain exhaust treatment systems for diesel- and petrol-driven passenger vehicles. The allegation concerns selective catalytic reduction (SCR) systems and the use of petrol particulate filters (OPF). The Commission's preliminary view is that the conduct objected to may be in breach of EU competition rules. The Statement of Objections leads **the BMW Group** to conclude that it is probable ("more likely than not") that the EU Commission will issue a significant fine. The resulting requirement to recognise a provision increased other operating expenses by approximately €1.4 billion in 2019.

Output

The BMW Group has confirmed that it is likely to be fined 1.4 billion by the EU Commission for allegedly colluding with other car manufacturers to fix the exhaust systems of passenger vehicles.

Parsing ...Tokenization?

Tokenization: Breaking text into pieces

Parsing: figuring out how those pieces relate to each other

Aspect	Tokenization	Parsing
Purpose	Split text	Understand structure
Output	List of tokens	Tree or graph
Grammar-aware	No	Yes
Typical position	Very early	Later in pipeline
Complexity	Low	Higher
Example question	“What are the words?”	“Who did what to whom?”

Parsing

- **Types:**
 - Syntactic Parsing
 - Constituency Parsing
 - Dependency Parsing
 - Semantic Parsing

Parsing

- **Types:**

- **Syntactic Parsing**

- Constituency Parsing
 - Dependency Parsing

- Semantic Parsing

process of analyzing a string of words (typically a sentence) according to the rules of a formal grammar to determine its grammatical structure: that is, how words group into phrases and how those phrases relate to one another hierarchically.

A **constituent** is a group of words (or a single word) that function together as a single unit within a sentence's structure.

Parsing

- **Types:**

- Syntactic Parsing
 - Constituency Parsing
 - Dependency Parsing
- **Semantic Parsing**

goes beyond syntactic structure to extract a sentence's meaning or semantics. It attempts to understand the roles of words in the context of a certain task and how they interact with one another.

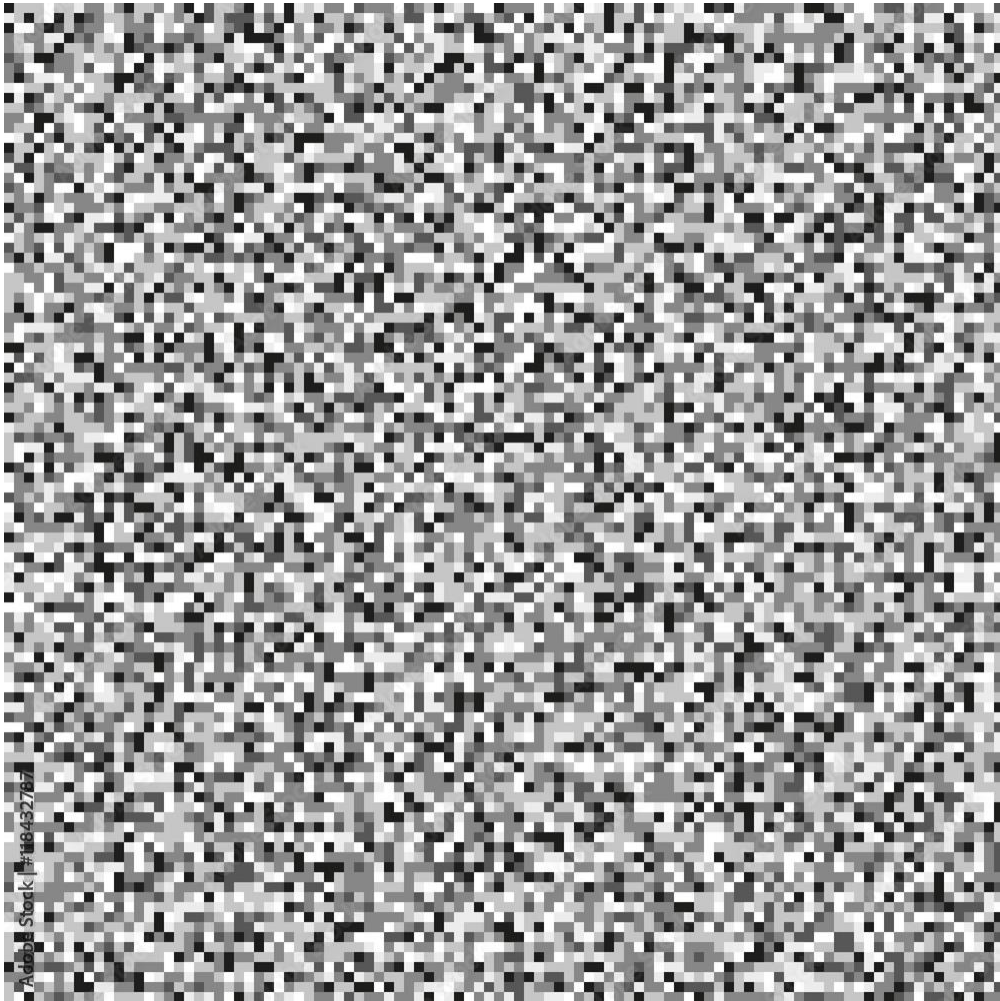
Parsing

- **Types:**
 - **Syntactic Parsing**
 - Constituency Parsing
 - Dependency Parsing
 - Semantic Parsing

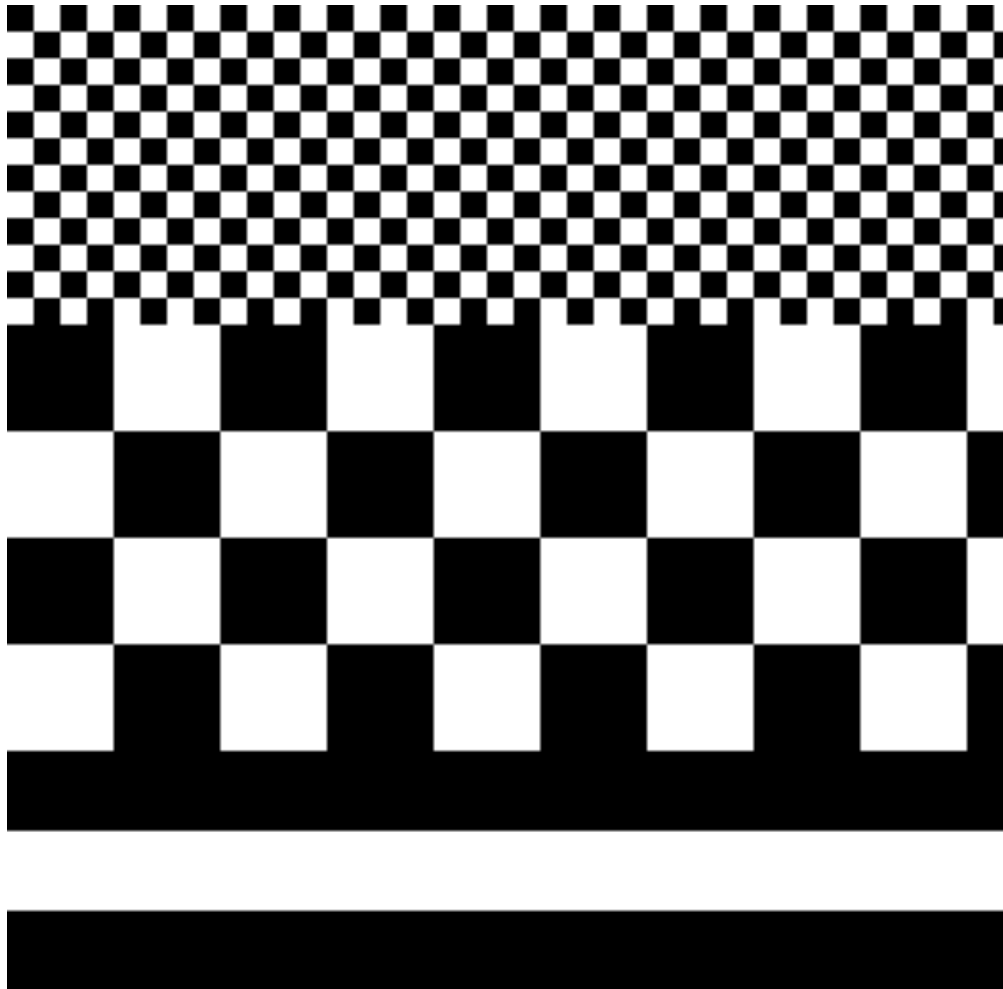
Parsing: *terminology*

- **Grammar**
- **Syntax**
- **Part-of-Speech (POS) tags**
- **Constituency**
- **Constituency Grammar**
- **Context-free Grammar (CFG)**

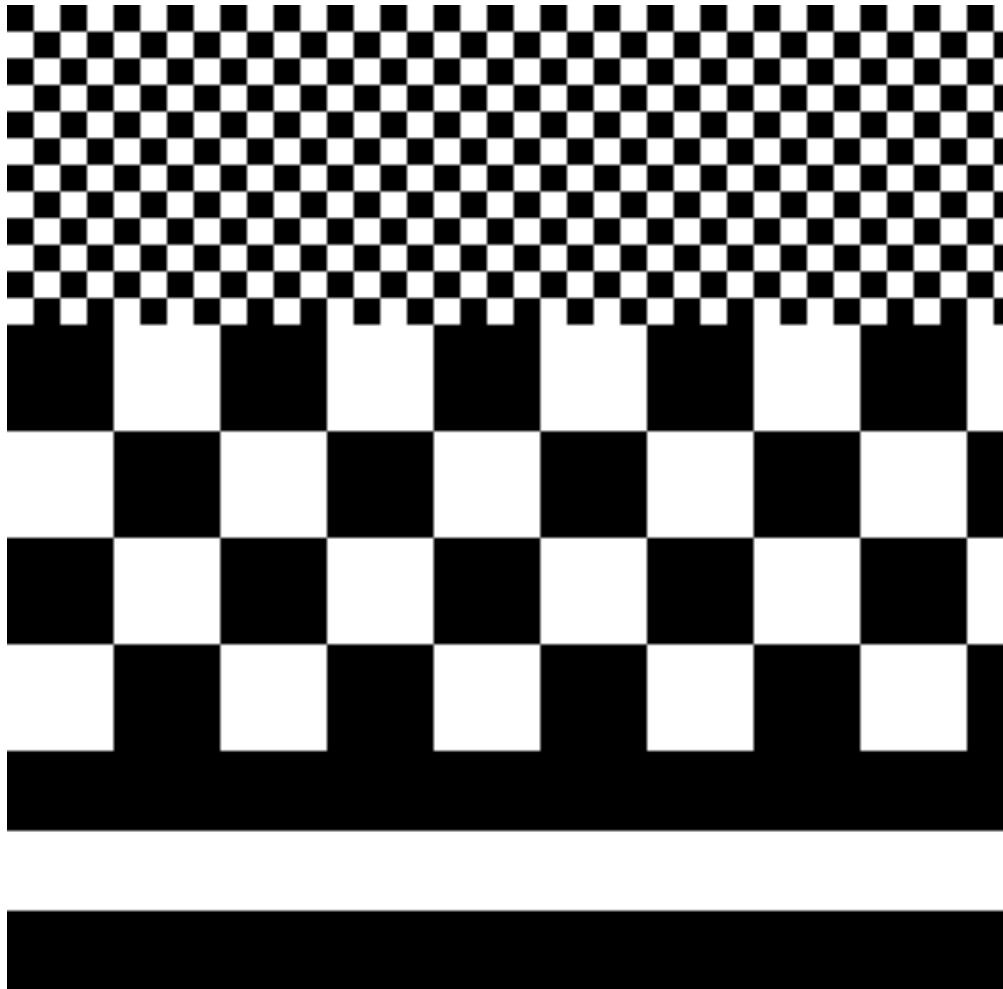
Syntax and Grammar



Syntax and Grammar



Syntax and Grammar



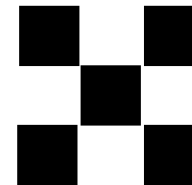
Building
blocks



edge



pattern



Syntax and Grammar

IRUIQK
BROWN
JUMPSS
LAYZZY
THE DOG



Syntax and Grammar



"The quick brown fox jumps
over the lazy dog"

Syntax and Grammar

"The quick brown fox jumps over the lazy dog"

Nouns: "fox", "dog"

Verbs: "jumps"

Adjectives: "quick", "brown", "lazy"

Articles: "the"

"The quick brown fox": **Subject**

"jumps": **Verb**

"over the lazy dog": **Adverbial Phrase**.

Syntax and Grammar

"The quick brown fox jumps over the lazy dog"

Nouns: "fox", "dog"

Verbs: "jumps"

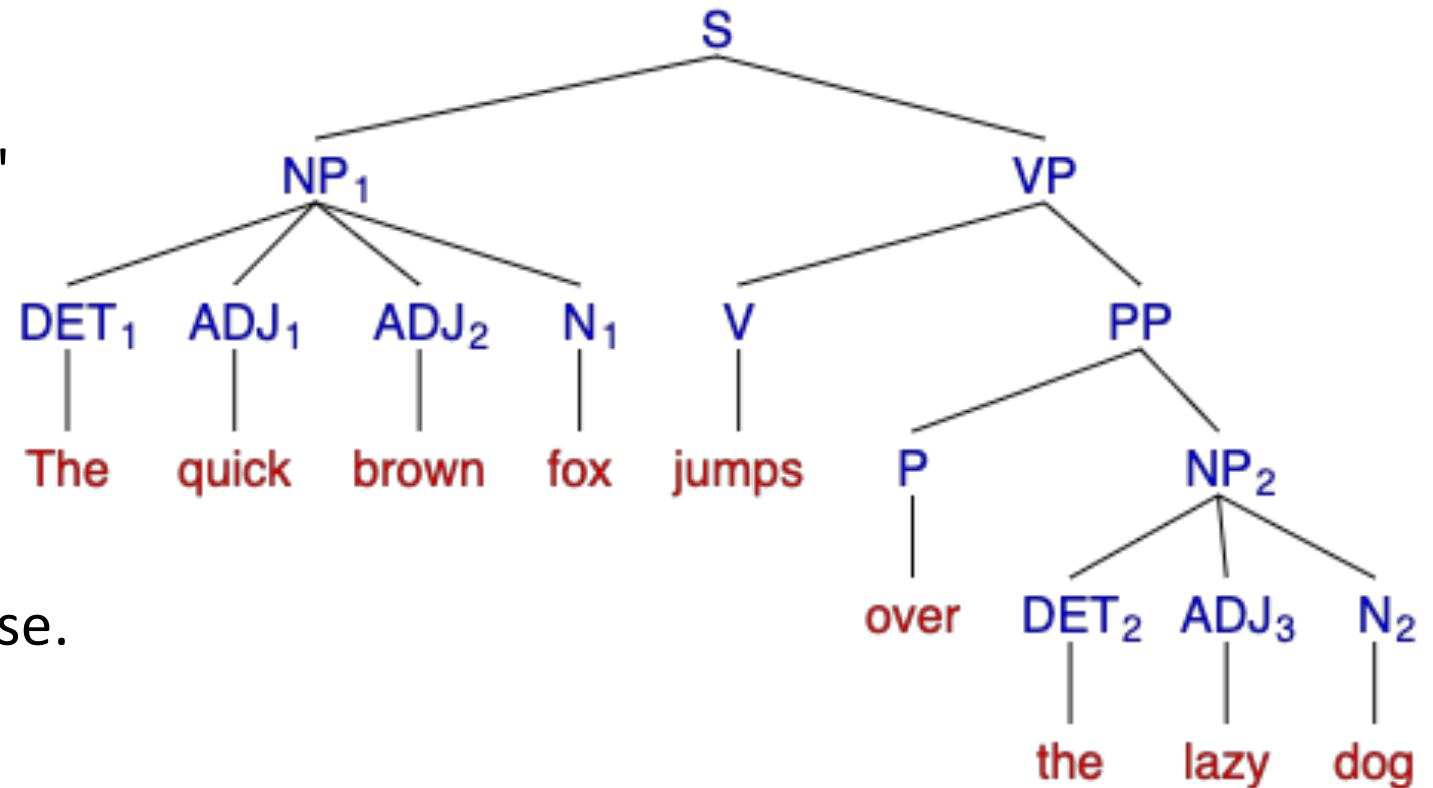
Adjectives: "quick", "brown", "lazy"

Articles: "the"

"The quick brown fox": **Subject**

"jumps": **Verb**

"over the lazy dog": **Adverbial Phrase.**



Grammar

Grammar: set of rules that govern the composition of phrases, clauses, and words in any given natural language

encompasses the structure of a language including:

- **Phonology:** sound patterns (in spoken language)
- **Morphology:** how words are formed from smaller units (e.g. adding "-ed" for past tense)
- **Syntax:** how words are arranged into phrases and sentences
- **Semantics:** how meaning is constructed and interpreted

Syntax

Syntax: arrangement of words and phrases to create well-formed sentences in a language in the form of rules that dictate how words combine to form phrases and sentences.

Syntactic categories and their common denotations in NLP:

NP - noun phrase,

VP - verb phrase,

S - sentence,

DET - determiner (article),

N/NN - noun,

TV - transitive verb (takes an object),

IV - intransitive verb,

PREP - preposition,

PP - prepositional phrase,

ADJ - adjective

Syntax

Syntax: arrangement of words and phrases to create well-formed sentences in a language in the form of rules that dictate how words combine to form phrases and sentences.

Syntactic categories and their common denotations in NLP:

NP - noun phrase,
VP - verb phrase,
S - sentence,
DET - determiner (article),
N/NN - noun,
TV - transitive verb (takes an object),
IV - intransitive verb,
PREP - preposition,
PP - prepositional phrase,
ADJ - adjective



Part-Of-Speech (**POS**) Tags

Part-of-Speech (POS) Tagging

Process of assigning grammatical categories to words in a sentence to identify their roles and functions within a sentence

POS Tag	Description	Examples
CC	Coordinating conjunction	and, but, or, so, yet
CD	Cardinal number	one, two, three, four, five
DT	Determiner	the, a, an, some, all, every
EX	Existential there	there
FW	Foreign word	coup d'état, laissez-faire
IN	Preposition or subordinating conjunction	in, on, at, to, from, by, like, because, if
JJ	Adjective	big, small, beautiful, ugly, fast, slow
JJR	Comparative adjective	bigger, smaller, more beautiful, uglier, faster, slower
JJS	Superlative adjective	biggest, smallest, most beautiful, ugliest, fastest, slowest
LS	List item marker	1, 2, a, b, i, ii
MD	Modal verb	can, could, may, might, must, should, will, would
NN	Noun (singular or mass)	dog, cat, tree, book, water
NNS	Noun (plural)	dogs, cats, trees, books
NNP	Proper noun (singular)	John, Mary, Paris, London
NNPS	Proper noun (plural)	the Johnsons, the Smiths, Paris and London
PDT	Predeterminer	all, both, half, many, much, several
POS	Possessive ending ('s)	John's, Mary's
PRP	Personal pronoun	I, you, he, she, it, we, they, me, him, her, us, them
PRP\$	Possessive pronoun	mine, yours, his, hers, its, ours, theirs
RB	Adverb	quickly, slowly, carefully, well, badly
RBR	Comparative adverb	more quickly, more slowly, more carefully, better, worse
RBS	Superlative adverb	most quickly, most slowly, most carefully, best, worst
RP	Particle	up, down, in, out, on, off, at, to
SYM	Symbol	%, &, #, @, *
TO	To infinitive marker	to
UH	Interjection	oh, ah, wow, hey, hello
VB	Base verb	eat, sleep, walk, talk, run
VBD	Past tense verb	ate, slept, walked, talked, ran
VBG	Verb gerund (-ing)	eating, sleeping, walking, talking, running
VBN	Past participle verb	eaten, slept, walked, talked, run
VBP	Non-past (present) tense verb	eat, sleep, walk, talk, run
VBZ	Present tense verb (third person singular)	eats, sleeps, walks, talks, runs
WDT	Wh-determiner	who, which, what, that, whose
WP	Wh-pronoun	who, which, what, whom
WRB	Wh-adverb	when, where, why, how

Penn Treebank Tagset

Tag	Description	Example	Tag	Description	Example
CC	Coordin. Conjunction	<i>and, but, or</i>	SYM	Symbol	<i>+, %, &</i>
CD	Cardinal number	<i>one, two, three</i>	TO	“to”	<i>to</i>
DT	Determiner	<i>a, the</i>	UH	Interjection	<i>ah, oops</i>
EX	Existential ‘there’	<i>there</i>	VB	Verb, base form	<i>eat</i>
FW	Foreign word	<i>mea culpa</i>	VBD	Verb, past tense	<i>ate</i>
IN	Preposition/sub-conj	<i>of, in, by</i>	VBG	Verb, gerund	<i>eating</i>
JJ	Adjective	<i>yellow</i>	VCN	Verb, past participle	<i>eaten</i>
JJR	Adj., comparative	<i>bigger</i>	VBP	Verb, non-3sg pres	<i>eat</i>
JJS	Adj., superlative	<i>wildest</i>	VBZ	Verb, 3sg pres	<i>eats</i>
LS	List item marker	<i>1, 2, One</i>	WDT	Wh-determiner	<i>which, that</i>
MD	Modal	<i>can, should</i>	WP	Wh-pronoun	<i>what, who</i>
NN	Noun, sing. or mass	<i>llama</i>	WP\$	Possessive wh-	<i>whose</i>
NNS	Noun, plural	<i>llamas</i>	WRB	Wh-adverb	<i>how, where</i>
NNP	Proper noun, singular	<i>IBM</i>	\$	Dollar sign	<i>\$</i>
NNPS	Proper noun, plural	<i>Carolinas</i>	#	Pound sign	<i>#</i>
PDT	Predeterminer	<i>all, both</i>	“	Left quote	<i>(‘ or “</i>
POS	Possessive ending	<i>’s</i>	”	Right quote	<i>(’ or ”</i>
PP	Personal pronoun	<i>I, you, he</i>	(	Left parenthesis	<i>([, { , <</i>
PP\$	Possessive pronoun	<i>your, one’s</i>	)	Right parenthesis	<i>(] , } , ></i>
RB	Adverb	<i>quickly, never</i>	,	Comma	<i>,</i>
RBR	Adverb, comparative	<i>faster</i>	.	Sentence-final punc	<i>(. ! ?</i>
RBS	Adverb, superlative	<i>fastest</i>	:	Mid-sentence punc	<i>(: ; ... - -)</i>
RP	Particle	<i>up, off</i>			

POS Tags: *Ambiguity*

Words may have more than one POS tag:

Example:

Back

The back door: JJ

On my back: NN

Win the voters back: RB

Promised to back the bill: VB

		87-tag Original Brown	45-tag Treebank Brown
Unambiguous (1 tag)		44,019	38,857
Ambiguous (2–7 tags)		5,490	8844
Details:	2 tags	4,967	6,731
	3 tags	411	1621
	4 tags	91	357
	5 tags	17	90
	6 tags	2 (<i>well, beat</i>)	32
	7 tags	2 (<i>still, down</i>)	6 (<i>well, set, round, open, fit, down</i>)
	8 tags		4 (<i>'s, half, back, a</i>)
	9 tags		3 (<i>that, more, in</i>)

Parsing: *terminology*

- Grammar
- Syntax
- Part-of-Speech (POS) tags
- **Constituency**
- **Constituency Grammar**
- **Context-free Grammar (CFC)**

Constituency

- **IDEA:** groups of words can behave as single units -
Constituents

Harry the Horse
the Broadway coppers
they

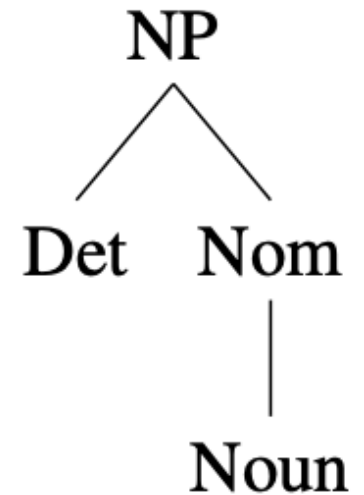
a high-class spot such as Mindy's
the reason he comes into the Hot Box
three parties from Brooklyn

WHY can be grouped?

BECAUSE they can all appear in similar syntactic environments

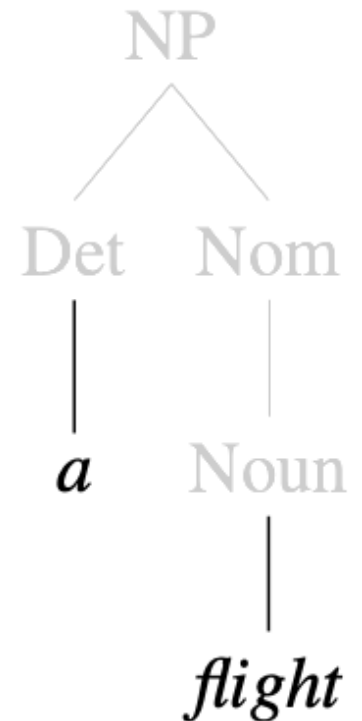
Constituency Trees

- Internal nodes correspond to phrases
 - *S* (sentence)
 - *NP* (noun phrase): *a flight*
 - *VP* (verb phrase): *landed*
 - *PP* (prepositional phrase): *with a friend*
- Nodes immediately above words are part-of-speech (POS) tags
 - *PN*: Pronoun
 - *Det*: Determiner
 - *V*: Verb
 - *N*: Noun
 - *P*: Preposition



Constituency Trees

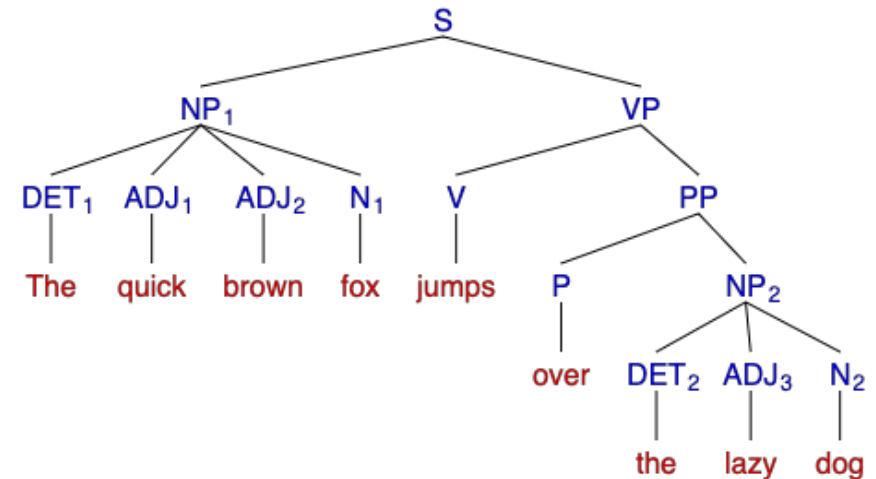
- Internal nodes correspond to phrases
 - *S* (sentence)
 - *NP* (noun phrase): *a flight*
 - *VP* (verb phrase): *landed*
 - *PP* (prepositional phrase): *with a friend*
- Nodes immediately above words are part-of-speech (POS) tags
 - *PN*: Pronoun
 - *Det*: Determiner
 - *V*: Verb
 - *N*: Noun
 - *P*: Preposition



Grammar: Types

Types:

- **Constituency grammar**
 - a type of syntax theory in linguistics that views a sentence's structure in terms of the constitution or arrangement of its subparts, known as constituents, often hierarchically organized in a tree structure.
 - Context-free grammar (CFG)
- Dependency grammar



Grammar: Types

Types:

- Constituency grammar
 - **Context-free grammar (CFG)**
 - a formal grammar that consists of a set of rules for how to combine smaller grammatical units to form larger ones [based on the syntactic categories of the units, rather than their specific context in a sentence].

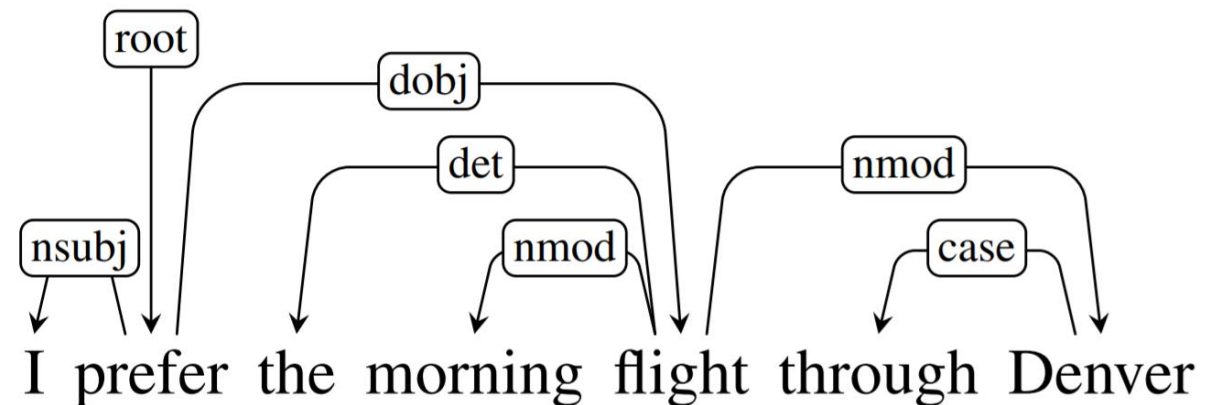
- Dependency grammar

```
⟨SENTENCE⟩ → ⟨NOUN-PHRASE⟩⟨VERB-PHRASE⟩
⟨NOUN-PHRASE⟩ → ⟨CMPLX-NOUN⟩ | ⟨CMPLX-NOUN⟩⟨PREP-PHRASE⟩
⟨VERB-PHRASE⟩ → ⟨CMPLX-VERB⟩ | ⟨CMPLX-VERB⟩⟨PREP-PHRASE⟩
⟨PREP-PHRASE⟩ → ⟨PREP⟩⟨CMPLX-NOUN⟩
⟨CMPLX-NOUN⟩ → ⟨ARTICLE⟩⟨NOUN⟩
⟨CMPLX-VERB⟩ → ⟨VERB⟩ | ⟨VERB⟩⟨NOUN-PHRASE⟩
  ⟨ARTICLE⟩ → a | the
    ⟨NOUN⟩ → boy | girl | flower
      ⟨VERB⟩ → touches | likes | sees
        ⟨PREP⟩ → with
```

Grammar: Types

Types:

- Constituency grammar
 - Context-free grammar (CFG)
- **Dependency grammar**
 - a class of syntax theory in linguistics that is based on the dependency relation between words in a sentence.



Grammar: Types

Types:

- Constituency grammar
 - **Context-free grammar (CFG)**
- Dependency grammar

Constituency / Context-Free Grammar (CFG)

- CFGs can be used to generate constituency trees
- Not all constituency trees can be generated by CFGs

Feature	Context-Free Grammar (CFG)	Constituency Grammar
Emphasis	Syntactic categories	Hierarchical structure of sentences
Formalization	Formal rules	Phrase structure trees
Applications	Parsing, machine translation	Language modeling, information extraction

Context-Free Grammar (CFG)

- Also known as *phrase-structure* grammars
- Consists of a set of rules [or productions], each of which expresses the ways that symbols of the language can be grouped and ordered together

Context-Free Grammar (CFG)

- Also known as *phrase-structure* grammars
- Consists of a set of rules [or productions], each of which expresses the ways that symbols of the language can be grouped and ordered together

Terminology:

Root: represents the entire grammatical unit.

Terminal: symbols that correspond to words in the language

Non-terminal: symbols that express abstractions over terminals

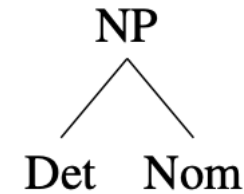
Derivation: sequence of rule expansions

Context-Free Grammar (CFG)

Rule:

Starting from the symbol: NP
we can rewrite NP as: Det* Nominal**

NP $\rightarrow$ Det Nom



* **Det**: word which is used at the beginning of a noun group. E.g., 'a,' 'the,' 'some,' 'this,' and 'each.'

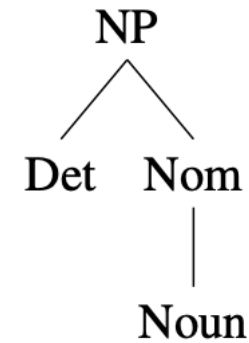
** **Nominal**: describes words or groups of words that function together as a noun

Context-Free Grammar (CFG)

Rule:

Starting from the symbol: NP
we can rewrite NP as: Det Nominal
and then rewrite Nominal as: Noun

$NP \rightarrow Det\ Nom$

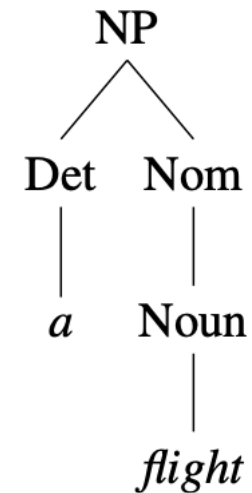


Context-Free Grammar (CFG)

Rule:

Starting from the symbol: NP
we can rewrite NP as: Det Nominal
and then rewrite Nominal as: Noun
and finally rewrite these parts-of-speech as: *a flight*

NP $\rightarrow$ Det Nom



Context-Free Grammar (CFG)

Rule:

A sentence can consist of a noun phrase followed by a verb phrase

$$S \rightarrow NP VP$$

I prefer a morning flight

Context-Free Grammar (CFG)

Rule:

A sentence can consist of a noun phrase followed by a verb phrase

$$S \rightarrow NP VP$$

A verb phrase consists of a verb followed by assorted other things

$$VP \rightarrow Verb NP$$

prefer a morning flight

Context-Free Grammar (CFG)

Rule:

A sentence can consist of a noun phrase followed by a verb phrase

$$S \rightarrow NP VP$$

A verb phrase consists of a verb followed by assorted other things

$$VP \rightarrow Verb NP$$

A verb may be followed by a noun phrase and a prepositional phrase

$$Verb \rightarrow Verb NP PP^*$$

leave Boston in the morning

Context-Free Grammar (CFG)

Rule:

A sentence can consist of a noun phrase followed by a verb phrase

$$S \rightarrow NP VP$$

A verb phrase consists of a verb followed by assorted other things

$$VP \rightarrow Verb NP$$

A verb may be followed by a noun phrase and a prepositional phrase

$$Verb \rightarrow Verb NP PP$$

A verb phrase may have a verb followed by a prepositional phrase alone

$$VP \rightarrow Verb PP$$

leaving on Thursday

Context-Free Grammar (CFG)

Rule:

A sentence can consist of a noun phrase followed by a verb phrase

$$S \rightarrow NP VP$$

A verb phrase consists of a verb followed by assorted other things

$$VP \rightarrow Verb NP$$

A verb may be followed by a noun phrase and a prepositional phrase

$$Verb \rightarrow Verb NP PP$$

A verb phrase may have a verb followed by a prepositional phrase alone

$$VP \rightarrow Verb PP$$

A prepositional phrase generally has a preposition followed by a noun phrase

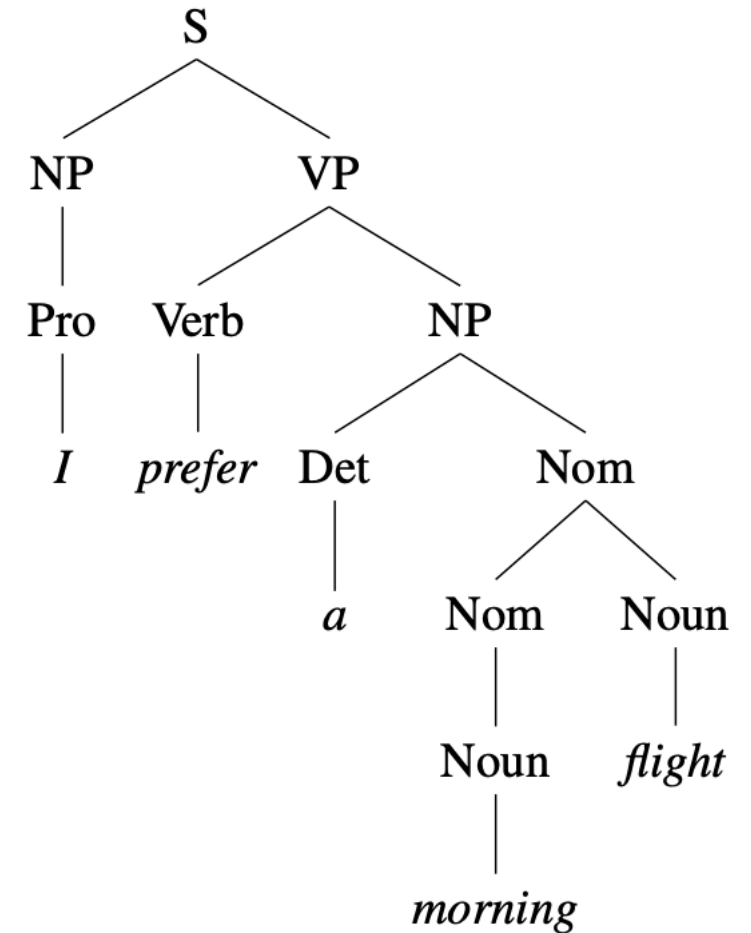
$$PP \rightarrow Preposition NP$$

from Los Angeles

Context-Free Grammar (CFG)

Representation

[S [NP [Pro I]] [VP [V prefer] [NP [Det a] [Nom[Nom [N morning]] [N flight]]]]]



Context-Free Grammar (CFG)

Formal Definition:

A context-free grammar G is defined by four parameters: N , Σ , R , S (“4-tuple”).

N a set of **non-terminal symbols** (or **variables**)

Σ a set of **terminal symbols** (disjoint from N)

R a set of **rules** or productions, each of the form $A \rightarrow \beta$,
where A is a non-terminal,

β is a string of symbols from the infinite set of strings $(\Sigma \cup N)^*$

S a designated **start symbol** and a member of N

Context-Free Grammar (CFG)

Grammar vs. Lexicon

Grammar:

- Production Rules
- use Non-terminal symbols (placeholders like S, NP, VP) to show the structure

Example: $S \rightarrow NP VP$ (a sentence consists of a Noun Phrase followed by a Verb Phrase)

Lexicon:

- set of Terminal symbols
- represented by rules that lead directly to a word

Example: $Det \rightarrow \text{"the"} \mid \text{"a"}$

Context-Free Grammar (CFG)

Grammar	Lexicon
$S \rightarrow NP VP$	$Det \rightarrow that \mid this \mid the \mid a$
$S \rightarrow Aux NP VP$	$Noun \rightarrow book \mid flight \mid meal \mid money$
$S \rightarrow VP$	$Verb \rightarrow book \mid include \mid prefer$
$NP \rightarrow Pronoun$	$Pronoun \rightarrow I \mid she \mid me$
$NP \rightarrow Proper-Noun$	$Proper-Noun \rightarrow Houston \mid NWA$
$NP \rightarrow Det Nominal$	$Aux \rightarrow does$
$Nominal \rightarrow Noun$	$Preposition \rightarrow from \mid to \mid on \mid near \mid through$
$Nominal \rightarrow Nominal Noun$	
$Nominal \rightarrow Nominal PP$	
$VP \rightarrow Verb$	
$VP \rightarrow Verb NP$	
$VP \rightarrow Verb NP PP$	
$VP \rightarrow Verb PP$	
$VP \rightarrow VP PP$	
$PP \rightarrow Preposition NP$	

Figure 17.8 The $\mathcal{L}_1$ miniature English grammar and lexicon.

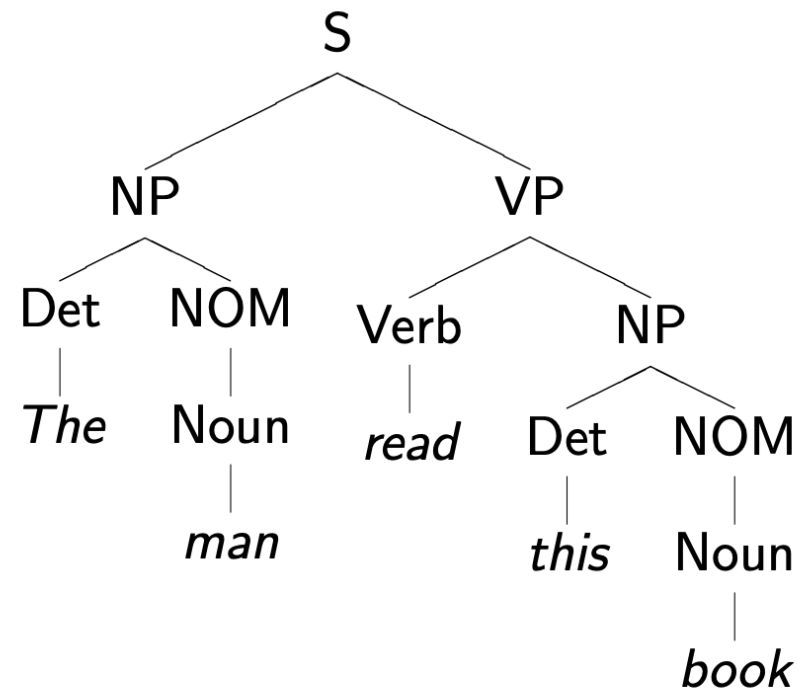
Context-Free Grammar (CFG)

Grammar / Lexicon: The interaction

Derivation

Given a CFG, a derivation is the sequence of productions used to generate a string of words/terminal symbols (e.g., a sentence), often visualized as a parse tree.

$S \rightarrow NP VP$
 $\rightarrow \text{Det NOM VP}$
 $\rightarrow \textit{The} \text{ NOM VP}$
 $\rightarrow \textit{The} \text{ Noun VP}$
 $\rightarrow \textit{The man} \text{ VP}$
 $\rightarrow \textit{The man} \text{ Verb NP}$
 $\rightarrow \textit{The man read} \text{ NP}$
 $\rightarrow \textit{The man read} \text{ Det NOM}$
 $\rightarrow \textit{The man read this} \text{ NOM}$
 $\rightarrow \textit{The man read this} \text{ Noun}$
 $\rightarrow \textit{The man read this book}$



Context-Free Grammar (CFG)

Exercise:

“The man read this book”

$$G = \langle T, N, S, R \rangle$$

$$T = \{that, this, a, the, man, book, flight, meal, include, read, does\}$$

$$N = \{S, NP, NOM, VP, Det, Noun, Verb, Aux\}$$

$$S = S$$

$$R = \{$$

$$S \rightarrow NP VP$$

$$S \rightarrow Aux NP VP$$

$$S \rightarrow VP$$

$$NP \rightarrow Det NOM$$

$$NOM \rightarrow Noun$$

$$NOM \rightarrow Noun NOM$$

$$VP \rightarrow Verb$$

$$VP \rightarrow Verb NP$$

$$Det \rightarrow that \mid this \mid a \mid the$$

$$Noun \rightarrow book \mid flight \mid meal \mid man$$

$$Verb \rightarrow book \mid include \mid read$$

$$Aux \rightarrow does$$

$$\}$$

Context-Free Grammar (CFG)

Exercise:

$S \rightarrow NP VP$

$\rightarrow Det NOM VP$

$\rightarrow The NOM VP$

$\rightarrow The Noun VP$

$\rightarrow The man VP$

$\rightarrow The man Verb NP$

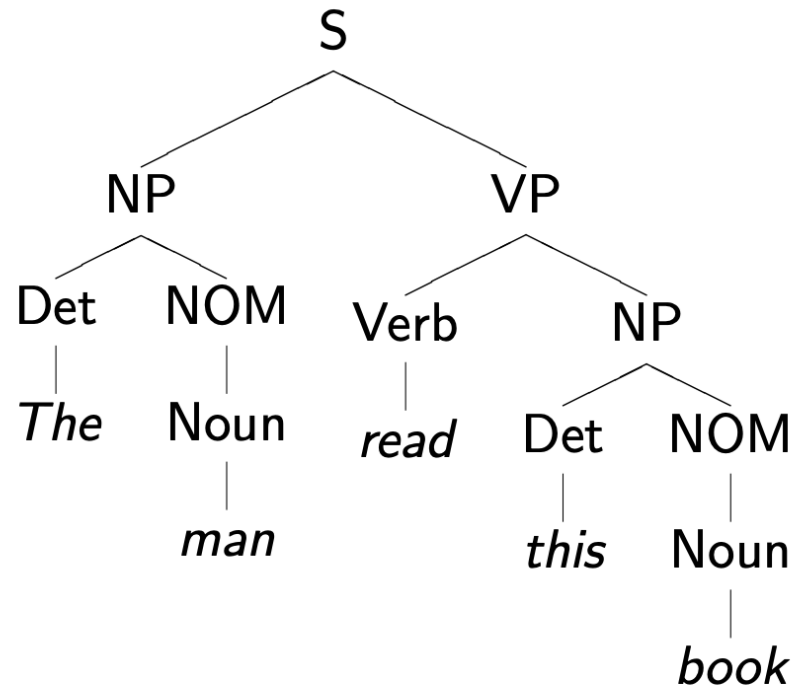
$\rightarrow The man read NP$

$\rightarrow The man read Det NOM$

$\rightarrow The man read this NOM$

$\rightarrow The man read this Noun$

$\rightarrow The man read this book$



$G = \langle T, N, S, R \rangle$

$T = \{that, this, a, the, man, book, flight, meal, include, read, does\}$

$N = \{S, NP, NOM, VP, Det, Noun, Verb, Aux\}$

$S = S$

$R = \{$

$S \rightarrow NP VP$

$S \rightarrow Aux NP VP$

$S \rightarrow VP$

$NP \rightarrow Det NOM$

$NOM \rightarrow Noun$

$NOM \rightarrow Noun NOM$

$VP \rightarrow Verb$

$VP \rightarrow Verb NP$

$\}$

$Det \rightarrow that | this | a | the$

$Noun \rightarrow book | flight | meal | man$

$Verb \rightarrow book | include | read$

$Aux \rightarrow does$

Context-Free Grammar (CFG)

Why called Context-Free?

- the rules/productions in a CFG apply regardless of the surrounding symbols or context

$S \rightarrow NP VP$

$NP \rightarrow Det N$

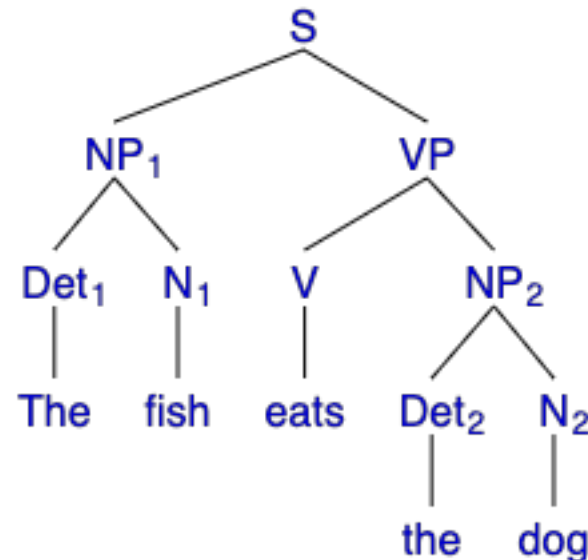
$VP \rightarrow V NP$

$Det \rightarrow "the" \mid "a"$

$N \rightarrow "cat" \mid "dog" \mid "fish"$

$V \rightarrow "eats" \mid "chases"$

"The fish eats the dog."



Parsing

Now back to parsing ...

To a computer, a sentence is just a long string of characters. **Parsing** is the process of analyzing that string to determine if it conforms to the rules of a specific **grammar**.

Input: A sequence of tokens from the **lexicon** (e.g., "The", "robot", "paints").

The Processor: The **Parser**

The Reference: The **Grammar**

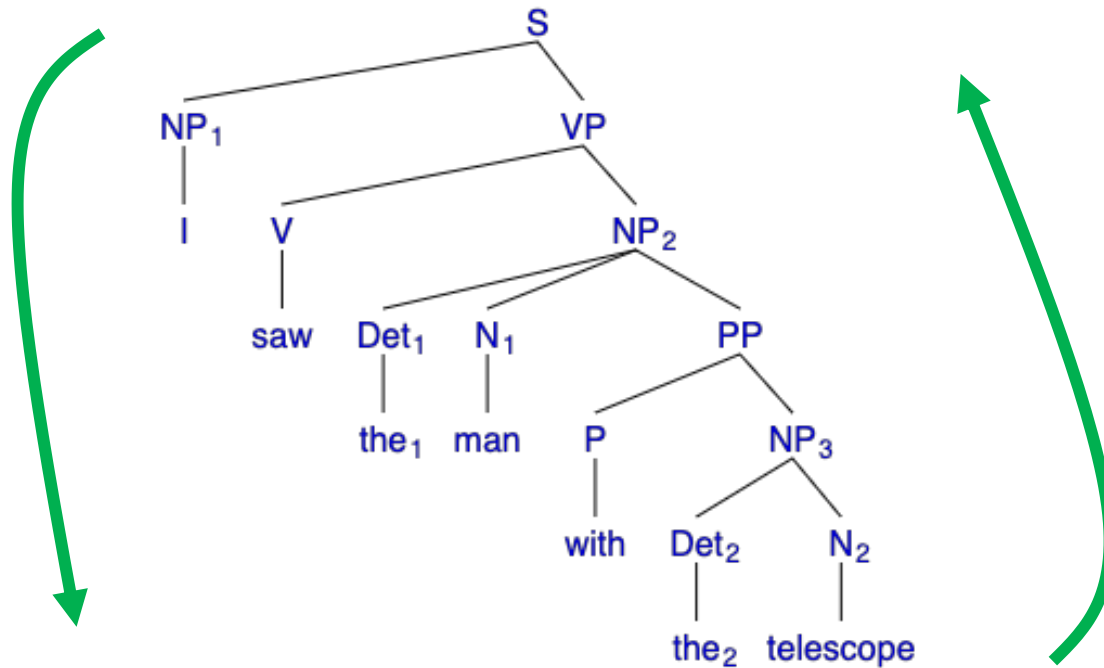
Output: A structured representation, usually a **Parse Tree**.

Parsing

- **Types:**
 - **Syntactic Parsing**
 - **Constituency Parsing**
 - **Dependency Parsing**
 - Semantic Parsing

Constituency Parsing: *Strategies*

- Top-Down



- Bottom-Up

Top-Down Parsing

- Start with the highest-level non-terminal symbol (usually the entire sentence shown with **s**)
- Find the tops of all trees that can start with s.
 - *look for production rules with s on the left side*
- Find the tops of all trees the constituents
- Repeat till reaching terminal nodes
- *Trees that their leaves fail to match ALL words in the sentence are rejected.*

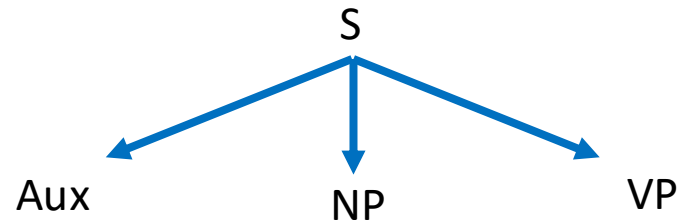
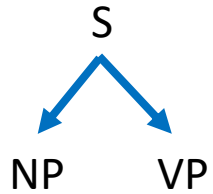
Top-Down Parsing

Example: Book that flight.

S	→ NP VP		
S	→ Aux NP VP	Det	→ that this a
S	→ VP	Noun	→ book flight meal money
NP	→ Pronoun	Verb	→ book include prefer
NP	→ Proper-Noun	Pronoun	→ I she me
NP	→ Det Nominal	Proper-Noun	→ Houston NWA
Nominal	→ Noun	Aux	→ does
Nominal	→ Nominal Noun	Preposition	→ from to on near through
Nominal	→ Nominal PP		
VP	→ Verb		
VP	→ Verb NP		
VP	→ Verb NP PP		
VP	→ Verb PP		
VP	→ VP PP		
PP	→ Preposition NP		

Top-Down Parsing

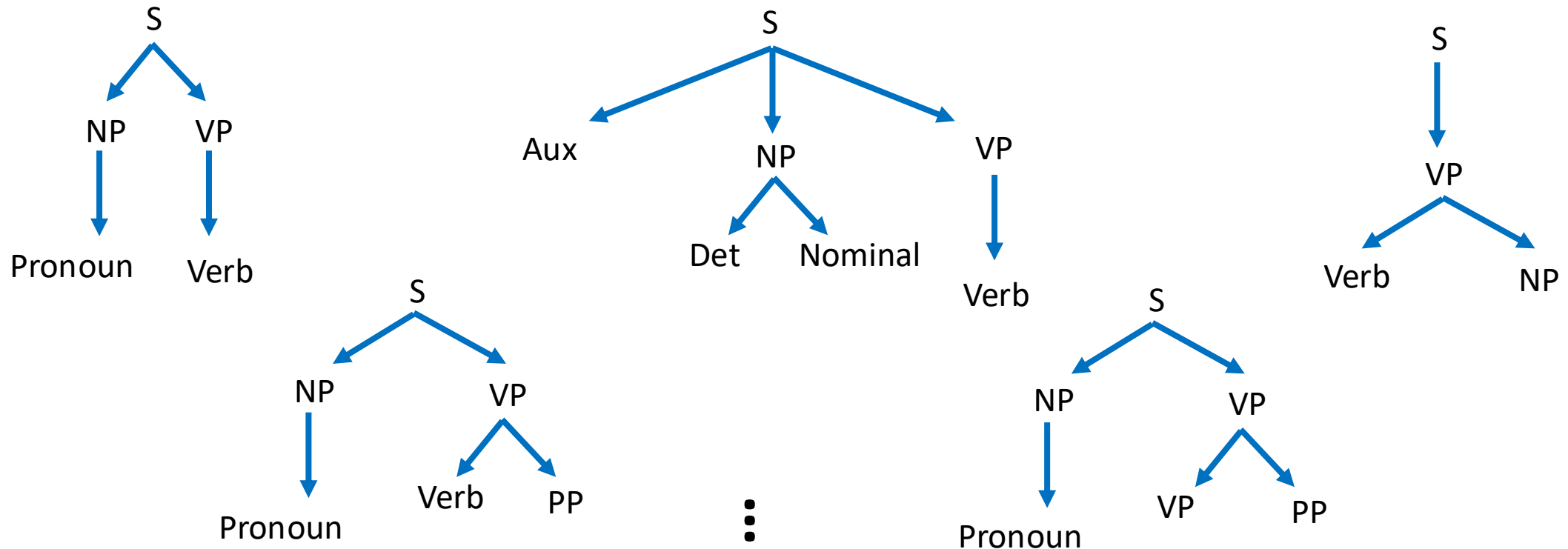
Example: Book that flight.



S → NP VP
S → Aux NP VP
S → VP

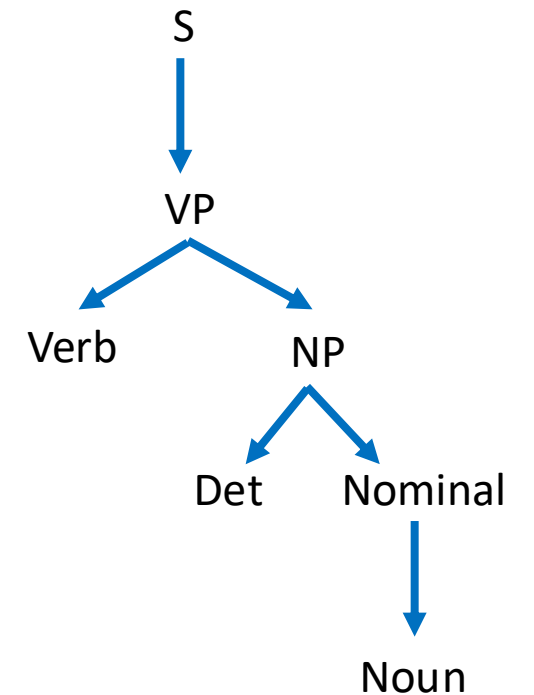
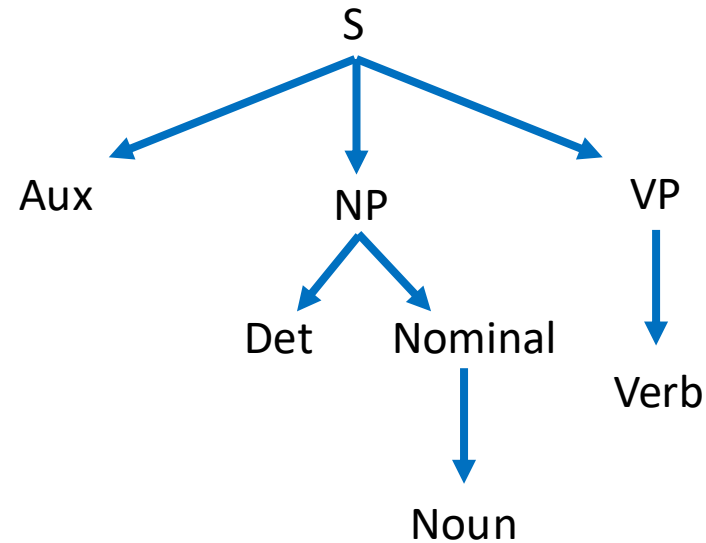
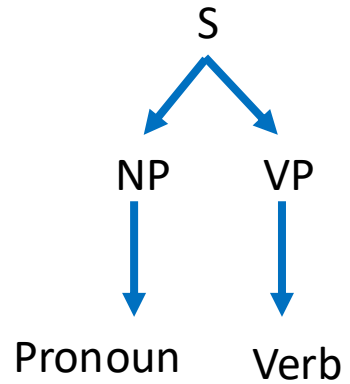
Top-Down Parsing

Example: Book that flight.



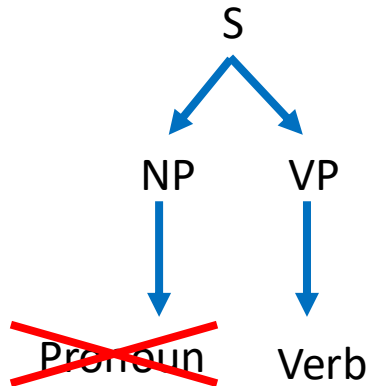
Top-Down Parsing

Example: Book that flight.



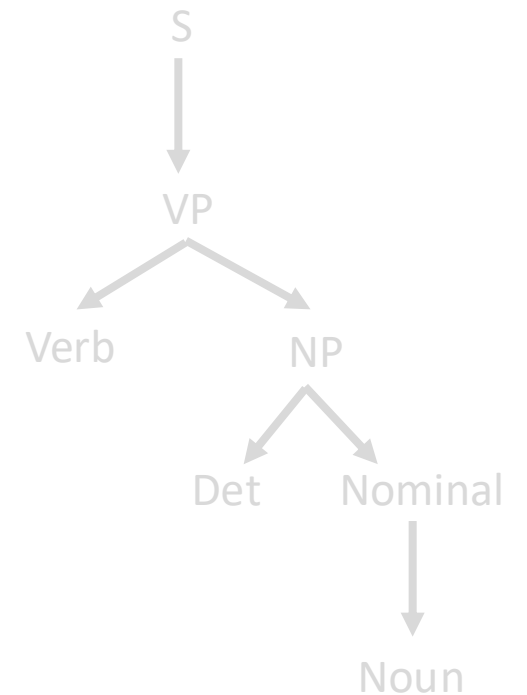
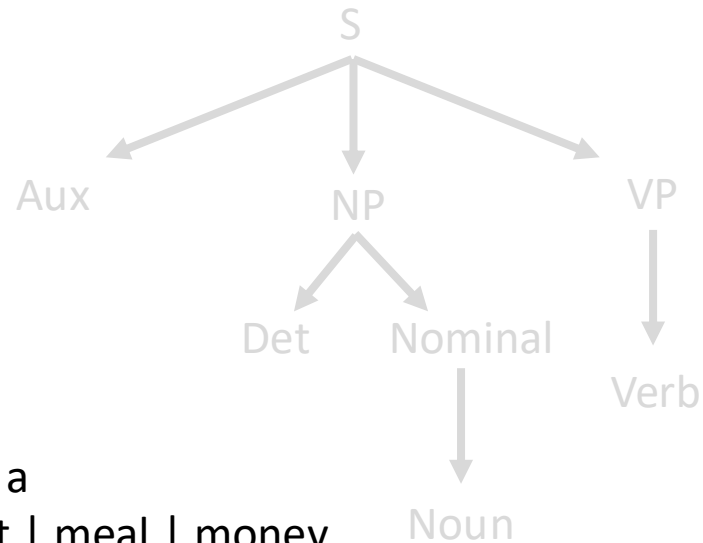
Top-Down Parsing

Example: Book that flight.



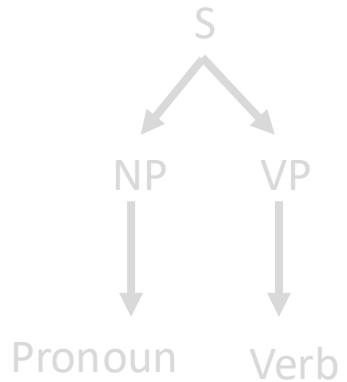
Det
Noun
Verb
Pronoun
Proper-Noun
Aux
Preposition

→ that | this | a
→ book | flight | meal | money
→ book | include | prefer
→ I | she | me
→ Houston | NWA
→ does
→ from | to | on | near | through



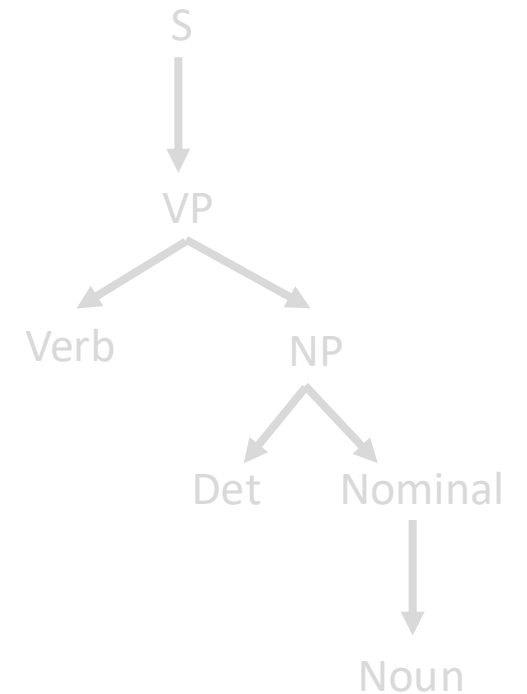
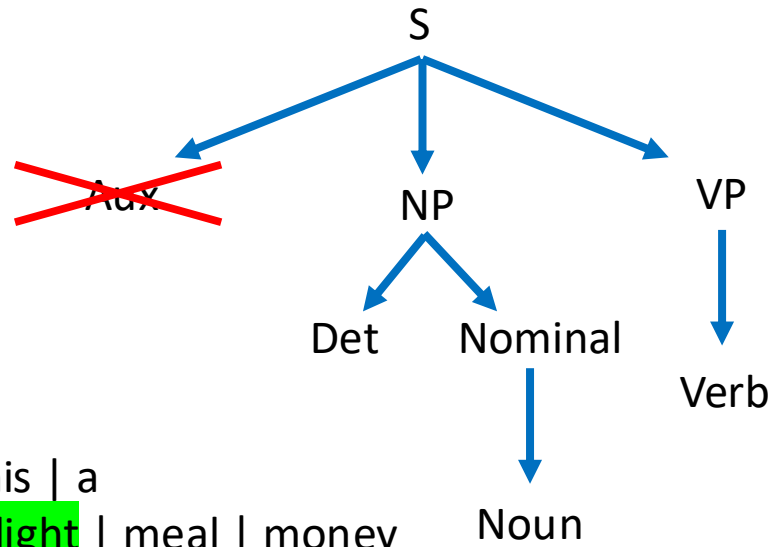
Top-Down Parsing

Example: Book that flight.



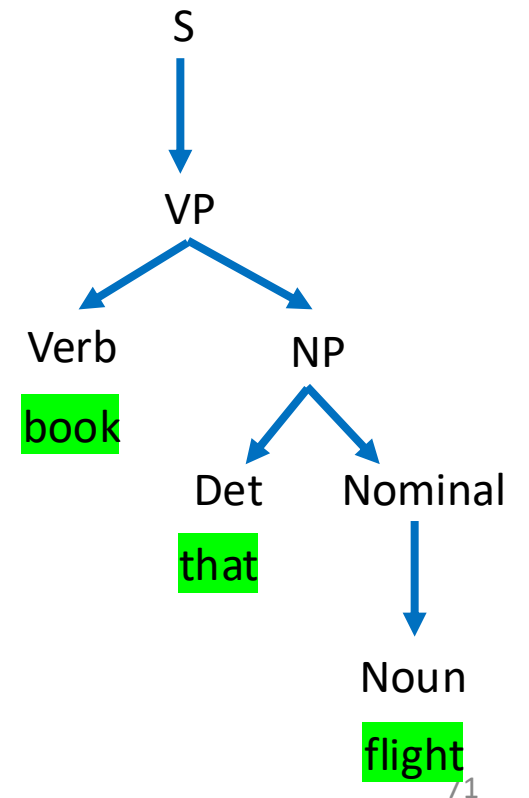
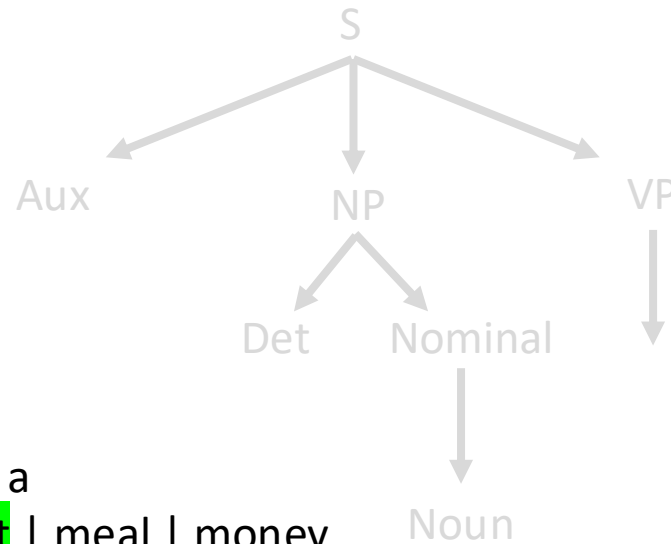
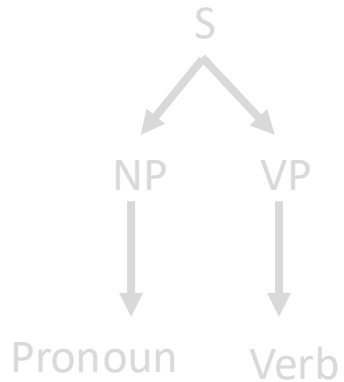
Det
Noun
Verb
Pronoun
Proper-Noun
Aux
Preposition

→ that | this | a
→ book | flight | meal | money
→ book | include | prefer
→ I | she | me
→ Houston | NWA
→ does
→ from | to | on | near | through



Top-Down Parsing

Example: Book that flight.



Det	→ that this a
Noun	→ book flight meal money
Verb	→ book include prefer
Pronoun	→ I she me
Proper-Noun	→ Houston NWA
Aux	→ does
Preposition	→ from to on near through

Bottom-Up Parsing

- Start with the individual words in the sentence
- Work way up, applying production rules to combine words into larger constituents.
 - Look for places where the right-hand side of a production may match.
- Success: reaching to start symbol **s**

Bottom-Up Parsing

Example: Book that flight.

S	→ NP VP		
S	→ Aux NP VP	Det	→ that this a
S	→ VP	Noun	→ book flight meal money
NP	→ Pronoun	Verb	→ book include prefer
NP	→ Proper-Noun	Pronoun	→ I she me
NP	→ Det Nominal	Proper-Noun	→ Houston NWA
Nominal	→ Noun	Aux	→ does
Nominal	→ Nominal Noun	Preposition	→ from to on near through
Nominal	→ Nominal PP		
VP	→ Verb		
VP	→ Verb NP		
VP	→ Verb NP PP		
VP	→ Verb PP		
VP	→ VP PP		
PP	→ Preposition NP		

Bottom-Up Parsing

Example: Book that flight.

Det	→ that this a
Noun	→ book flight meal money
Verb	→ book include prefer
Pronoun	→ I she me
Proper-Noun	→ Houston NWA
Aux	→ does
Preposition	→ from to on near through

Noun
↓
book

Det
↓
that

Noun
↓
flight

Verb
↓
book

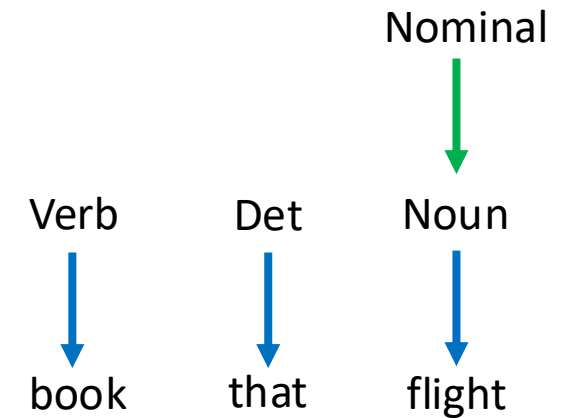
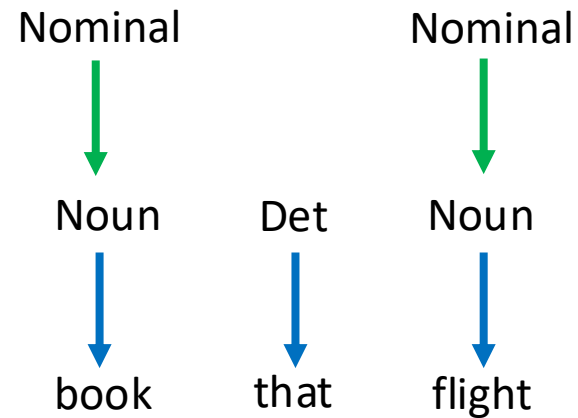
Det
↓
that

Noun
↓
flight

Bottom-Up Parsing

Example: Book that flight.

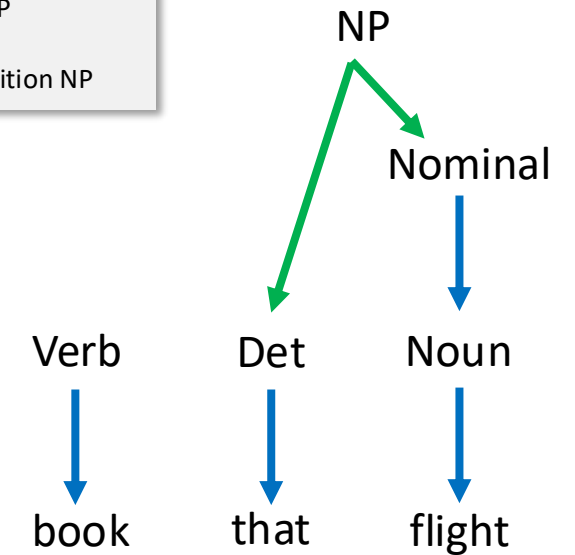
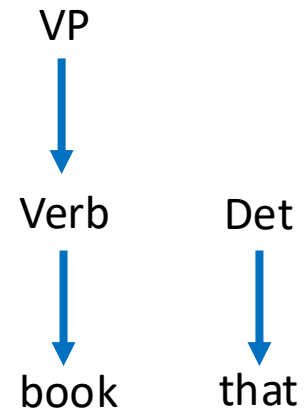
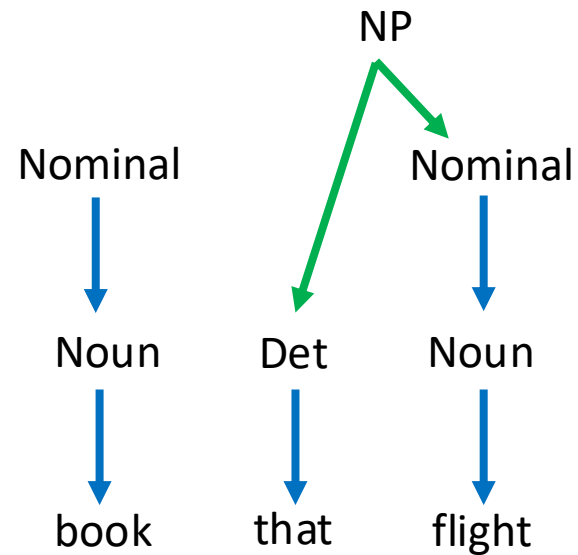
S	→ NP VP
S	→ Aux NP VP
S	→ VP
NP	→ Pronoun
NP	→ Proper-Noun
NP	→ Det Nominal
Nominal	→ Noun
Nominal	→ Nominal Noun
Nominal	→ Nominal PP
VP	→ Verb
VP	→ Verb NP
VP	→ Verb NP PP
VP	→ Verb PP
VP	→ VP PP
PP	→ Preposition NP



Bottom-Up Parsing

Example: Book that flight.

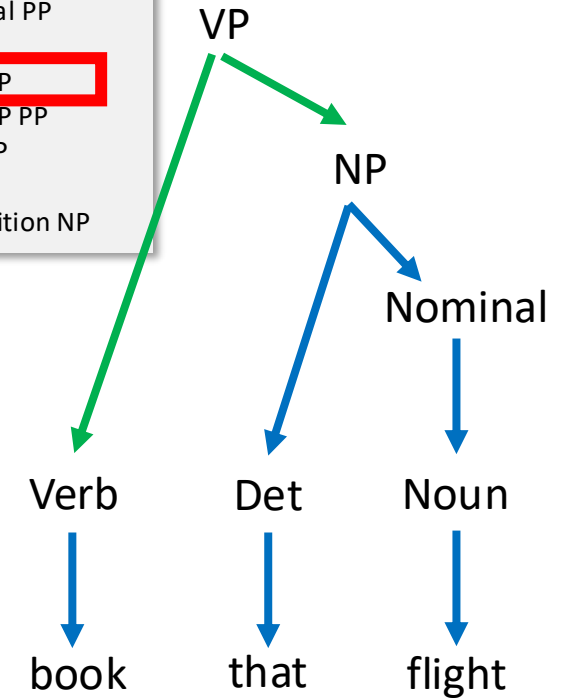
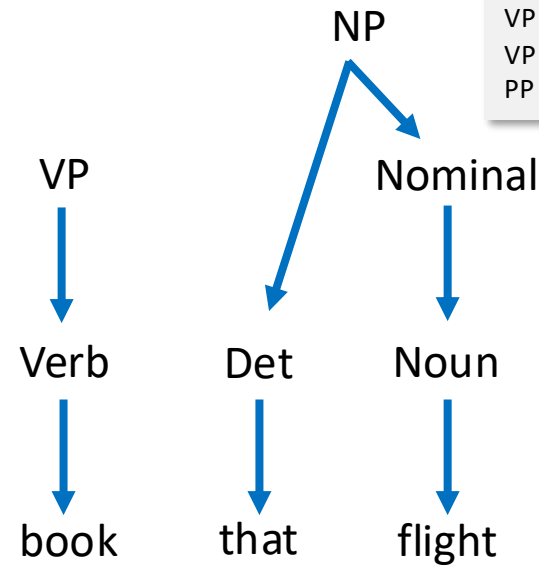
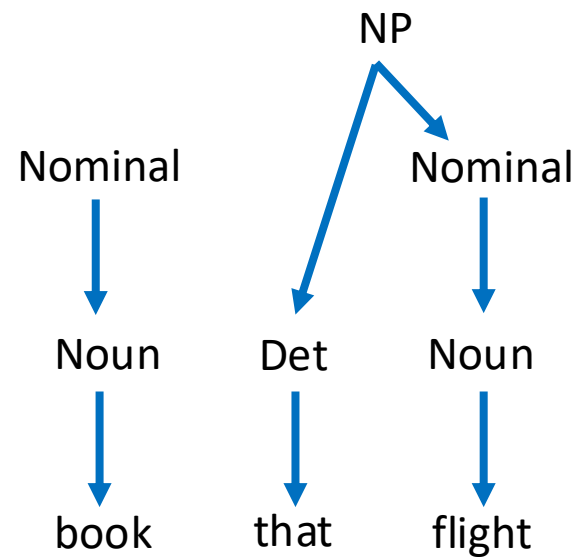
S	→ NP VP
S	→ Aux NP VP
S	→ VP
NP	→ Pronoun
NP	→ Proper-Noun
NP	→ Det Nominal
Nominal	→ Noun
Nominal	→ Nominal Noun
Nominal	→ Nominal PP
VP	→ Verb
VP	→ Verb NP
VP	→ Verb NP PP
VP	→ Verb PP
VP	→ VP PP
PP	→ Preposition NP



Bottom-Up Parsing

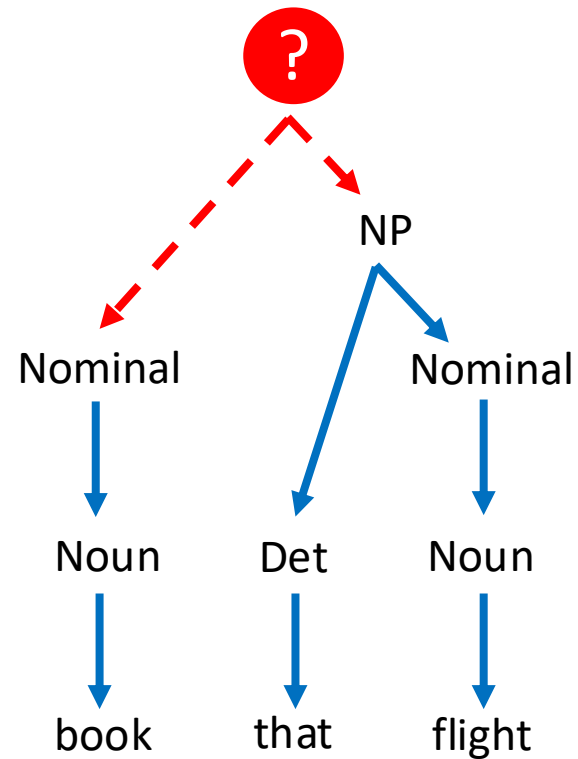
Example: Book that flight.

S	→ NP VP
S	→ Aux NP VP
S	→ VP
NP	→ Pronoun
NP	→ Proper-Noun
NP	→ Det Nominal
Nominal	→ Noun
Nominal	→ Nominal Noun
Nominal	→ Nominal PP
VP	→ Verb
VP	→ Verb NP
VP	→ Verb NP PP
VP	→ Verb PP
VP	→ VP PP
PP	→ Preposition NP

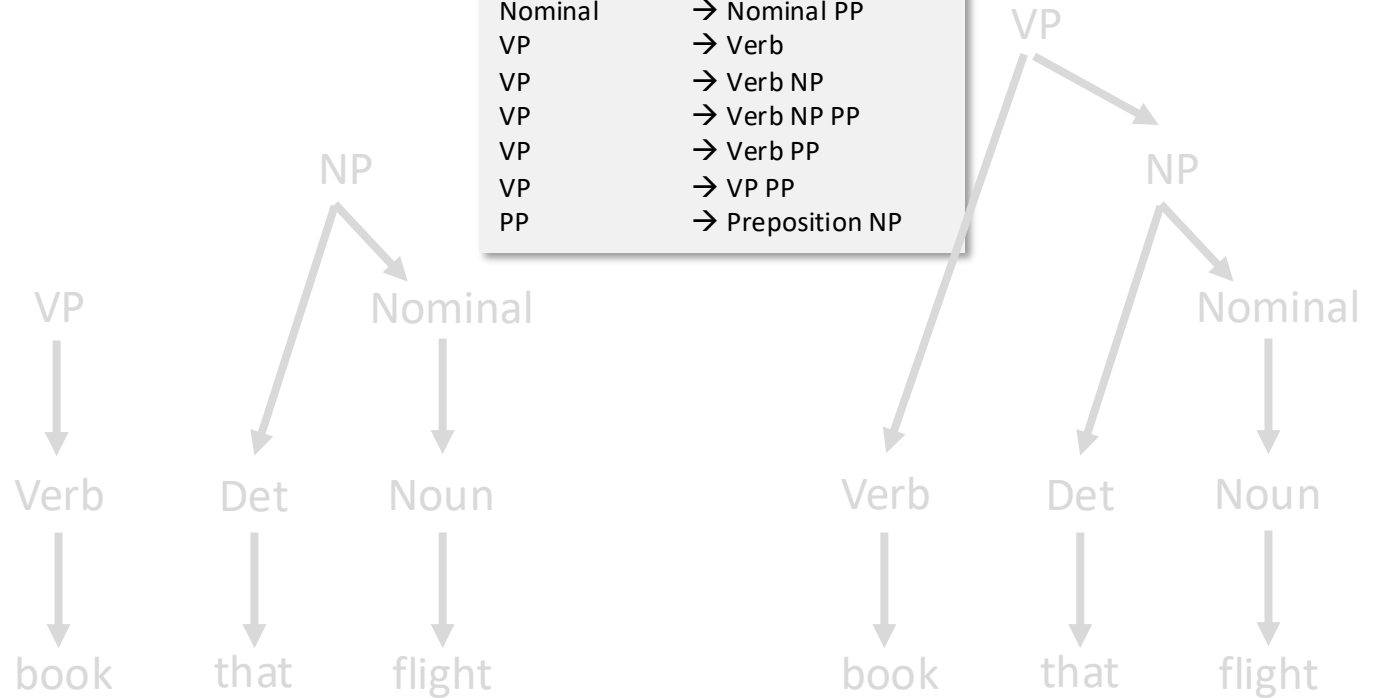


Bottom-Up Parsing

Example: Book that flight.

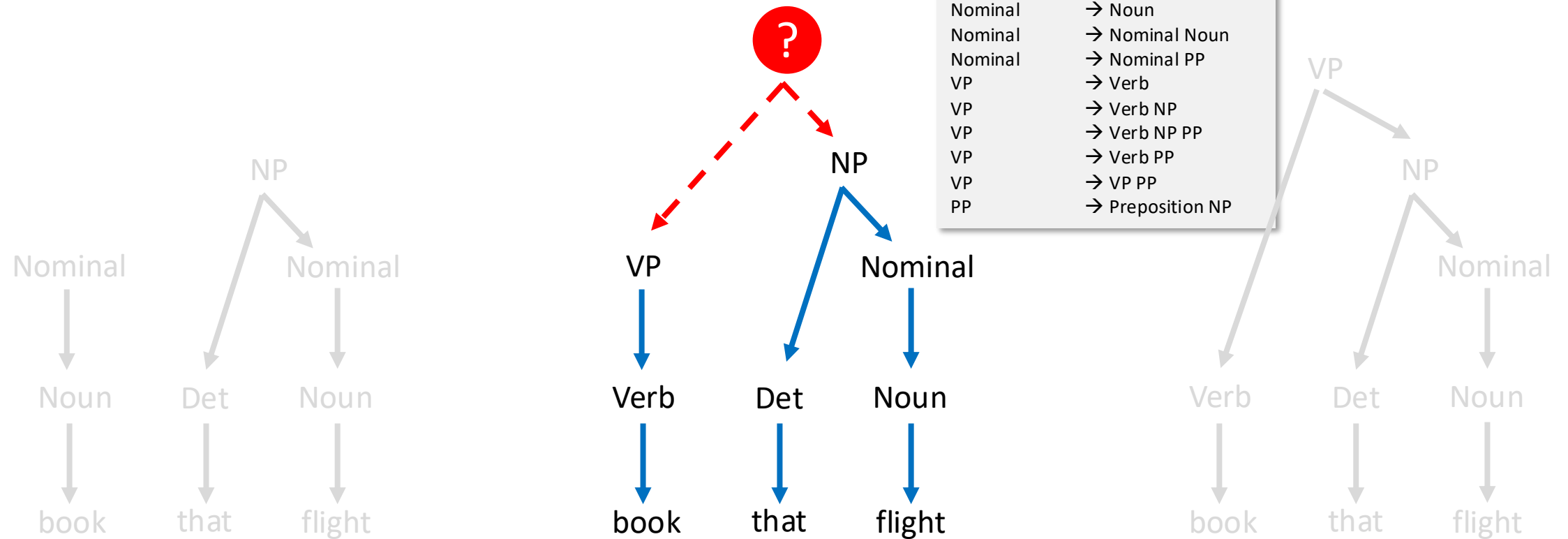


S	→ NP VP
S	→ Aux NP VP
S	→ VP
NP	→ Pronoun
NP	→ Proper-Noun
NP	→ Det Nominal
Nominal	→ Noun
Nominal	→ Nominal Noun
Nominal	→ Nominal PP
VP	→ Verb
VP	→ Verb NP
VP	→ Verb NP PP
VP	→ Verb PP
VP	→ VP PP
PP	→ Preposition NP



Bottom-Up Parsing

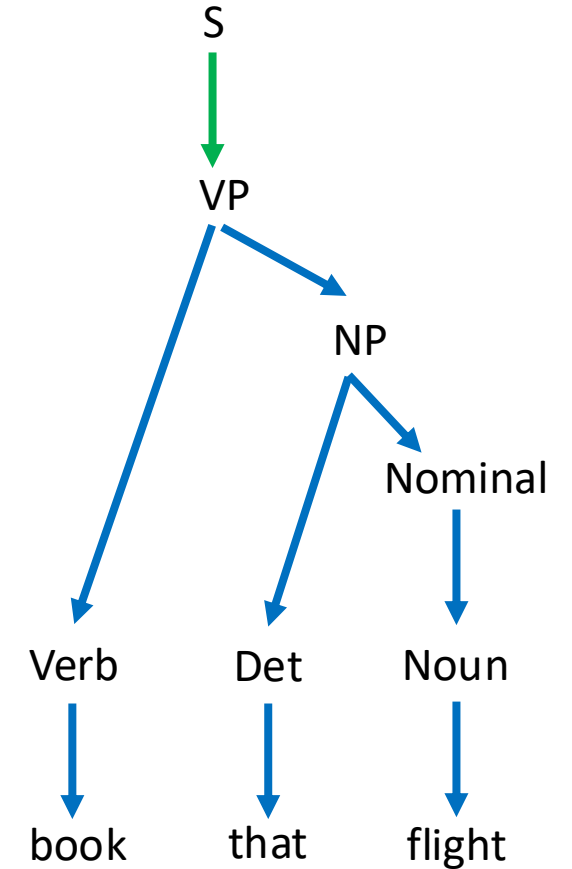
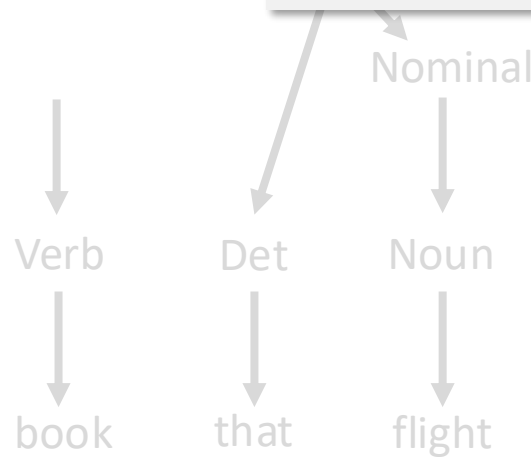
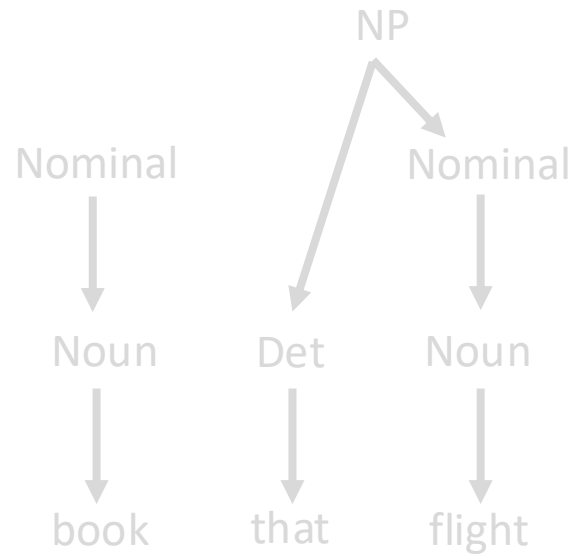
Example: Book that flight.



Bottom-Up Parsing

Example: Book that flight.

S	→ NP VP
S	→ Aux NP VP
S	→ VP
NP	→ Pronoun
NP	→ Proper-Noun
NP	→ Det Nominal
Nominal	→ Noun
Nominal	→ Nominal Noun
Nominal	→ Nominal PP
VP	→ Verb
VP	→ Verb NP
VP	→ Verb NP PP
VP	→ Verb PP
VP	→ VP PP
PP	→ Preposition NP



CYK/CKY Algorithm

Cocke-Younger-Kasami Algorithm (CYK/CKY Algorithm)

A bottom-up approach

A dynamic programming algorithm: partial solutions are stored and efficiently reused to find all possible parses for the entire sentence.

The CKY algorithm requires grammars to first be in Chomsky Normal Form (CNF).

Chomsky Normal Form (CNF)

N a set of **non-terminal symbols** (or **variables**)

Σ a set of **terminal symbols** (disjoint from N)

R a set of **rules** or productions, each of the form $A \rightarrow \beta$,
where A is a non-terminal,

β is a string of symbols from the infinite set of strings $(\Sigma \cup N)^*$

S a designated **start symbol** and a member of N

$A \rightarrow \beta ; \beta: (\Sigma \cup N)$ $\Rightarrow$

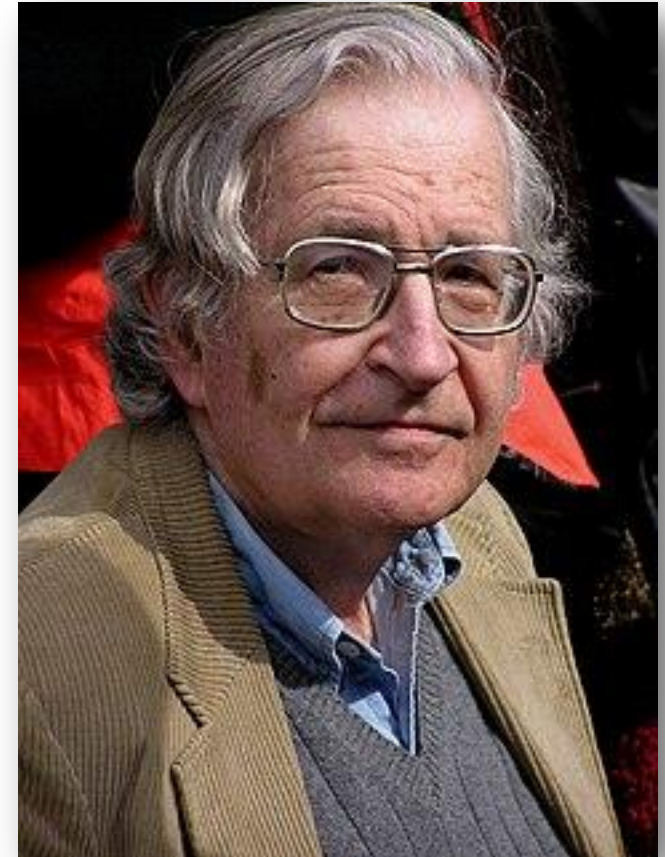
1. $A \rightarrow B C$, where A, B, C are *non-terminals*
2. $A \rightarrow s$, where A is a *non-terminal* s is a *terminal*

Chomsky Normal Form (CNF)

Noam Chomsky is a highly influential American linguist, philosopher, cognitive scientist, historian, social critic, and political activist. He was born on December 7, 1928, and is often considered the "father of modern linguistics".

“Language is a process of free creation; its laws and principles are fixed, but the manner in which the principles of generation are used is free and infinitely varied. Even the interpretation and use of words involves a process of free creation.

~ Noam Chomsky



wiki

Chomsky Normal Form (CNF)

$A \rightarrow B C$, where A, B, C are *non-terminals*

$A \rightarrow B C D \dots$ more than 2 non-terminals

Rewrite into:

$X_1 \rightarrow B C$

$A \rightarrow X_1 D \dots$

$\dots$

Chomsky Normal Form (CNF)

$A \rightarrow B C$, where A, B, C are *non-terminals*

$A \rightarrow B C D \dots$

$A \rightarrow s B$ mix of terminals and non-terminals

Rewrite into:

$X_1 \rightarrow B C$

$A \rightarrow X_1 D \dots$

...

Rewrite into:

$X_1 \rightarrow s$

$A \rightarrow X_1 B$

Chomsky Normal Form (CNF)

$A \rightarrow B C$, where A, B, C are *non-terminals*

$A \rightarrow B C D \dots$

$A \rightarrow s B$

$A \rightarrow B$ single nonterminal
 $B \rightarrow C$

Rewrite into:

$X_1 \rightarrow B C$
 $A \rightarrow X_1 D \dots$
...

Rewrite into:

$X_1 \rightarrow s$
 $A \rightarrow X_1 B$

Rewrite into:

$A \rightarrow C$

Chomsky Normal Form (CNF)

Example:

$S \rightarrow NP VP$	$DET \rightarrow \text{"an"} \mid \text{"my"}$
$VP \rightarrow VP PP$	$VBD \rightarrow \text{"shot"}$
$VP \rightarrow VBD NP$	$NNS \rightarrow \text{"pajamas"}$
$Nominal \rightarrow Nominal PP$	$NN \rightarrow \text{"elephant"}$
$Nominal \rightarrow NN$	$NP \rightarrow \text{"I"}$
$Nominal \rightarrow NNS$	$PRP \rightarrow \text{"I"}$
$PP \rightarrow IN NP$	$IN \rightarrow \text{"in"}$
$NP \rightarrow DET NP$	$PRP\$ \rightarrow \text{"my"}$
$NP \rightarrow Nominal$	
$NP \rightarrow PRP\$ Nominal$	

Chomsky Normal Form (CNF)

Example:

$S \rightarrow NP VP$

$VP \rightarrow VP PP$

$VP \rightarrow VBD NP$

$Nominal \rightarrow Nominal PP$

$Nominal \rightarrow NN$

$Nominal \rightarrow NNS$

$PP \rightarrow IN NP$

$NP \rightarrow DET NP$

$NP \rightarrow Nominal$

$NP \rightarrow PRP\$ Nominal$

$DET \rightarrow \text{"an"} \mid \text{"my"}$

$VBD \rightarrow \text{"shot"}$

$NNS \rightarrow \text{"pajamas"}$

$NN \rightarrow \text{"elephant"}$

$NP \rightarrow \text{"I"}$

$PRP \rightarrow \text{"I"}$

$IN \rightarrow \text{"in"}$

$PRP\$ \rightarrow \text{"my"}$

$A \rightarrow B$

$B \rightarrow C$

Rewrite into:

$A \rightarrow C$

Chomsky Normal Form (CNF)

Example:

$S \rightarrow NP VP$

$VP \rightarrow VP PP$

$VP \rightarrow VBD NP$

$Nominal \rightarrow Nominal PP$

$Nominal \rightarrow \text{"pajamas"}$
 $\quad \quad \quad | \text{"elephant"}$

$PP \rightarrow IN NP$

$NP \rightarrow DET NP$

$NP \rightarrow PRP\$ Nominal$

$DET \rightarrow \text{"an"} \mid \text{"my"}$

$VBD \rightarrow \text{"shot"}$

$NP \rightarrow \text{"elephant"}$
 $\quad \quad \quad | \text{"I"}$

$PRP \rightarrow \text{"I"}$

$IN \rightarrow \text{"in"}$

$PRP\$ \rightarrow \text{"my"}$

$A \rightarrow B$

$B \rightarrow C$

Rewrite into:

$A \rightarrow C$

CYK Parsing

Bottom-Up Dynamic Programming

function CKY-PARSE(*words*, *grammar*) **returns** *table*

for $j \leftarrow$ **from** 1 **to** LENGTH(*words*) **do**

for all $\{A \mid A \rightarrow \text{words}[j] \in \text{grammar}\}$

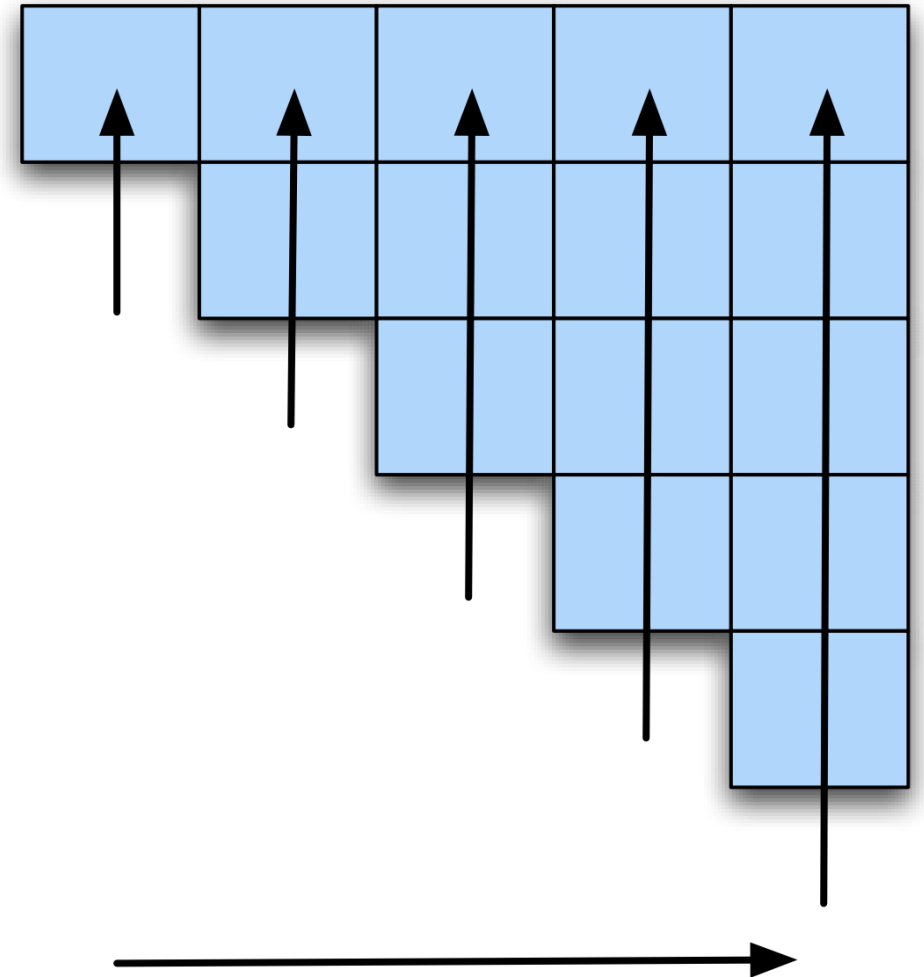
$\text{table}[j-1, j] \leftarrow \text{table}[j-1, j] \cup A$

for $i \leftarrow$ **from** $j-2$ **down to** 0 **do**

for $k \leftarrow i+1$ **to** $j-1$ **do**

for all $\{A \mid A \rightarrow BC \in \text{grammar} \text{ and } B \in \text{table}[i, k] \text{ and } C$

$\text{table}[i, j] \leftarrow \text{table}[i, j] \cup A$



CYK Parsing

$S \rightarrow NP VP$

$VP \rightarrow VP PP$

$VP \rightarrow VBD NP$

$Nominal \rightarrow Nominal PP$

$Nominal \rightarrow N$

$PP \rightarrow IN NP$

$NP \rightarrow DET Nominal$

$NP \rightarrow PRP\$ Nominal$

$NP \rightarrow PRP$

$DET \rightarrow \text{"an"}$

$VBD \rightarrow \text{"shot"}$

$N \rightarrow \text{"elephant"} \mid \text{"pajamas"}$

$PRP \rightarrow \text{"I"}$

$IN \rightarrow \text{"in"}$

$PRP\$ \rightarrow \text{"my"}$

I	shot	an	elephant	in	my	pajamas

CYK Parsing

$S \rightarrow NP VP$

$VP \rightarrow VP PP$

$VP \rightarrow VBD NP$

$Nominal \rightarrow Nominal PP$

$Nominal \rightarrow N$

$PP \rightarrow IN NP$

$NP \rightarrow DET Nominal$

$NP \rightarrow PRP\$ Nominal$

$NP \rightarrow PRP$

$DET \rightarrow \text{"an"}$

$VBD \rightarrow \text{"shot"}$

$N \rightarrow \text{"elephant"} \mid \text{"pajamas"}$

$PRP \rightarrow \text{"I"}$

$IN \rightarrow \text{"in"}$

$PRP\$ \rightarrow \text{"my"}$

I	shot	an	elephant	in	my	pajamas
PRP						

CYK Parsing

$S \rightarrow NP VP$

$VP \rightarrow VP PP$

$VP \rightarrow VBD NP$

$Nominal \rightarrow Nominal PP$

$Nominal \rightarrow N$

$PP \rightarrow IN NP$

$NP \rightarrow DET Nominal$

$NP \rightarrow PRP\$ Nominal$

$NP \rightarrow PRP$

$DET \rightarrow \text{"an"}$

$VBD \rightarrow \text{"shot"}$

$N \rightarrow \text{"elephant"} \mid \text{"pajamas"}$

$PRP \rightarrow \text{"I"}$

$IN \rightarrow \text{"in"}$

$PRP\$ \rightarrow \text{"my"}$

I	shot	an	elephant	in	my	pajamas
PRP						

CYK Parsing

$S \rightarrow NP VP$

$VP \rightarrow VP PP$

$VP \rightarrow VBD NP$

$Nominal \rightarrow Nominal PP$

$Nominal \rightarrow N$

$PP \rightarrow IN NP$

$NP \rightarrow DET Nominal$

$NP \rightarrow PRP\$ Nominal$

$NP \rightarrow PRP$

$DET \rightarrow \text{"an"}$

$VBD \rightarrow \text{"shot"}$

$N \rightarrow \text{"elephant"} \mid \text{"pajamas"}$

$PRP \rightarrow \text{"I"}$

$IN \rightarrow \text{"in"}$

$PRP\$ \rightarrow \text{"my"}$

I	shot	an	elephant	in	my	pajamas
PRP						
	VBD					

CYK Parsing

$S \rightarrow NP VP$

$VP \rightarrow VP PP$

$VP \rightarrow VBD NP$

$Nominal \rightarrow Nominal PP$

$Nominal \rightarrow N$

$PP \rightarrow IN NP$

$NP \rightarrow DET Nominal$

$NP \rightarrow PRP\$ Nominal$

$NP \rightarrow PRP$

$DET \rightarrow \text{"an"}$

$VBD \rightarrow \text{"shot"}$

$N \rightarrow \text{"elephant"} \mid \text{"pajamas"}$

$PRP \rightarrow \text{"I"}$

$IN \rightarrow \text{"in"}$

$PRP\$ \rightarrow \text{"my"}$

I shot an elephant in my pajamas						
PRP						
	VBD					

CYK Parsing

$S \rightarrow NP VP$

$VP \rightarrow VP PP$

$VP \rightarrow VBD NP$

$Nominal \rightarrow Nominal PP$

$Nominal \rightarrow N$

$PP \rightarrow IN NP$

$NP \rightarrow DET Nominal$

$NP \rightarrow PRP\$ Nominal$

$NP \rightarrow PRP$

$DET \rightarrow \text{"an"}$

$VBD \rightarrow \text{"shot"}$

$N \rightarrow \text{"elephant"} \mid \text{"pajamas"}$

$PRP \rightarrow \text{"I"}$

$IN \rightarrow \text{"in"}$

$PRP\$ \rightarrow \text{"my"}$

I		shot	an	elephant	in	my	pajamas
PRP							
	VBD						

PRP VBD (?)

CYK Parsing

$S \rightarrow NP VP$

$VP \rightarrow VP PP$

$VP \rightarrow VBD NP$

$Nominal \rightarrow Nominal PP$

$Nominal \rightarrow N$

$PP \rightarrow IN NP$

$NP \rightarrow DET Nominal$

$NP \rightarrow PRP\$ Nominal$

$NP \rightarrow PRP$

$DET \rightarrow \text{"an"}$

$VBD \rightarrow \text{"shot"}$

$N \rightarrow \text{"elephant"} \mid \text{"pajamas"}$

$PRP \rightarrow \text{"I"}$

$IN \rightarrow \text{"in"}$

$PRP\$ \rightarrow \text{"my"}$

	I	shot	an	elephant	in	my	pajamas
PRP	.						
	VBD						

~~PRP VBD~~

CYK Parsing

$S \rightarrow NP VP$

$VP \rightarrow VP PP$

$VP \rightarrow VBD NP$

$Nominal \rightarrow Nominal PP$

$Nominal \rightarrow N$

$PP \rightarrow IN NP$

$NP \rightarrow DET Nominal$

$NP \rightarrow PRP\$ Nominal$

$NP \rightarrow PRP$

DET $\rightarrow$ "an"

VBD $\rightarrow$ "shot"

N $\rightarrow$ "elephant" | "pajamas"

PRP $\rightarrow$ "I"

IN $\rightarrow$ "in"

PRP\$ $\rightarrow$ "my"

I	shot	an	elephant	in	my	pajamas
PRP	.					
	VBD					
		DET				

CYK Parsing

$S \rightarrow NP VP$

$VP \rightarrow VP PP$

$VP \rightarrow VBD NP$

$Nominal \rightarrow Nominal PP$

$Nominal \rightarrow N$

$PP \rightarrow IN NP$

$NP \rightarrow DET Nominal$

$NP \rightarrow PRP\$ Nominal$

$NP \rightarrow PRP$

$DET \rightarrow \text{"an"}$

$VBD \rightarrow \text{"shot"}$

$N \rightarrow \text{"elephant"} \mid \text{"pajamas"}$

$PRP \rightarrow \text{"I"}$

$IN \rightarrow \text{"in"}$

$PRP\$ \rightarrow \text{"my"}$

I	shot	an	elephant	in	my	pajamas
PRP	.					
	VBD					
		DET				

~~VBD-DET~~

CYK Parsing

$S \rightarrow NP VP$

$VP \rightarrow VP PP$

$VP \rightarrow VBD NP$

$Nominal \rightarrow Nominal PP$

$Nominal \rightarrow N$

$PP \rightarrow IN NP$

$NP \rightarrow DET Nominal$

$NP \rightarrow PRP\$ Nominal$

$NP \rightarrow PRP$

$DET \rightarrow \text{"an"}$

$VBD \rightarrow \text{"shot"}$

$N \rightarrow \text{"elephant"} \mid \text{"pajamas"}$

$PRP \rightarrow \text{"I"}$

$IN \rightarrow \text{"in"}$

$PRP\$ \rightarrow \text{"my"}$

I	shot	an	elephant	in	my	pajamas
PRP	.					
	VBD	.				
		DET				

~~VBD-DET~~

CYK Parsing

$S \rightarrow NP VP$

$VP \rightarrow VP PP$

$VP \rightarrow VBD NP$

$Nominal \rightarrow Nominal PP$

$Nominal \rightarrow N$

$PP \rightarrow IN NP$

$NP \rightarrow DET Nominal$

$NP \rightarrow PRP\$ Nominal$

$NP \rightarrow PRP$

$DET \rightarrow \text{"an"}$

$VBD \rightarrow \text{"shot"}$

$N \rightarrow \text{"elephant"} \mid \text{"pajamas"}$

$PRP \rightarrow \text{"I"}$

$IN \rightarrow \text{"in"}$

$PRP\$ \rightarrow \text{"my"}$

I shot an elephant in my pajamas						
PRP	.					
	VBD					
		DET				

CYK Parsing

$S \rightarrow NP VP$

$VP \rightarrow VP PP$

$VP \rightarrow VBD NP$

$Nominal \rightarrow Nominal PP$

$Nominal \rightarrow N$

$PP \rightarrow IN NP$

$NP \rightarrow DET Nominal$

$NP \rightarrow PRP\$ Nominal$

$NP \rightarrow PRP$

$DET \rightarrow \text{"an"}$

$VBD \rightarrow \text{"shot"}$

$N \rightarrow \text{"elephant"} \mid \text{"pajamas"}$

$PRP \rightarrow \text{"I"}$

$IN \rightarrow \text{"in"}$

$PRP\$ \rightarrow \text{"my"}$

I	shot	an	elephant	in	my	pajamas
PRP	.					
	VBD	.				
		DET				

CYK Parsing

$S \rightarrow NP VP$

$VP \rightarrow VP PP$

$VP \rightarrow VBD NP$

$Nominal \rightarrow Nominal PP$

$Nominal \rightarrow N$

$PP \rightarrow IN NP$

$NP \rightarrow DET Nominal$

$NP \rightarrow PRP\$ Nominal$

$NP \rightarrow PRP$

$DET \rightarrow \text{"an"}$

$VBD \rightarrow \text{"shot"}$

$N \rightarrow \text{"elephant"} \mid \text{"pajamas"}$

$PRP \rightarrow \text{"I"}$

$IN \rightarrow \text{"in"}$

$PRP\$ \rightarrow \text{"my"}$

	I	shot	an	elephant	in	my	pajamas
PRP	.						
	VBD	.					
		DET					

CYK Parsing

$S \rightarrow NP VP$

$VP \rightarrow VP PP$

$VP \rightarrow VBD NP$

$Nominal \rightarrow Nominal PP$

$Nominal \rightarrow N$

$PP \rightarrow IN NP$

$NP \rightarrow DET Nominal$

$NP \rightarrow PRP\$ Nominal$

$NP \rightarrow PRP$

$DET \rightarrow \text{"an"}$

$VBD \rightarrow \text{"shot"}$

$N \rightarrow \text{"elephant"} \mid \text{"pajamas"}$

$PRP \rightarrow \text{"I"}$

$IN \rightarrow \text{"in"}$

$PRP\$ \rightarrow \text{"my"}$

	I	shot	an	elephant	in	my	pajamas
PRP	.	.					
	VBD	.					
		DET					

CYK Parsing

$S \rightarrow NP VP$

$VP \rightarrow VP PP$

$VP \rightarrow VBD NP$

$Nominal \rightarrow Nominal PP$

$Nominal \rightarrow N$

$PP \rightarrow IN NP$

$NP \rightarrow DET Nominal$

$NP \rightarrow PRP\$ Nominal$

$NP \rightarrow PRP$

$DET \rightarrow \text{"an"}$

$VBD \rightarrow \text{"shot"}$

$N \rightarrow \text{"elephant"} \mid \text{"pajamas"}$

$PRP \rightarrow \text{"I"}$

$IN \rightarrow \text{"in"}$

$PRP\$ \rightarrow \text{"my"}$

I	shot	an	elephant	in	my	pajamas
PRP	.	.				
	VBD	.				
		DET				
			N			

CYK Parsing

$S \rightarrow NP VP$

$VP \rightarrow VP PP$

$VP \rightarrow VBD NP$

$Nominal \rightarrow Nominal PP$

$Nominal \rightarrow N$

$PP \rightarrow IN NP$

$NP \rightarrow DET Nominal$

$NP \rightarrow PRP\$ Nominal$

$NP \rightarrow PRP$

$DET \rightarrow \text{"an"}$

$VBD \rightarrow \text{"shot"}$

$N \rightarrow \text{"elephant"} \mid \text{"pajamas"}$

$PRP \rightarrow \text{"I"}$

$IN \rightarrow \text{"in"}$

$PRP\$ \rightarrow \text{"my"}$

I	shot	an	elephant	in	my	pajamas
PRP	.	.				
	VBD	.				
		DET				
			N			

CYK Parsing

$S \rightarrow NP VP$

$VP \rightarrow VP PP$

$VP \rightarrow VBD NP$

$Nominal \rightarrow Nominal PP$

$Nominal \rightarrow N$

$PP \rightarrow IN NP$

$NP \rightarrow DET Nominal$

$NP \rightarrow PRP\$ Nominal$

$NP \rightarrow PRP$

$DET \rightarrow \text{"an"}$

$VBD \rightarrow \text{"shot"}$

$N \rightarrow \text{"elephant"} \mid \text{"pajamas"}$

$PRP \rightarrow \text{"I"}$

$IN \rightarrow \text{"in"}$

$PRP\$ \rightarrow \text{"my"}$

I	shot	an	elephant	in	my	pajamas
PRP	.	.				
	VBD	.				
		DET				
			N			

CYK Parsing

$S \rightarrow NP VP$

$VP \rightarrow VP PP$

$VP \rightarrow VBD NP$

$Nominal \rightarrow Nominal PP$

$Nominal \rightarrow N$

$PP \rightarrow IN NP$

$NP \rightarrow DET Nominal$

$NP \rightarrow PRP\$ Nominal$

$NP \rightarrow PRP$

$DET \rightarrow \text{"an"}$

$VBD \rightarrow \text{"shot"}$

$N \rightarrow \text{"elephant"} \mid \text{"pajamas"}$

$PRP \rightarrow \text{"I"}$

$IN \rightarrow \text{"in"}$

$PRP\$ \rightarrow \text{"my"}$

I	shot	an	elephant	in	my	pajamas
PRP	.	.				
	VBD	.				
		DET	NP			
			N			

CYK Parsing

$S \rightarrow NP VP$

$VP \rightarrow VP PP$

$VP \rightarrow VBD NP$

$Nominal \rightarrow Nominal PP$

$Nominal \rightarrow N$

$PP \rightarrow IN NP$

$NP \rightarrow DET Nominal$

$NP \rightarrow PRP\$ Nominal$

$NP \rightarrow PRP$

$DET \rightarrow \text{"an"}$

$VBD \rightarrow \text{"shot"}$

$N \rightarrow \text{"elephant"} \mid \text{"pajamas"}$

$PRP \rightarrow \text{"I"}$

$IN \rightarrow \text{"in"}$

$PRP\$ \rightarrow \text{"my"}$

I	shot	an	elephant	in	my	pajamas
PRP	.	.				
	VBD	.				
		DET	NP			
			N			

CYK Parsing

$S \rightarrow NP VP$

$VP \rightarrow VP PP$

$VP \rightarrow VBD NP$

$Nominal \rightarrow Nominal PP$

$Nominal \rightarrow N$

$PP \rightarrow IN NP$

$NP \rightarrow DET Nominal$

$NP \rightarrow PRP\$ Nominal$

$NP \rightarrow PRP$

$DET \rightarrow \text{"an"}$

$VBD \rightarrow \text{"shot"}$

$N \rightarrow \text{"elephant"} \mid \text{"pajamas"}$

$PRP \rightarrow \text{"I"}$

$IN \rightarrow \text{"in"}$

$PRP\$ \rightarrow \text{"my"}$

I	shot	an	elephant	in	my	pajamas
PRP	.	.				
	VBD	.				
		DET	NP			
			N			

CYK Parsing

$S \rightarrow NP VP$

$VP \rightarrow VP PP$

$VP \rightarrow VBD NP$

$Nominal \rightarrow Nominal PP$

$Nominal \rightarrow N$

$PP \rightarrow IN NP$

$NP \rightarrow DET Nominal$

$NP \rightarrow PRP\$ Nominal$

$NP \rightarrow PRP$

$DET \rightarrow \text{"an"}$

$VBD \rightarrow \text{"shot"}$

$N \rightarrow \text{"elephant"} \mid \text{"pajamas"}$

$PRP \rightarrow \text{"I"}$

$IN \rightarrow \text{"in"}$

$PRP\$ \rightarrow \text{"my"}$

I	shot	an	elephant	in	my	pajamas
PRP	.	.				
	VBD	.				
		DET	NP			
			N			

CYK Parsing

$S \rightarrow NP VP$

$VP \rightarrow VP PP$

$VP \rightarrow VBD NP$

$Nominal \rightarrow Nominal PP$

$Nominal \rightarrow N$

$PP \rightarrow IN NP$

$NP \rightarrow DET Nominal$

$NP \rightarrow PRP\$ Nominal$

$NP \rightarrow PRP$

$DET \rightarrow \text{"an"}$

$VBD \rightarrow \text{"shot"}$

$N \rightarrow \text{"elephant"} \mid \text{"pajamas"}$

$PRP \rightarrow \text{"I"}$

$IN \rightarrow \text{"in"}$

$PRP\$ \rightarrow \text{"my"}$

I	shot	an	elephant	in	my	pajamas
PRP	.	.				
	VBD	.	VP			
		DET	NP			
			N			

CYK Parsing

$S \rightarrow NP VP$

$VP \rightarrow VP PP$

$VP \rightarrow VBD NP$

$Nominal \rightarrow Nominal PP$

$Nominal \rightarrow N$

$PP \rightarrow IN NP$

$NP \rightarrow DET Nominal$

$NP \rightarrow PRP\$ Nominal$

$NP \rightarrow PRP$

$DET \rightarrow \text{"an"}$

$VBD \rightarrow \text{"shot"}$

$N \rightarrow \text{"elephant"} \mid \text{"pajamas"}$

$PRP \rightarrow \text{"I"}$

$IN \rightarrow \text{"in"}$

$PRP\$ \rightarrow \text{"my"}$

I shot an elephant in my pajamas						
PRP	.	.				
	VBD	.	VP			
		DET	NP			
			N			

CYK Parsing

$S \rightarrow NP VP$

$VP \rightarrow VP PP$

$VP \rightarrow VBD NP$

$Nominal \rightarrow Nominal PP$

$Nominal \rightarrow N$

$PP \rightarrow IN NP$

$NP \rightarrow DET Nominal$

$NP \rightarrow PRP\$ Nominal$

$NP \rightarrow PRP$

$DET \rightarrow \text{"an"}$

$VBD \rightarrow \text{"shot"}$

$N \rightarrow \text{"elephant"} \mid \text{"pajamas"}$

$PRP \rightarrow \text{"I"}$

$IN \rightarrow \text{"in"}$

$PRP\$ \rightarrow \text{"my"}$

	I	shot	an	elephant	in	my	pajamas
PRP	.	.					
	VBD	.	VP				
		DET	NP				
			N				

CYK Parsing

$S \rightarrow NP VP$

$VP \rightarrow VP PP$

$VP \rightarrow VBD NP$

$Nominal \rightarrow Nominal PP$

$Nominal \rightarrow N$

$PP \rightarrow IN NP$

$NP \rightarrow DET Nominal$

$NP \rightarrow PRP\$ Nominal$

$NP \rightarrow PRP$

$DET \rightarrow \text{"an"}$

$VBD \rightarrow \text{"shot"}$

$N \rightarrow \text{"elephant"} \mid \text{"pajamas"}$

$PRP \rightarrow \text{"I"}$

$IN \rightarrow \text{"in"}$

$PRP\$ \rightarrow \text{"my"}$

	I	shot	an	elephant	in	my	pajamas
PRP	.	.					
	VBD	.	VP				
		DET	NP				
			N				

CYK Parsing

$S \rightarrow NP VP$

$VP \rightarrow VP PP$

$VP \rightarrow VBD NP$

$Nominal \rightarrow Nominal PP$

$Nominal \rightarrow N$

$PP \rightarrow IN NP$

$NP \rightarrow DET Nominal$

$NP \rightarrow PRP\$ Nominal$

$NP \rightarrow PRP$

$DET \rightarrow \text{"an"}$

$VBD \rightarrow \text{"shot"}$

$N \rightarrow \text{"elephant"} \mid \text{"pajamas"}$

$PRP \rightarrow \text{"I"}$

$IN \rightarrow \text{"in"}$

$PRP\$ \rightarrow \text{"my"}$

I	shot	an	elephant	in	my	pajamas
PRP	.	.	.			
	VBD	.	VP			
		DET	NP			
			N			

CYK Parsing

$S \rightarrow NP VP$

$VP \rightarrow VP PP$

$VP \rightarrow VBD NP$

$Nominal \rightarrow Nominal PP$

$Nominal \rightarrow N$

$PP \rightarrow IN NP$

$NP \rightarrow DET Nominal$

$NP \rightarrow PRP\$ Nominal$

$NP \rightarrow PRP$

$DET \rightarrow \text{"an"}$

$VBD \rightarrow \text{"shot"}$

$N \rightarrow \text{"elephant"} \mid \text{"pajamas"}$

$PRP \rightarrow \text{"I"}$

$IN \rightarrow \text{"in"}$

$PRP\$ \rightarrow \text{"my"}$

	I	shot	an	elephant	in	my	pajamas
PRP	.	.	.	.	.		
	VBD	.	VP	.	.		
		DET	NP	.	.		
			N	.	.		Nom
				IN	.		PP
					PRP\$		NP
							N

CYK Parsing

$S \rightarrow NP VP$

$VP \rightarrow VP PP$

$VP \rightarrow VBD NP$

$Nominal \rightarrow Nominal PP$

$Nominal \rightarrow N$

$PP \rightarrow IN NP$

$NP \rightarrow DET Nominal$

$NP \rightarrow PRP\$ Nominal$

$NP \rightarrow PRP$

$DET \rightarrow \text{"an"}$

$VBD \rightarrow \text{"shot"}$

$N \rightarrow \text{"elephant"} \mid \text{"pajamas"}$

$PRP \rightarrow \text{"I"}$

$IN \rightarrow \text{"in"}$

$PRP\$ \rightarrow \text{"my"}$

	I	shot	an	elephant	in	my	pajamas
PRP	.	.	.	.	.	.	.
	VBD	.	VP	.	.	.	.
		DET	NP	.	.	.	.
			N	.	.	.	Nom
				IN	.	.	PP
					PRP\$	.	NP
						N	.



CYK Parsing

$S \rightarrow NP VP$

$VP \rightarrow VP PP$

$VP \rightarrow VBD NP$

$Nominal \rightarrow Nominal PP$

$Nominal \rightarrow N$

$PP \rightarrow IN NP$

$NP \rightarrow DET Nominal$

$NP \rightarrow PRP\$ Nominal$

$NP \rightarrow PRP$

$DET \rightarrow \text{"an"}$

$VBD \rightarrow \text{"shot"}$

$N \rightarrow \text{"elephant"} \mid \text{"pajamas"}$

$PRP \rightarrow \text{"I"}$

$IN \rightarrow \text{"in"}$

$PRP\$ \rightarrow \text{"my"}$

	I	shot	an	elephant	in	my	pajamas
PRP	.	.	.	.	.	.	
	VBD	.	VP	.	.	.	
		DET	NP	.	.	.	
			N	.	.	.	Nom
				IN	.	.	PP
					PRP\$	.	NP
							N

CYK Parsing

$S \rightarrow NP VP$

$VP \rightarrow VP PP$

$VP \rightarrow VBD NP$

$Nominal \rightarrow Nominal PP$

$Nominal \rightarrow N$

$PP \rightarrow IN NP$

$NP \rightarrow DET Nominal$

$NP \rightarrow PRP\$ Nominal$

$NP \rightarrow PRP$

$DET \rightarrow \text{"an"}$

$VBD \rightarrow \text{"shot"}$

$N \rightarrow \text{"elephant"} \mid \text{"pajamas"}$

$PRP \rightarrow \text{"I"}$

$IN \rightarrow \text{"in"}$

$PRP\$ \rightarrow \text{"my"}$

	I	shot	an	elephant	in	my	pajamas
PRP	.	.	.	.	.	.	
	VBD	.	VP	.	.	.	
		DET	NP	.	.	.	
			N	.	.	.	Nom
				IN	.	.	PP
					PRP\$	.	NP
							N

CYK Parsing

$S \rightarrow NP VP$

$VP \rightarrow VP PP$

$VP \rightarrow VBD NP$

$Nominal \rightarrow Nominal PP$

$Nominal \rightarrow N$

$PP \rightarrow IN NP$

$NP \rightarrow DET Nominal$

$NP \rightarrow PRP\$ Nominal$

$NP \rightarrow PRP$

$DET \rightarrow \text{"an"}$

$VBD \rightarrow \text{"shot"}$

$N \rightarrow \text{"elephant"} \mid \text{"pajamas"}$

$PRP \rightarrow \text{"I"}$

$IN \rightarrow \text{"in"}$

$PRP\$ \rightarrow \text{"my"}$

	I	shot	an	elephant	in	my	pajamas
PRP	.	.	.	.	.	.	
	VBD	.	VP	.	.	.	
		DET	NP	.	.	.	
			N	.	.	.	Nom
				IN	.	.	PP
					PRP\$	.	NP
							N

CYK Parsing

$S \rightarrow NP VP$

$VP \rightarrow VP PP$

$VP \rightarrow VBD NP$

$Nominal \rightarrow Nominal PP$

$Nominal \rightarrow N$

$PP \rightarrow IN NP$

$NP \rightarrow DET Nominal$

$NP \rightarrow PRP\$ Nominal$

$NP \rightarrow PRP$

$DET \rightarrow \text{"an"}$

$VBD \rightarrow \text{"shot"}$

$N \rightarrow \text{"elephant"} \mid \text{"pajamas"}$

$PRP \rightarrow \text{"I"}$

$IN \rightarrow \text{"in"}$

$PRP\$ \rightarrow \text{"my"}$

	I	shot	an	elephant	in	my	pajamas
PRP	.	.	.	.	.	.	
	VBD	.	VP	.	.	.	
		DET	NP	.	.	.	NP
			N	.	.	.	Nom
				IN	.	.	PP
					PRP\$	.	NP
							N

CYK Parsing

$S \rightarrow NP VP$

$VP \rightarrow VP PP$

$VP \rightarrow VBD NP$

$Nominal \rightarrow Nominal PP$

$Nominal \rightarrow N$

$PP \rightarrow IN NP$

$NP \rightarrow DET Nominal$

$NP \rightarrow PRP\$ Nominal$

$NP \rightarrow PRP$

$DET \rightarrow \text{"an"}$

$VBD \rightarrow \text{"shot"}$

$N \rightarrow \text{"elephant"} \mid \text{"pajamas"}$

$PRP \rightarrow \text{"I"}$

$IN \rightarrow \text{"in"}$

$PRP\$ \rightarrow \text{"my"}$

	I	shot	an	elephant	in	my	pajamas
PRP	.	.	.	.	.	.	
	VBD	.	VP	.	.	.	
		DET	NP	.	.	.	NP
			N	.	.	.	Nom
				IN	.	.	PP
					PRP\$	.	NP
							N



CYK Parsing

$S \rightarrow NP VP$

$VP \rightarrow VP PP$

$VP \rightarrow VBD NP$

$Nominal \rightarrow Nominal PP$

$Nominal \rightarrow N$

$PP \rightarrow IN NP$

$NP \rightarrow DET Nominal$

$NP \rightarrow PRP\$ Nominal$

$NP \rightarrow PRP$

$DET \rightarrow \text{"an"}$

$VBD \rightarrow \text{"shot"}$

$N \rightarrow \text{"elephant"} \mid \text{"pajamas"}$

$PRP \rightarrow \text{"I"}$

$IN \rightarrow \text{"in"}$

$PRP\$ \rightarrow \text{"my"}$

	I	shot	an	elephant	in	my	pajamas
NP PRP	.	.	.	.	.	.	.
	VBD	.	VP	.	.	.	VP1
		DET	NP	.	.	.	NP
			N	.	.	.	Nom
				IN	.	.	PP
					PRP\$	.	NP
						.	N

CYK Parsing

$S \rightarrow NP VP$

$VP \rightarrow VP PP$

$VP \rightarrow VBD NP$

$Nominal \rightarrow Nominal PP$

$Nominal \rightarrow N$

$PP \rightarrow IN NP$

$NP \rightarrow DET Nominal$

$NP \rightarrow PRP\$ Nominal$

$NP \rightarrow PRP$

$DET \rightarrow \text{"an"}$

$VBD \rightarrow \text{"shot"}$

$N \rightarrow \text{"elephant"} \mid \text{"pajamas"}$

$PRP \rightarrow \text{"I"}$

$IN \rightarrow \text{"in"}$

$PRP\$ \rightarrow \text{"my"}$

	I	shot	an	elephant	in	my	pajamas
NP PRP	.	.	.	.	.	.	.
VBD		.	VP	.	.	.	.
DET			NP	.	.	.	NP
N				.	.	.	Nom
IN					.	.	PP
PRP\$						.	NP
N							N

CYK Parsing

$S \rightarrow NP VP$

$VP \rightarrow VP PP$

$VP \rightarrow VBD NP$

$Nominal \rightarrow Nominal PP$

$Nominal \rightarrow N$

$PP \rightarrow IN NP$

$NP \rightarrow DET Nominal$

$NP \rightarrow PRP\$ Nominal$

$NP \rightarrow PRP$

$DET \rightarrow \text{"an"}$

$VBD \rightarrow \text{"shot"}$

$N \rightarrow \text{"elephant"} \mid \text{"pajamas"}$

$PRP \rightarrow \text{"I"}$

$IN \rightarrow \text{"in"}$

$PRP\$ \rightarrow \text{"my"}$

	I	shot	an	elephant	in	my	pajamas
NP PRP	.	.	.	.	.	.	.
VBD		.	VP	.	.	VP2	
		DET	NP	.	.	NP	
			N	.	.	Nom	
				IN	.	PP	
					PRP\$	NP	
						N	

CYK Parsing

$S \rightarrow NP VP$

$VP \rightarrow VP PP$

$VP \rightarrow VBD NP$

$Nominal \rightarrow Nominal PP$

$Nominal \rightarrow N$

$PP \rightarrow IN NP$

$NP \rightarrow DET Nominal$

$NP \rightarrow PRP\$ Nominal$

$NP \rightarrow PRP$

$DET \rightarrow \text{"an"}$

$VBD \rightarrow \text{"shot"}$

$N \rightarrow \text{"elephant"} \mid \text{"pajamas"}$

$PRP \rightarrow \text{"I"}$

$IN \rightarrow \text{"in"}$

$PRP\$ \rightarrow \text{"my"}$

	I	shot	an	elephant	in	my	pajamas
PRP	.	.	.	.	.	.	
	VBD	.	VP	.	.		VP1 VP2
		DET	NP	.	.		NP
			N	.	.		Nom
				IN	.		PP
					PRP\$		NP
							N

CYK Parsing

$S \rightarrow NP VP$

$VP \rightarrow VP PP$

$VP \rightarrow VBD NP$

$Nominal \rightarrow Nominal PP$

$Nominal \rightarrow N$

$PP \rightarrow IN NP$

$NP \rightarrow DET Nominal$

$NP \rightarrow PRP\$ Nominal$

$NP \rightarrow PRP$

$DET \rightarrow \text{"an"}$

$VBD \rightarrow \text{"shot"}$

$N \rightarrow \text{"elephant"} \mid \text{"pajamas"}$

$PRP \rightarrow \text{"I"}$

$IN \rightarrow \text{"in"}$

$PRP\$ \rightarrow \text{"my"}$

	I	shot	an	elephant	in	my	pajamas
PRP	.	.	.	.	.	.	
	VBD	.	VP	.	.	.	VP1 VP2
		DET	NP	.	.	.	NP
			N	.	.	.	Nom
				IN	.	.	PP
					PRP\$	.	NP
						N	

CYK Parsing

$S \rightarrow NP VP$

$VP \rightarrow VP PP$

$VP \rightarrow VBD NP$

$Nominal \rightarrow Nominal PP$

$Nominal \rightarrow N$

$PP \rightarrow IN NP$

$NP \rightarrow DET Nominal$

$NP \rightarrow PRP\$ Nominal$

$NP \rightarrow PRP$

$DET \rightarrow \text{"an"}$

$VBD \rightarrow \text{"shot"}$

$N \rightarrow \text{"elephant"} \mid \text{"pajamas"}$

$PRP \rightarrow \text{"I"}$

$IN \rightarrow \text{"in"}$

$PRP\$ \rightarrow \text{"my"}$

	I	shot	an	elephant	in	my	pajamas
PRP	.	.	.	.	.	.	S1 S2
	VBD	.	VP	.	.		VP1 VP2
		DET	NP	.	.		NP
			N	.	.		Nom
				IN	.		PP
					PRP\$		NP
							N

CYK Parsing

$S \rightarrow NP VP$

$VP \rightarrow VP PP$

$VP \rightarrow VBD NP$

$Nominal \rightarrow Nominal PP$

$Nominal \rightarrow N$

$PP \rightarrow IN NP$

$NP \rightarrow DET Nominal$

$NP \rightarrow PRP\$ Nominal$

$NP \rightarrow PRP$

$DET \rightarrow \text{"an"}$

$VBD \rightarrow \text{"shot"}$

$N \rightarrow \text{"elephant"} \mid \text{"pajamas"}$

$PRP \rightarrow \text{"I"}$

$IN \rightarrow \text{"in"}$

$PRP\$ \rightarrow \text{"my"}$

	I	shot	an	elephant	in	my	pajamas
PRP	.	.	.	.	.	.	S1
	VBD	.	VP	.	.	.	VP1
		DET	NP	.	.	.	NP
			N	.	.	.	Nom
				IN	.	.	PP
					PRP\$	.	NP
						.	N

CYK Parsing

$S \rightarrow NP VP$

$VP \rightarrow VP PP$

$VP \rightarrow VBD NP$

$Nominal \rightarrow Nominal PP$

$Nominal \rightarrow N$

$PP \rightarrow IN NP$

$NP \rightarrow DET Nominal$

$NP \rightarrow PRP\$ Nominal$

$NP \rightarrow PRP$

$DET \rightarrow \text{"an"}$

$VBD \rightarrow \text{"shot"}$

$N \rightarrow \text{"elephant"} \mid \text{"pajamas"}$

$PRP \rightarrow \text{"I"}$

$IN \rightarrow \text{"in"}$

$PRP\$ \rightarrow \text{"my"}$

	I	shot	an	elephant	in	my	pajamas
	PRP	.	.	.	.	.	S1
		VBD	.	VP	.	.	VP1
			DET	NP	.	.	NP
				N	.	.	Nom
					IN	.	PP
						PRP\$	NP
							N

CYK Parsing

$S \rightarrow NP VP$

$VP \rightarrow VP PP$

$VP \rightarrow VBD NP$

$Nominal \rightarrow Nominal PP$

$Nominal \rightarrow N$

$PP \rightarrow IN NP$

$NP \rightarrow DET Nominal$

$NP \rightarrow PRP\$ Nominal$

$NP \rightarrow PRP$

$DET \rightarrow \text{"an"}$

$VBD \rightarrow \text{"shot"}$

$N \rightarrow \text{"elephant"} \mid \text{"pajamas"}$

$PRP \rightarrow \text{"I"}$

$IN \rightarrow \text{"in"}$

$PRP\$ \rightarrow \text{"my"}$

	I	shot	an	elephant	in	my	pajamas
	PRP						S1
		VBD		VP			VP1
			DET	NP	.	.	NP
				N	.	.	Nom
					IN	.	PP
						PRP\$	NP
							N

CYK Parsing

$S \rightarrow NP VP$

$VP \rightarrow VP PP$

$VP \rightarrow VBD NP$

$Nominal \rightarrow Nominal PP$

$Nominal \rightarrow N$

$PP \rightarrow IN NP$

$NP \rightarrow DET Nominal$

$NP \rightarrow PRP\$ Nominal$

$NP \rightarrow PRP$

$DET \rightarrow \text{"an"}$

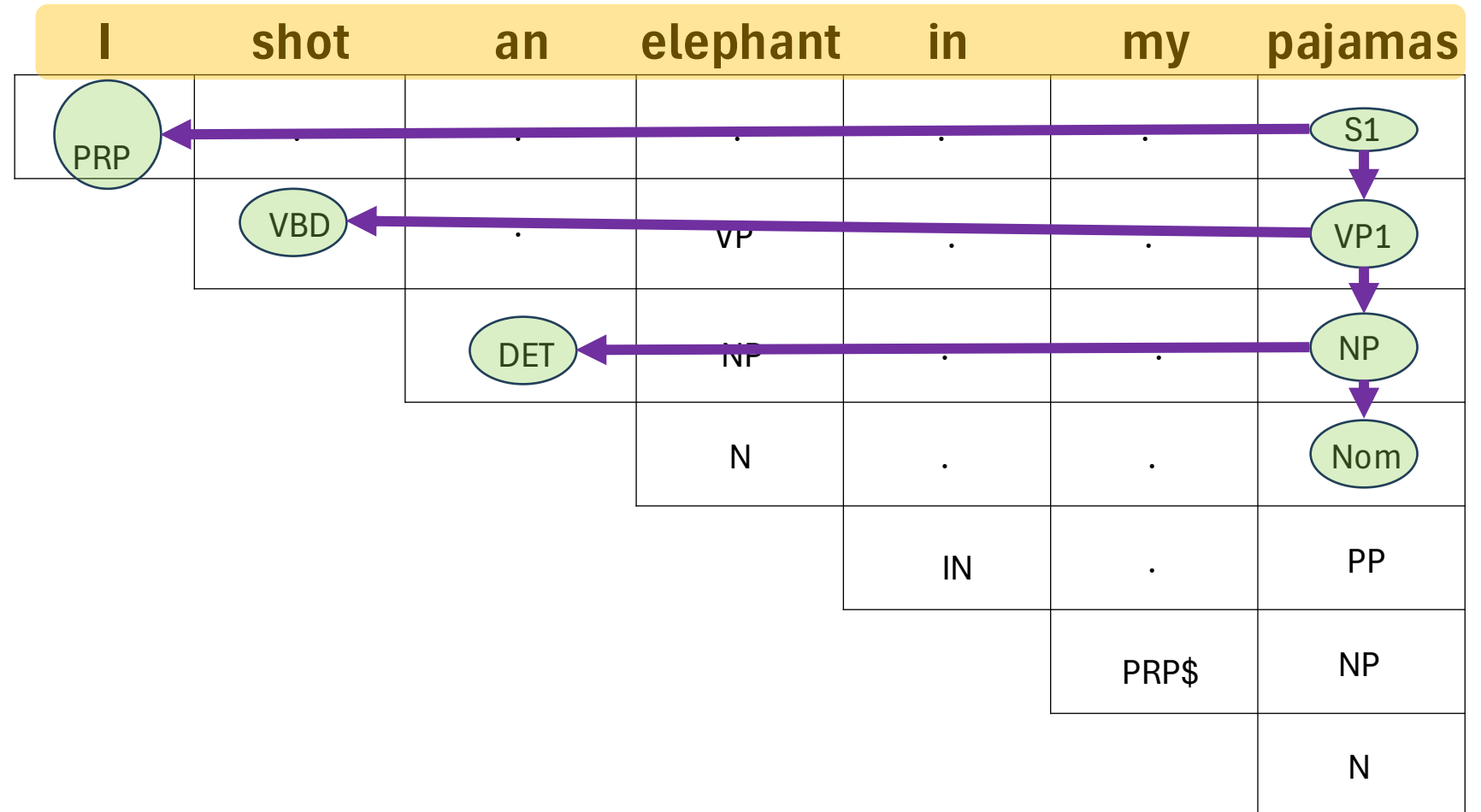
$VBD \rightarrow \text{"shot"}$

$N \rightarrow \text{"elephant" | "pajamas"}$

$PRP \rightarrow \text{"I"}$

$IN \rightarrow \text{"in"}$

$PRP\$ \rightarrow \text{"my"}$



CYK Parsing

$S \rightarrow NP VP$

$VP \rightarrow VP PP$

$VP \rightarrow VBD NP$

$Nominal \rightarrow Nominal PP$

$Nominal \rightarrow N$

$PP \rightarrow IN NP$

$NP \rightarrow DET Nominal$

$NP \rightarrow PRP\$ Nominal$

$NP \rightarrow PRP$

$DET \rightarrow \text{"an"}$

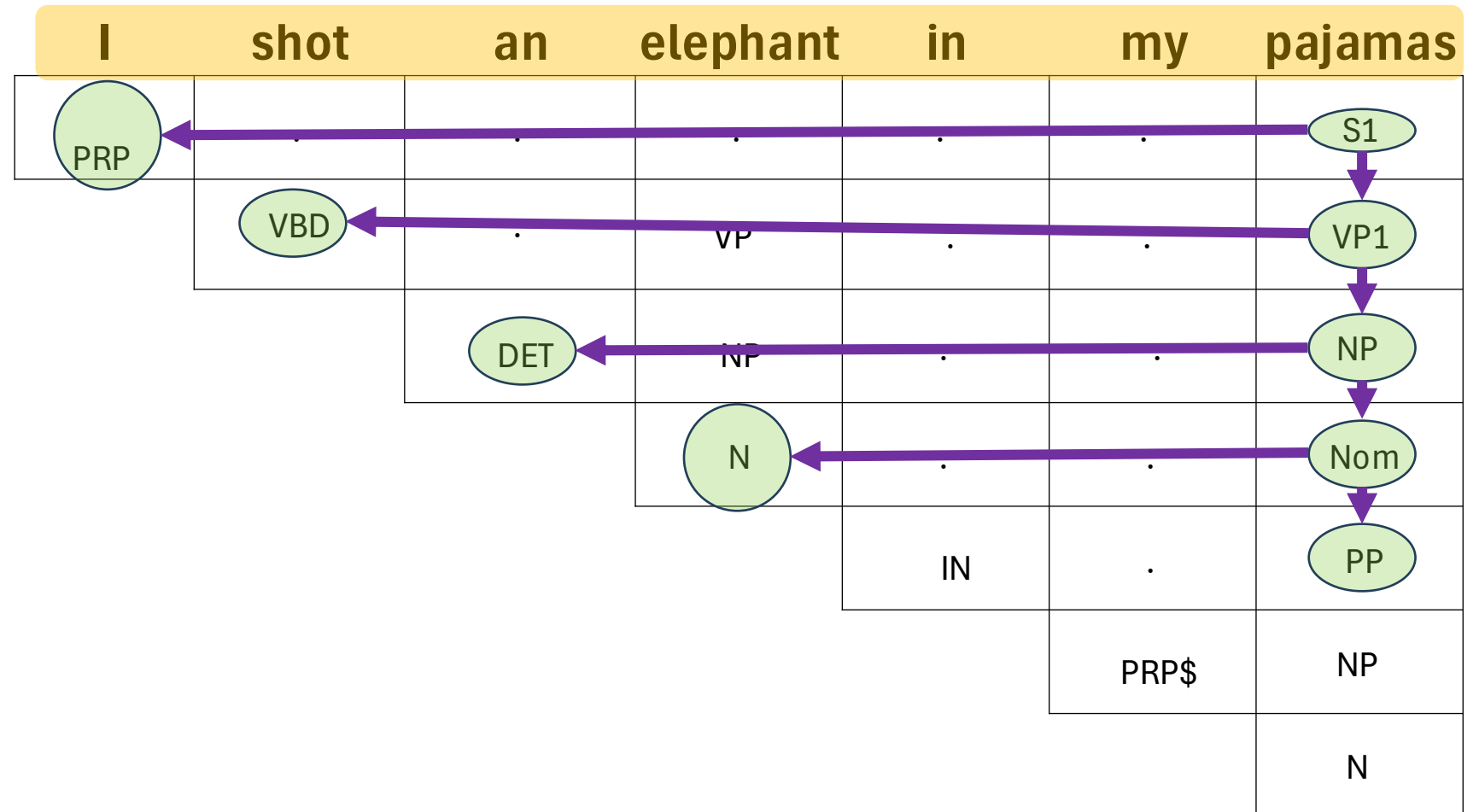
$VBD \rightarrow \text{"shot"}$

$N \rightarrow \text{"elephant"} \mid \text{"pajamas"}$

$PRP \rightarrow \text{"I"}$

$IN \rightarrow \text{"in"}$

$PRP\$ \rightarrow \text{"my"}$



CYK Parsing

$S \rightarrow NP VP$

$VP \rightarrow VP PP$

$VP \rightarrow VBD NP$

$Nominal \rightarrow Nominal PP$

$Nominal \rightarrow N$

$PP \rightarrow IN NP$

$NP \rightarrow DET Nominal$

$NP \rightarrow PRP\$ Nominal$

$NP \rightarrow PRP$

$DET \rightarrow \text{"an"}$

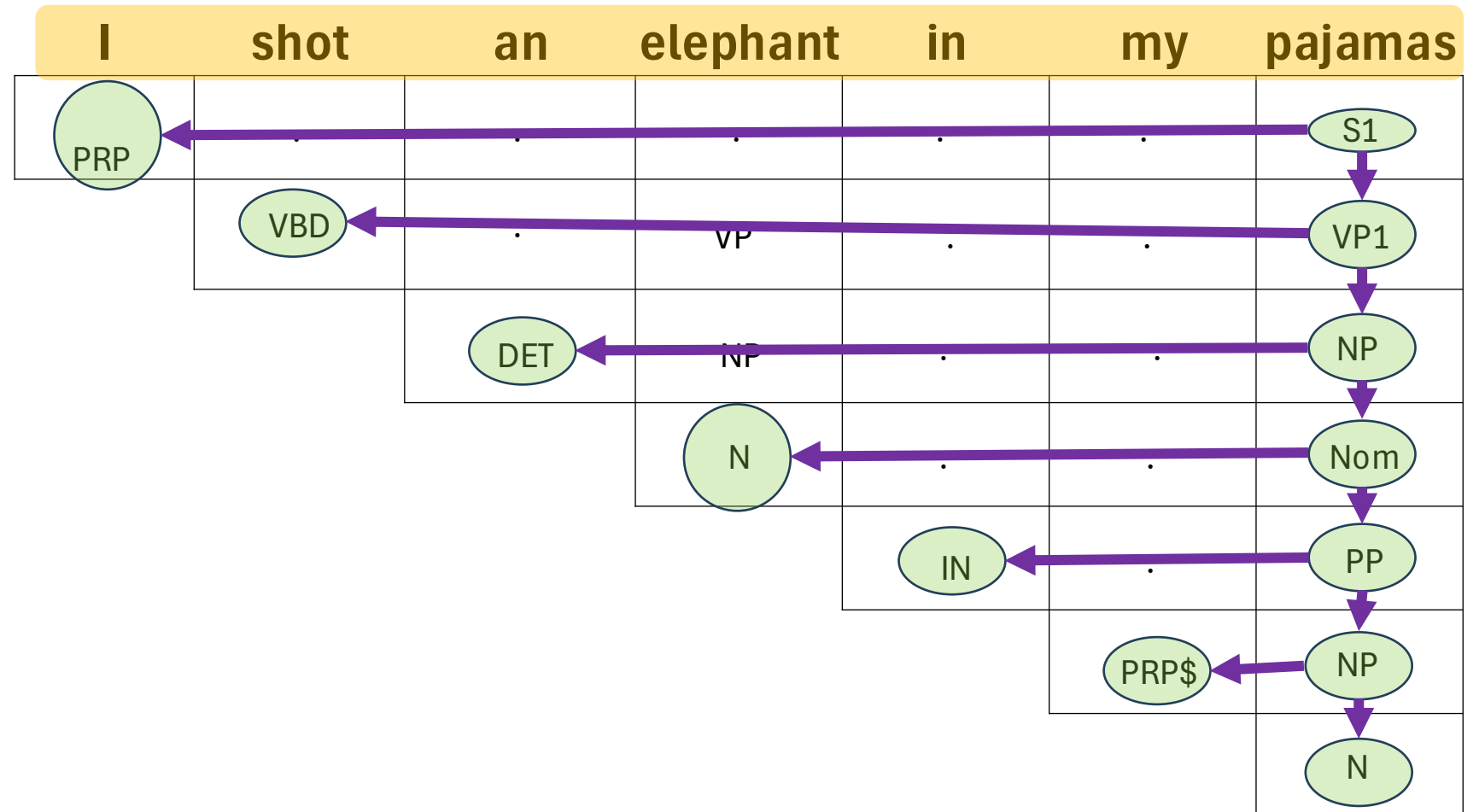
$VBD \rightarrow \text{"shot"}$

$N \rightarrow \text{"elephant"} \mid \text{"pajamas"}$

$PRP \rightarrow \text{"I"}$

$IN \rightarrow \text{"in"}$

$PRP\$ \rightarrow \text{"my"}$



CYK Parsing

$S \rightarrow NP VP$

$VP \rightarrow VP PP$

$VP \rightarrow VBD NP$

$Nominal \rightarrow Nominal PP$

$Nominal \rightarrow N$

$PP \rightarrow IN NP$

$NP \rightarrow DET Nominal$

$NP \rightarrow PRP\$ Nominal$

$NP \rightarrow PRP$

$DET \rightarrow \text{"an"}$

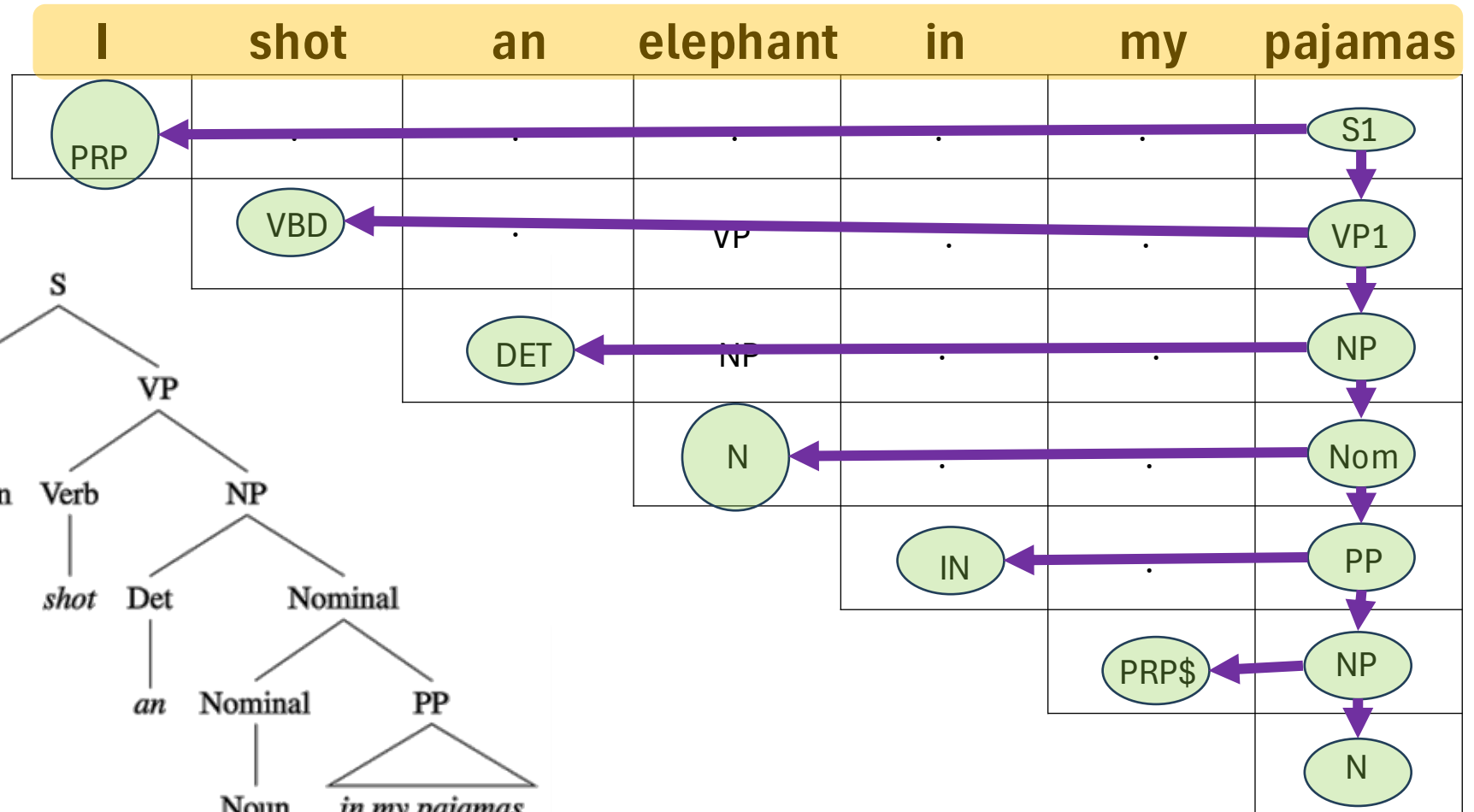
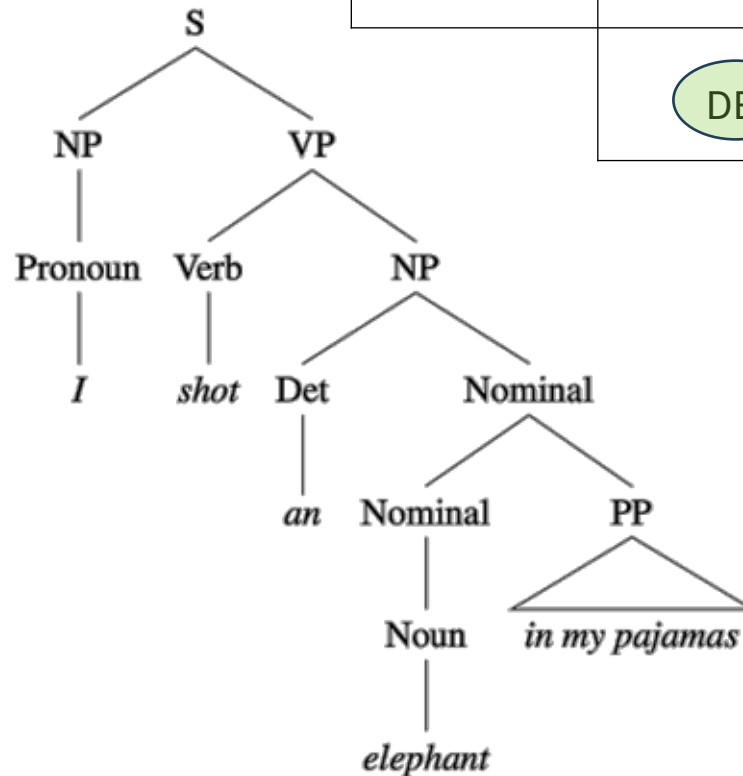
$VBD \rightarrow \text{"shot"}$

$N \rightarrow \text{"elephant"} \mid \text{"pajamas"}$

$PRP \rightarrow \text{"I"}$

$IN \rightarrow \text{"in"}$

$PRP\$ \rightarrow \text{"my"}$



CYK Parsing

$S \rightarrow NP VP$

$VP \rightarrow VP PP$

$VP \rightarrow VBD NP$

$Nominal \rightarrow Nominal PP$

$Nominal \rightarrow N$

$PP \rightarrow IN NP$

$NP \rightarrow DET Nominal$

$NP \rightarrow PRP\$ Nominal$

$NP \rightarrow PRP$

$DET \rightarrow \text{"an"}$

$VBD \rightarrow \text{"shot"}$

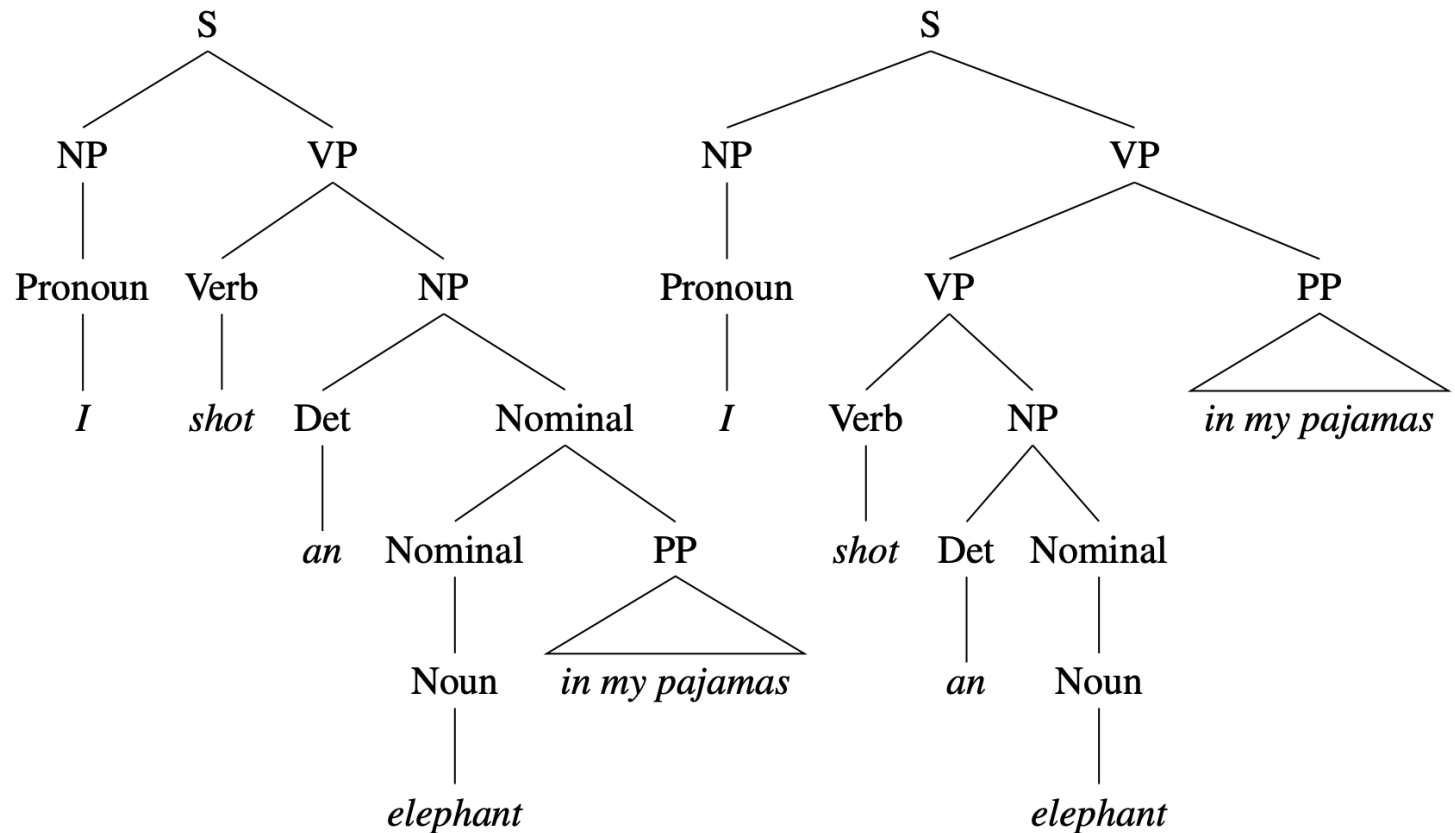
$N \rightarrow \text{"elephant"} \mid \text{"pajamas"}$

$PRP \rightarrow \text{"I"}$

$IN \rightarrow \text{"in"}$

$PRP\$ \rightarrow \text{"my"}$

"I shot an elephant in my pajamas"



CYK Parsing

Benefits:

- Verifies whether a sentence is grammatically correct in the language defined by the CFG
- Helps create all possible parses for a sentence in a given CFG

CYK Parsing

Benefits:

- Verifies whether a sentence is grammatically correct in the language defined by the CFG
- Helps create all possible parses for a sentence in a given CFG

HOWEVER,

It does NOT handle the structural ambiguity to determine the most likely parsing tree

Context-Free Grammar (CFG)

Ambiguity

- If a context free grammar G has more than one derivation tree for some string $w \in L(G)$, it is called an ambiguous grammar

"I saw the man with the telescope"

Context-Free Grammar (CFG)

Ambiguity

"I saw the man with the telescope"



<https://www.google.com/url?sa=i&url=https://www.adobe.com/search/2Fimages%3Fk%3Dtelescope%2Bman&psig=AOvVaw168o1FQLB1oivwDjb-nwnT&usq=1706714180460000&source=images&cd=vfe&opi=89978449&ved=0CBMQJRxqFwoTCMjm1fWzhYQDFQAAAAAdAAAAABAJ>



Context-Free Grammar (CFG)

Ambiguity

- If a context free grammar G has more than one derivation tree for some string $w \in L(G)$, it is called an ambiguous grammar

"I saw the man with the telescope"

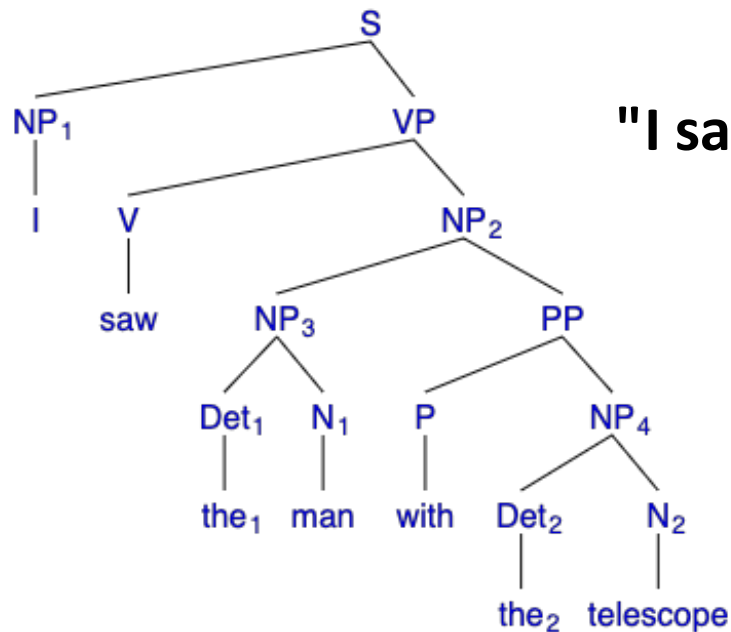
Interpretation 1: The speaker used a telescope to see the man.

Interpretation 2: The speaker saw a man who was holding or possessing a telescope.

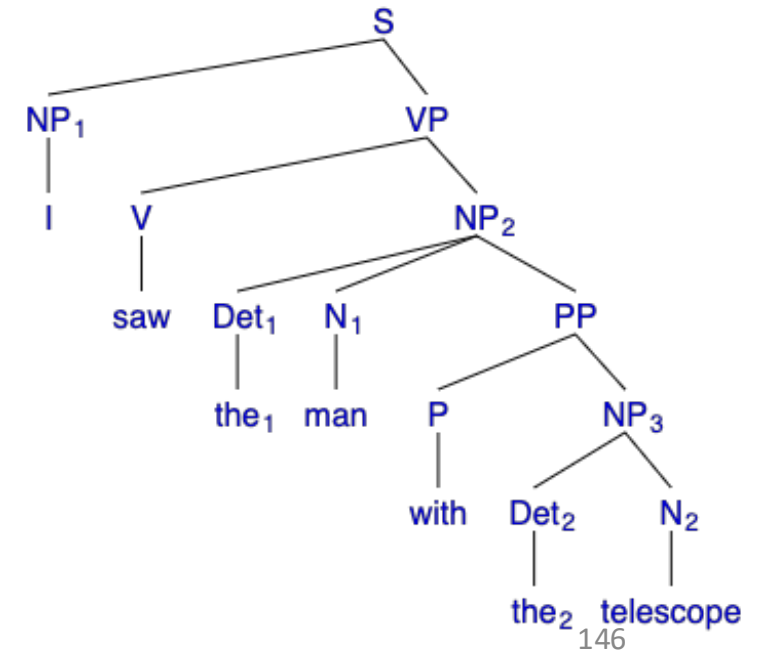
Context-Free Grammar (CFG)

Ambiguity

- If a context free grammar G has more than one derivation tree for some string $w \in L(G)$, it is called an ambiguous grammar



"I saw the man with the telescope"



Context-Free Grammar (CFG)

Ambiguity

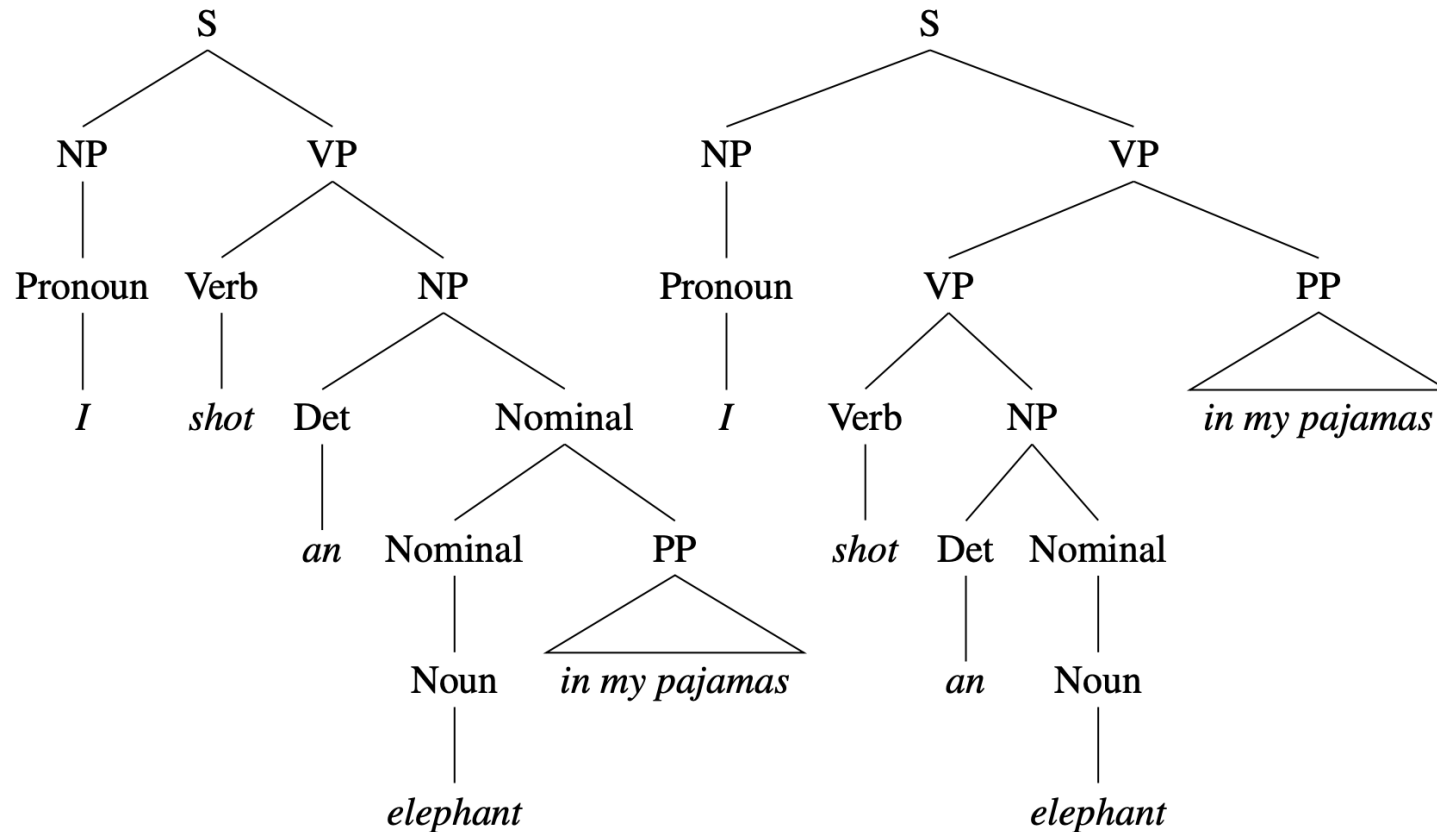
- **attachment ambiguity:** A sentence has an attachment ambiguity if a particular constituent can be attached to the parse tree at more than one place

"I shot an elephant in my pajamas"

Context-Free Grammar (CFG)

Ambiguity

"I shot an elephant in my pajamas"



Context-Free Grammar (CFG)

Ambiguity

- **attachment ambiguity:** A sentence has an attachment ambiguity if a particular constituent can be attached to the parse tree at more than one place
- **coordination ambiguity:** Phrases can be conjoined by a conjunction like *and*

“the old men and women”

[old [men and women]]

[old men] and [women]

Context-Free Grammar (CFG)

Ambiguity

- **attachment ambiguity:** A sentence has an attachment ambiguity if a particular constituent can be attached to the parse tree at more than one place
- **coordination ambiguity:** Phrases can be conjoined by a conjunction like *and*
- **scope ambiguity:** This type of ambiguity arises when it's unclear how far the scope of a modifier or operator extends.

“He didn't speak to her or acknowledge her”

he neither spoke to her nor acknowledged her

he did not speak but did acknowledge

Probabilistic Context-Free Grammar (PCFG)

Dealing with Ambiguity

- Probabilistic Context-Free Grammar (PCFG) can help mitigate syntactic ambiguities by assigning probabilities to each production rule in the grammar.
 - choose the most likely interpretation of a sentence based on the learned probabilities

Probabilistic Context-Free Grammar (PCFG)

- Each production rule is associated with a probability.

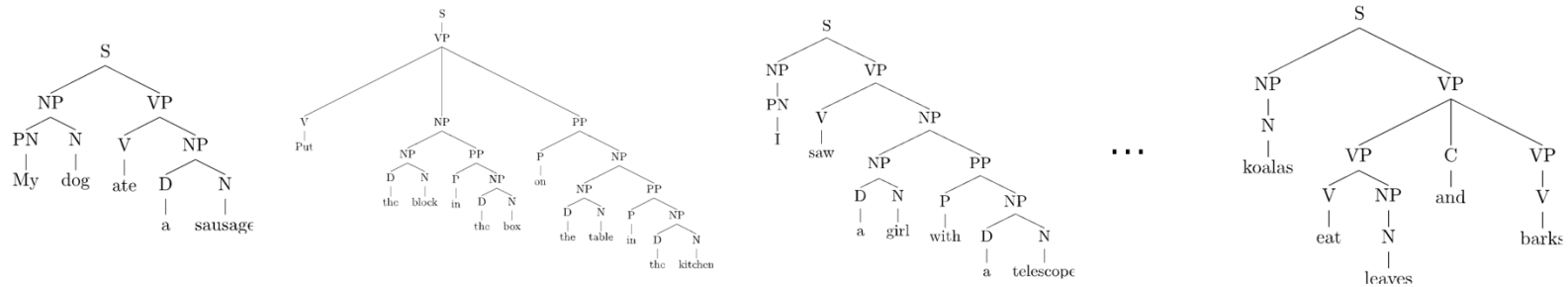
$$P(X \rightarrow \alpha) \quad \forall X \rightarrow \alpha \in R : 0 \leq P(X \rightarrow \alpha) \leq 1$$

$$\forall X \in N : \sum_{\alpha: X \rightarrow \alpha \in R} P(X \rightarrow \alpha) = 1$$

- Probabilities reflect how frequently a particular production is used in real-world text.

Probabilistic Context-Free Grammar (PCFG)

- Probabilities are typically learned from a labeled dataset (a treebank) and represent how likely a particular rule is used in the language.
- A treebank: a collection of sentences annotated with constituency trees



Penn Treebank Project

A large annotated corpus of English sentences with grammatical structure and parse tree representations.

- Developed in the late 1980s/early 1990s by University of Pennsylvania linguists
- Contains over 4.5 million words of American English annotated with parts of speech and parse trees
- Includes text from various domains like news, transcripts, academic papers etc.
- Provides hand-parsed syntactic trees for sentences identifying phrases and grammar relations
- Enabled development of statistical parsing by serving as a comprehensive treebank to train machine learning models
- Became a widely-used standard dataset in NLP research for tasks like parsing, POS tagging, language modeling etc.
- Inspired creation of similar syntactically annotated treebanks for other languages as well

Building a Large Annotated Corpus of English: The Penn Treebank

Mitchell P. Marcus^{*}
University of Pennsylvania

Beatrice Santorini[†]
Northwestern University

Mary Ann Marcinkiewicz[‡]
University of Pennsylvania

Probabilistic Context-Free Grammar (PCFG)

- Probabilities are typically learned from a labeled dataset (a treebank) and represent how likely a particular rule is used in the language.
 - A treebank: a collection of sentences annotated with constituency trees
- Estimated probability of a rule (maximum likelihood estimate):

$$P(X \rightarrow \alpha) = \frac{C(X \rightarrow \alpha)}{C(X)}$$

times the rule was used in the corpus

times nonterminal X appeared in the treebank

Probabilistic Context-Free Grammar (PCFG)

- Without access to treebank: inside-outside algorithm to estimate probabilities
 - Randomly initialize probabilities
 - Compute parses
 - Compute expected values (counts) for rules
 - Re-estimate probabilities accordingly
 - repeat

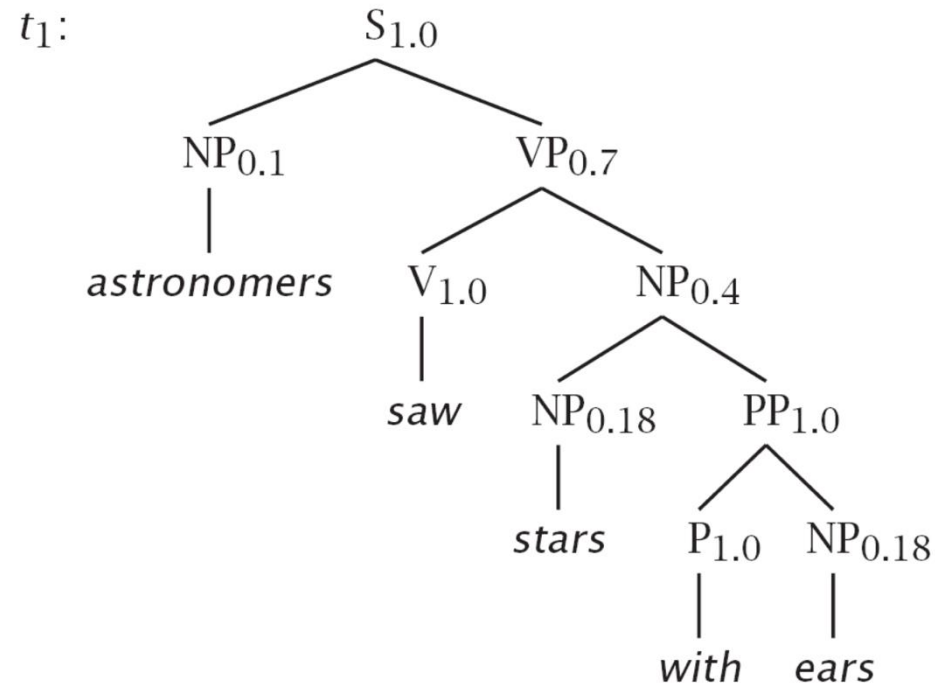
Probabilistic Context-Free Grammar (PCFG)

Example: “Astronomers saw stars with ears”

$S \rightarrow NP VP$	1.0	$NP \rightarrow NP PP$	0.4
$PP \rightarrow P NP$	1.0	$NP \rightarrow \textit{astronomers}$	0.1
$VP \rightarrow V NP$	0.7	$NP \rightarrow \textit{ears}$	0.18
$VP \rightarrow VP PP$	0.3	$NP \rightarrow \textit{saw}$	0.04
$P \rightarrow \textit{with}$	1.0	$NP \rightarrow \textit{stars}$	0.18
$V \rightarrow \textit{saw}$	1.0	$NP \rightarrow \textit{telescopes}$	0.1

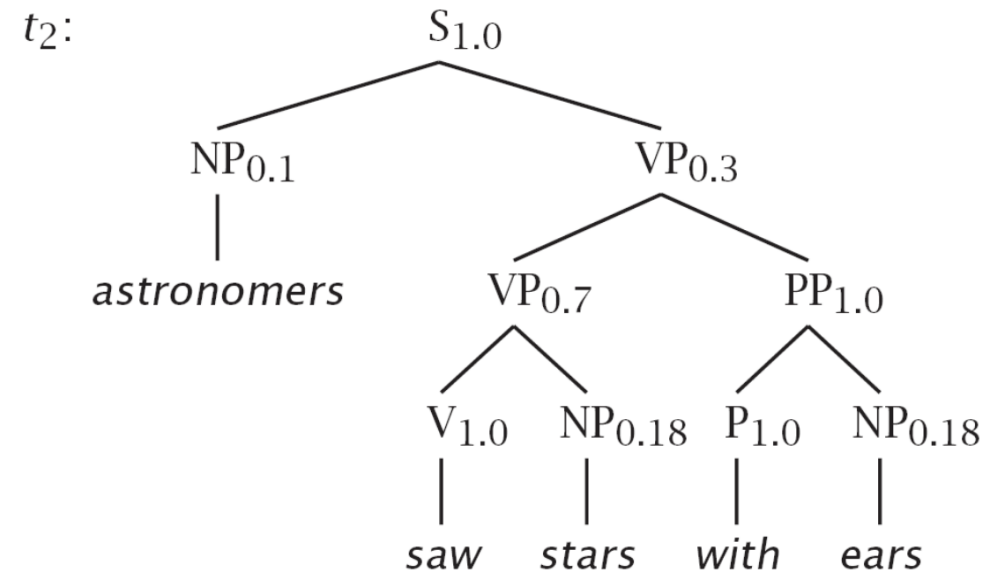
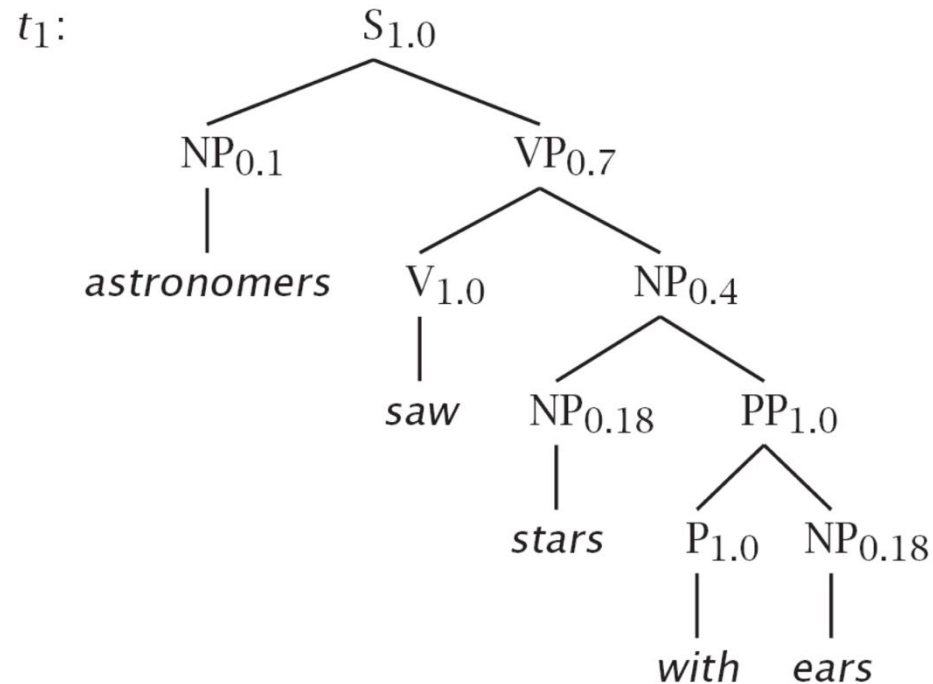
Probabilistic Context-Free Grammar (PCFG)

Example: “Astronomers saw stars with ears”



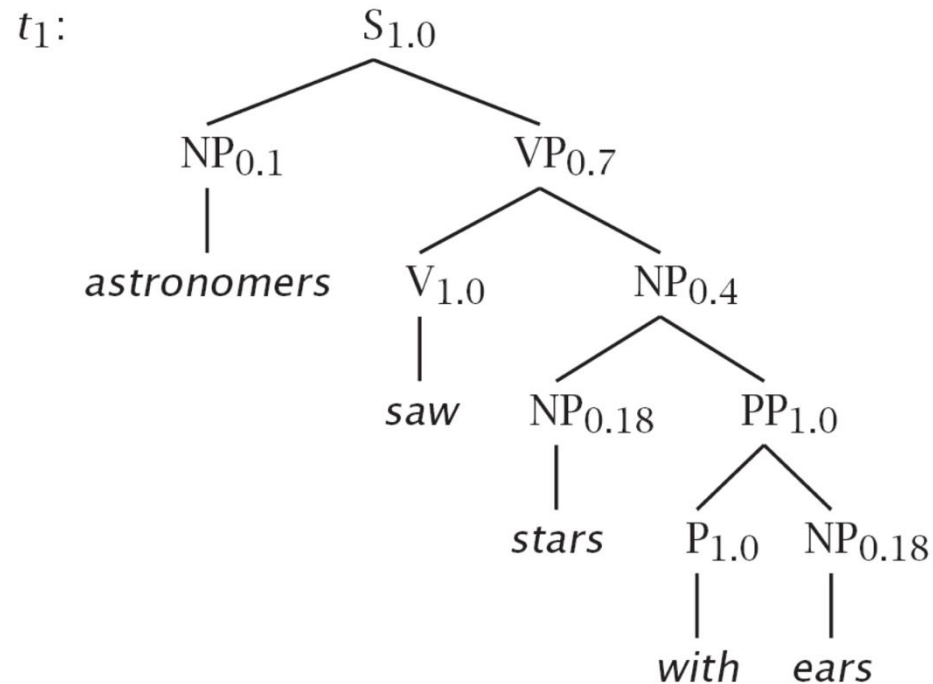
Probabilistic Context-Free Grammar (PCFG)

Example: “Astronomers saw stars with ears”

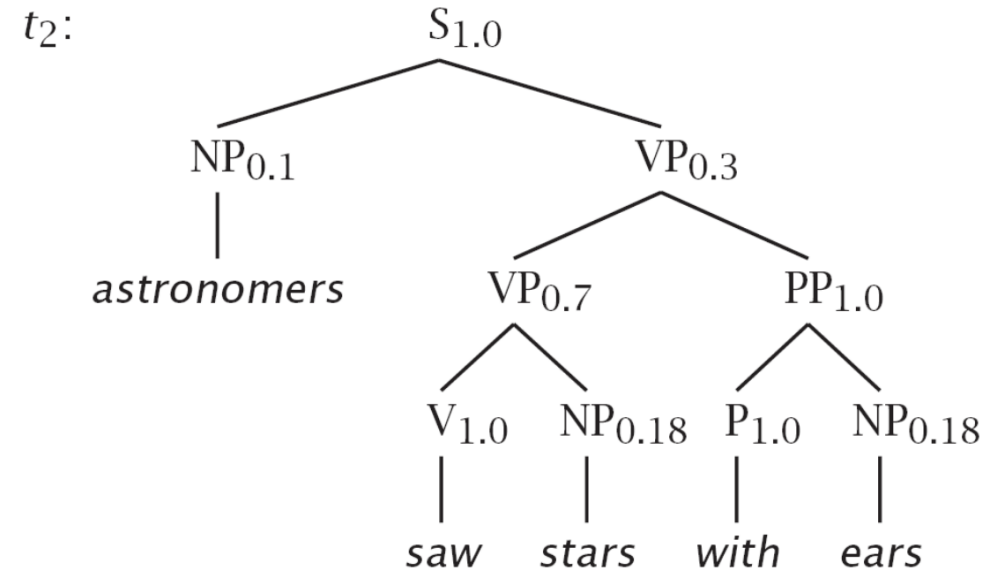


Probabilistic Context-Free Grammar (PCFG)

Example: “Astronomers saw stars with ears”



$$\begin{aligned} P(t_1) &= 1.0 \times 0.1 \times 0.7 \times 1.0 \times 0.4 \\ &\quad \times 0.18 \times 1.0 \times 1.0 \times 0.18 \\ &= 0.0009072 \end{aligned}$$



$$\begin{aligned} P(t_2) &= 1.0 \times 0.1 \times 0.3 \times 0.7 \times 1.0 \\ &\quad \times 0.18 \times 1.0 \times 1.0 \times 0.18 \\ &= 0.0006804 \end{aligned}$$

Probabilistic Context-Free Grammar (PCFG)

```
function PROBABILISTIC-CKY(words, grammar) returns most probable parse
                                     and its probability

for  $j \leftarrow$  from 1 to LENGTH(words) do
    for all {  $A \mid A \rightarrow \text{words}[j] \in \text{grammar}$  }
         $\text{table}[j-1, j, A] \leftarrow P(A \rightarrow \text{words}[j])$ 
    for  $i \leftarrow$  from  $j-2$  downto 0 do
        for  $k \leftarrow i+1$  to  $j-1$  do
            for all {  $A \mid A \rightarrow BC \in \text{grammar},$ 
                      and  $\text{table}[i, k, B] > 0$  and  $\text{table}[k, j, C] > 0$  }
                if ( $\text{table}[i, j, A] < P(A \rightarrow BC) \times \text{table}[i, k, B] \times \text{table}[k, j, C]$ ) then
                     $\text{table}[i, j, A] \leftarrow P(A \rightarrow BC) \times \text{table}[i, k, B] \times \text{table}[k, j, C]$ 
                     $\text{back}[i, j, A] \leftarrow \{k, B, C\}$ 
    return BUILD_TREE( $\text{back}[1, \text{LENGTH}(\text{words}), S]$ ),  $\text{table}[1, \text{LENGTH}(\text{words}), S]$ 
```

Figure 14.3 The probabilistic CKY algorithm for finding the maximum probability parse of a string of *num_words* words given a PCFG grammar with *num_rules* rules in Chomsky normal form. *back* is an array of backpointers used to recover the best parse. The *build_tree* function is left as an exercise to the reader.

Probabilistic Context-Free Grammar (PCFG)

Deficiencies:

Independence Assumption:

PCFGs assume that the production of a non-terminal symbol is independent of its context, which is not always true in natural languages.

Probabilistic Context-Free Grammar (PCFG)

Deficiencies:

Independence Assumption:

PCFGs assume that the production of a non-terminal symbol is independent of its context, which is not always true in natural languages.

No Lexical Conditioning:

The probabilities are conditioned solely on the non-terminal symbols involved in the productions, without considering the actual lexical items that fill those positions in the parse tree.

Probabilistic Context-Free Grammar (PCFG)

Deficiencies:

Independence Assumption:

PCFGs assume that the production of a non-terminal symbol is independent of its context, which is not always true in natural languages.

No Lexical Conditioning:

The probabilities are conditioned solely on the non-terminal symbols involved in the productions, without considering the actual lexical items that fill those positions in the parse tree.

Solution: Probabilistic Lexicalized Context-Free Grammars (L-PCFGs)

each production is associated with a head word, which is the central word that determines the syntactic type of the phrase.

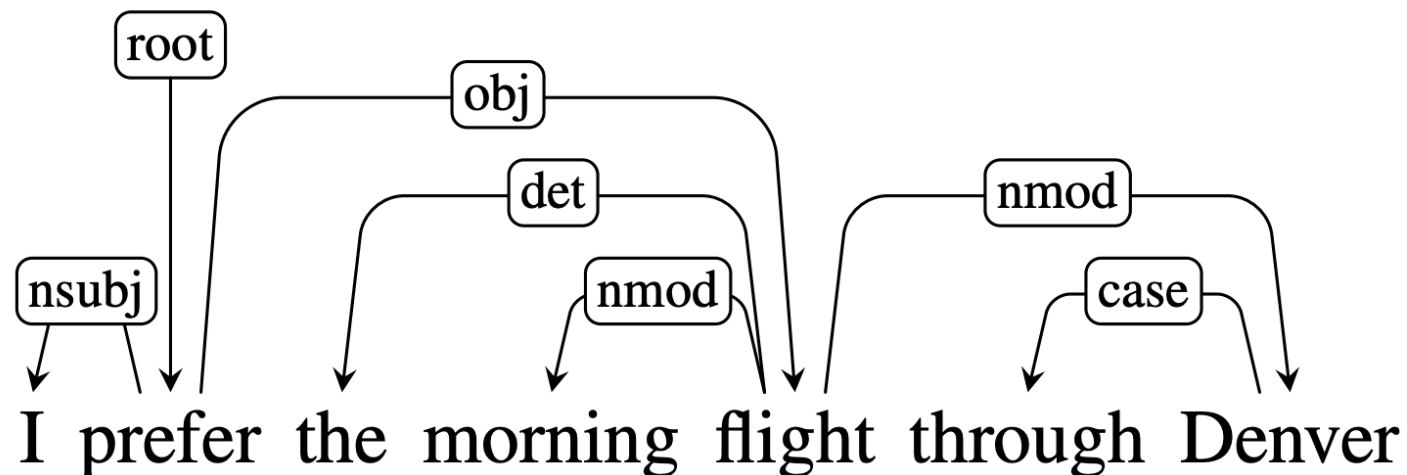
e.g., $NP(head:dog) \rightarrow Det(the) N(head:dog)$, where "dog" is the head of the noun phrase.

Parsing

- **Types:**
 - Syntactic Parsing
 - Constituency Parsing
 - **Dependency Parsing**
 - Semantic Parsing

Dependency Grammar

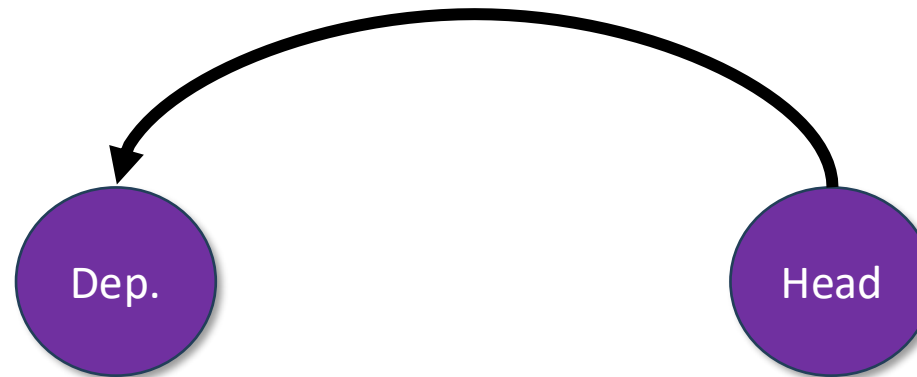
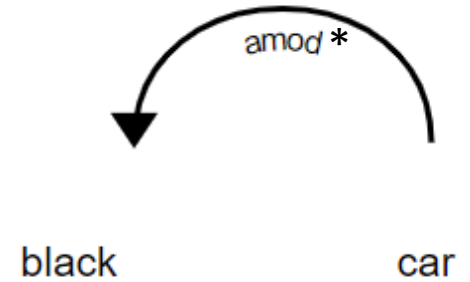
- The syntactic structure of a sentence is described solely in terms of directed binary grammatical relations between the words
 - words are connected to each other by directed links
 - Verb: center of the clause structure
 - Dependencies: every other syntactic unit is connected to the verb via a directed link



Dependency Parsing

Each relationship:

- Has one **head** and a **dependent** that modifies the **head**.



Dependent modifies the meaning of head

Dependency Parsing

Each relationship:

- Is labeled according to the nature of the dependency between the **head** and the **dependent**.

Universal Dependency Relations:

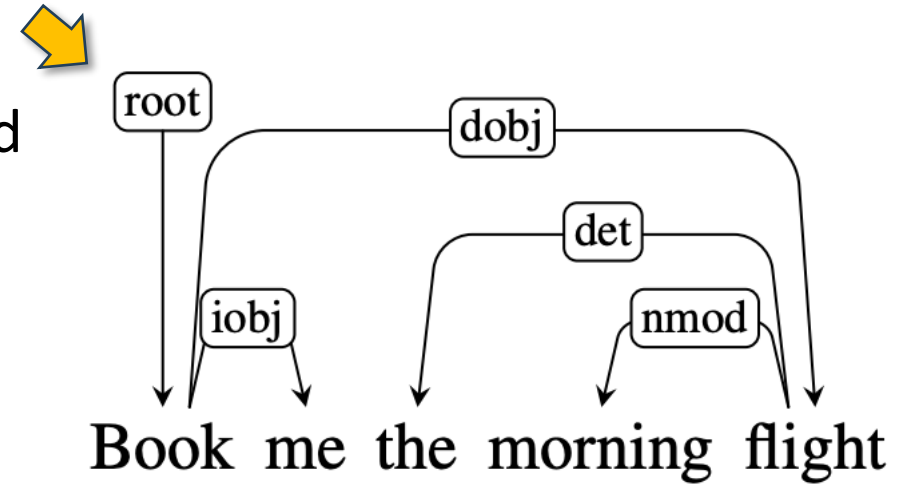
<https://universaldependencies.org/u/dep/>

Clausal Argument Relations	Description
NSUBJ	Nominal subject
DOBJ	Direct object
IOBJ	Indirect object
CCOMP	Clausal complement
XCOMP	Open clausal complement
Nominal Modifier Relations	Description
NMOD	Nominal modifier
AMOD	Adjectival modifier
NUMMOD	Numeric modifier
APPOS	Appositional modifier
DET	Determiner
CASE	Prepositions, postpositions and other case markers
Other Notable Relations	Description
CONJ	Conjunct
CC	Coordinating conjunction

Figure 15.2 Selected dependency relations from the Universal Dependency set. (de Marneffe et al., 2014)

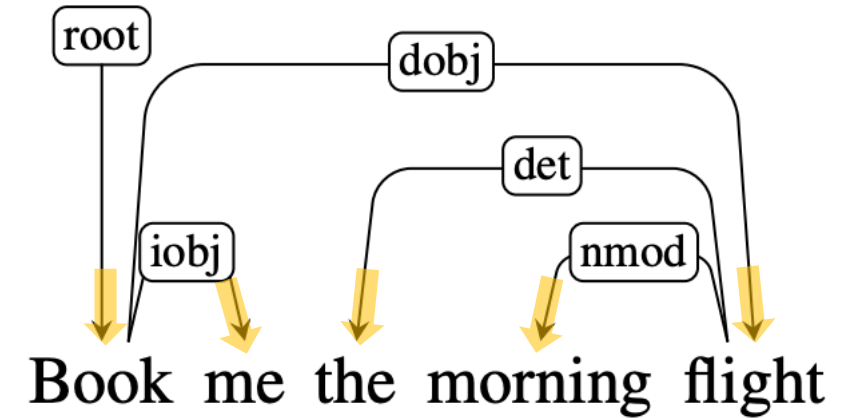
Dependency Parsing

- Usually add a fake **ROOT** at the beginning so every word has one head
 - Only one word is a dependent of **ROOT**



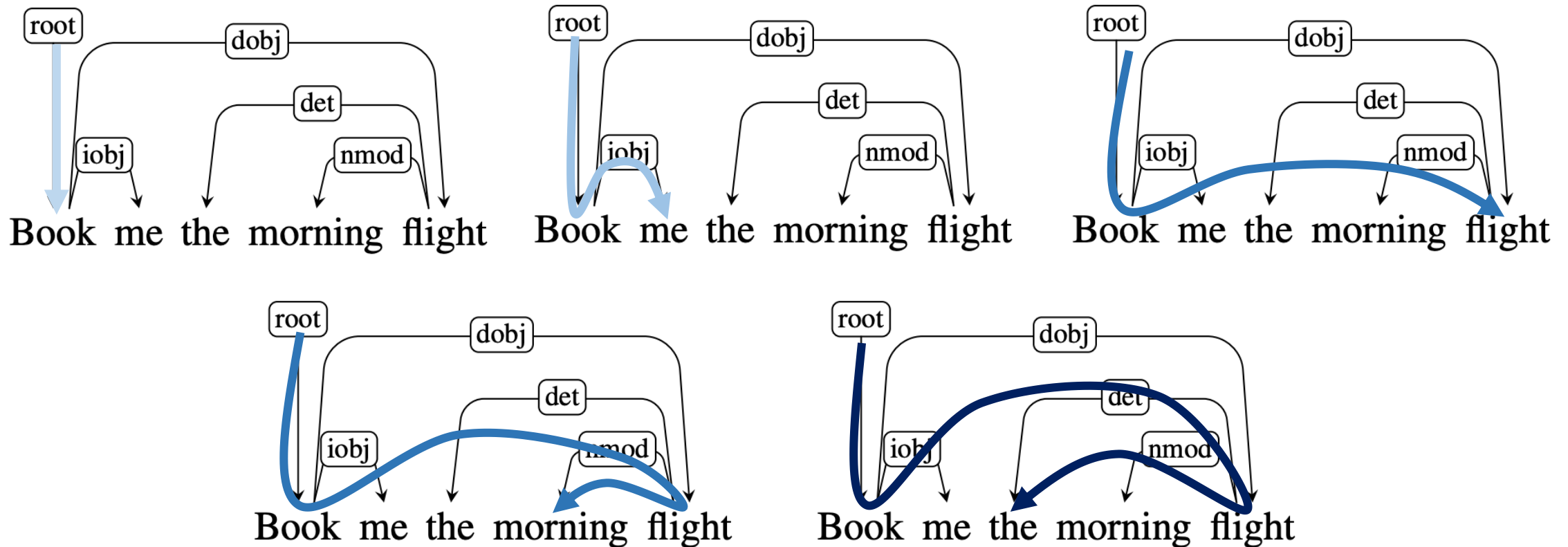
Dependency Parsing

- With the exception of the root, each word has exactly one incoming arc.



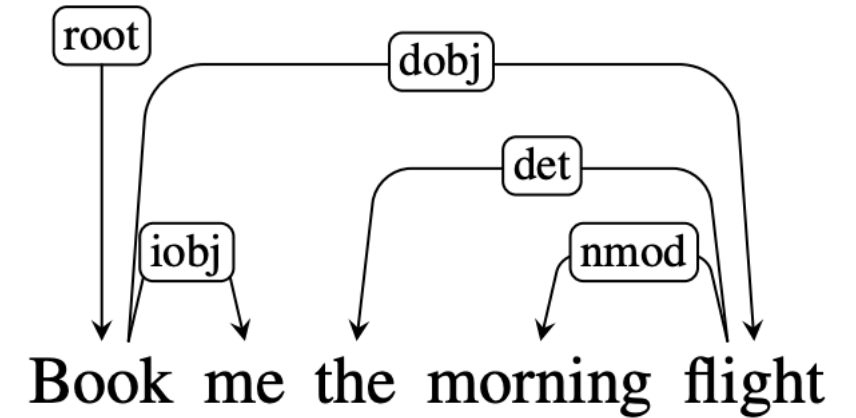
Dependency Parsing

- There is a unique path from the root node to each word.



Dependency Parsing

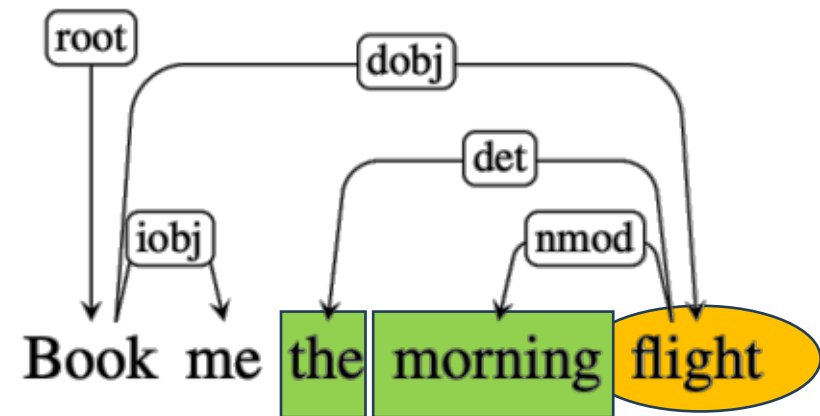
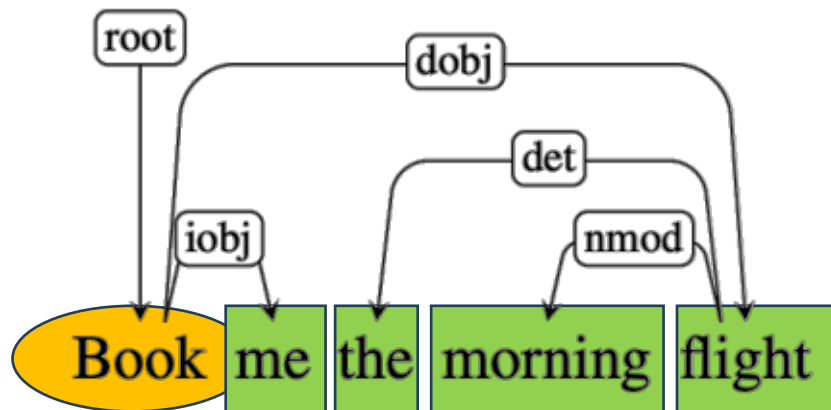
- No cycles: $A \rightarrow B, B \rightarrow C, C \rightarrow A$



Dependency Parsing: *Projectivity*

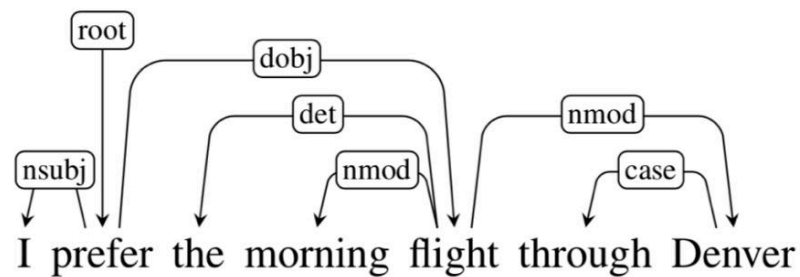
Definition: An arc from a head to a dependent is said to be projective if there is a path from the head to every word that lies between the head and the dependent in the sentence.

A dependency tree is then said to be projective if all the arcs that make it up are projective

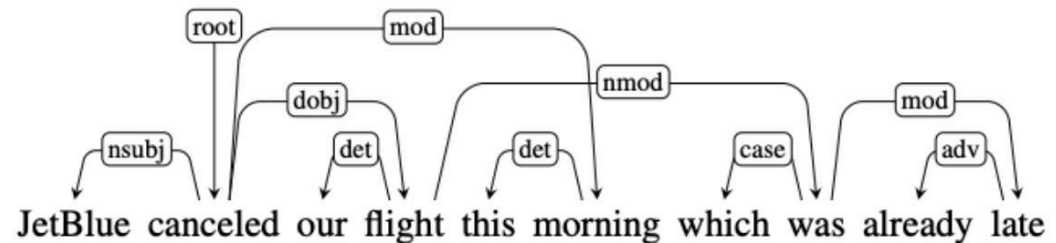


Dependency Parsing: *Projectivity*

Interpretation: there are no crossing dependency arcs when the words are laid out in their linear order, with all arcs above the words



projective

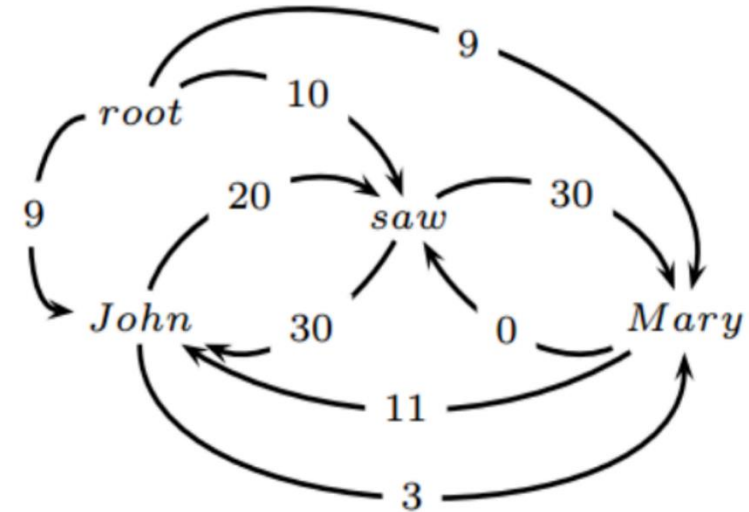
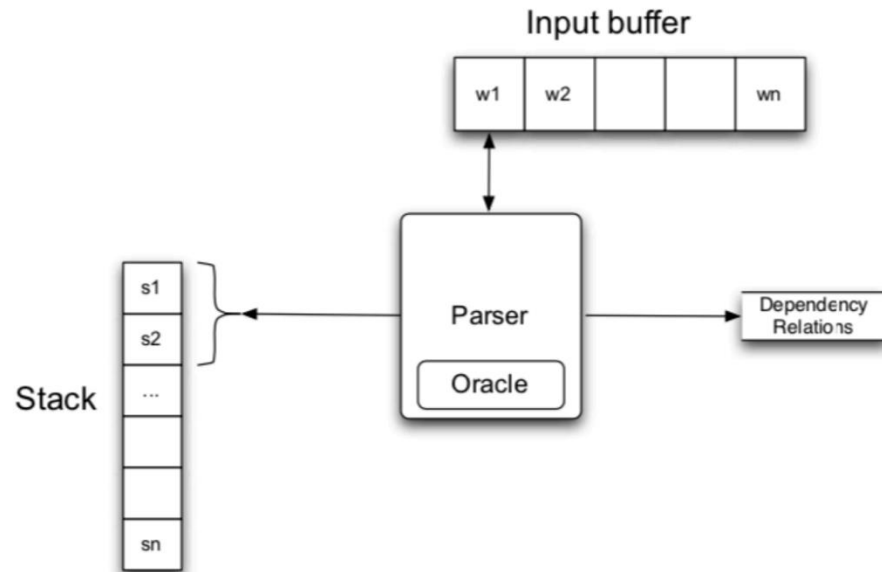


non-projective

NOTE: Non-projectivity arises due to long distance dependencies or in languages with flexible word order.

Dependency Parsing: *Algorithms*

Transition-based dependency parsing



Graph-based dependency parsing

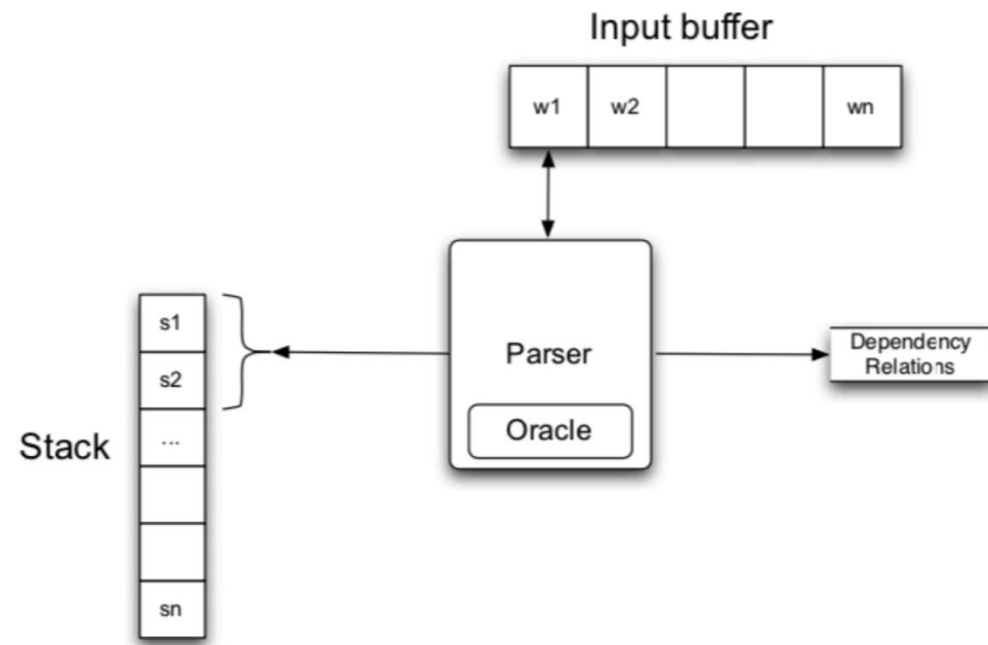
Dependency Parsing: *Algorithms*

Transition-based dependency parsing

- Sometimes referred to as “shift reduce”
- motivated by a stack-based approach called shift-reduce parsing originally developed for analyzing programming languages (Aho and Ullman, 1972).

- **Components**

- Buffer
- Stack
- Parser



Dependency Parsing: *Algorithms*

Transition-based dependency parsing

Initial and Final Condition

Initial

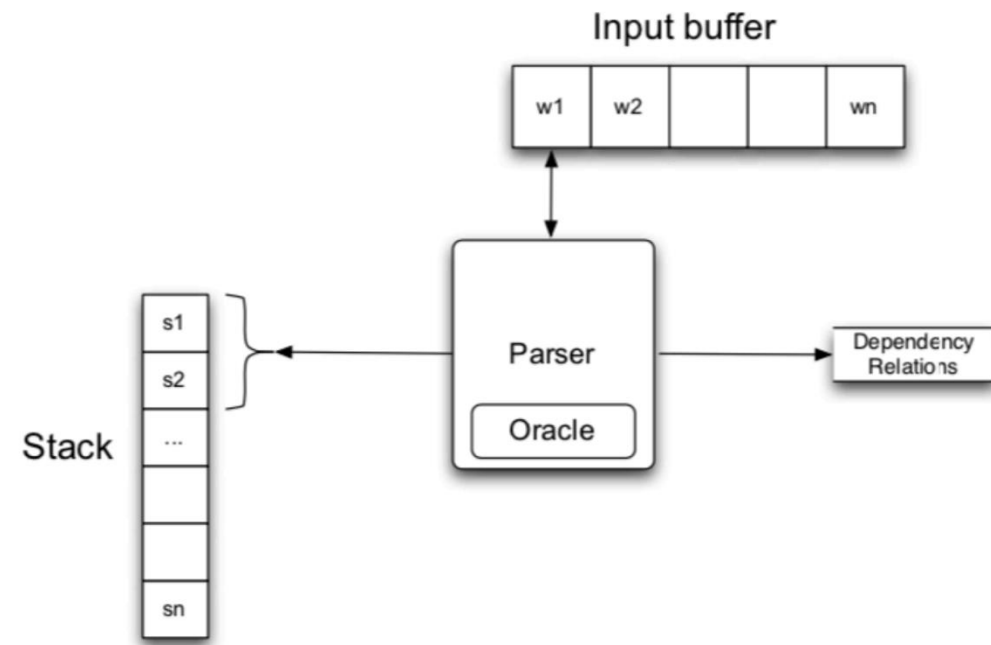
Stack: [root]

Buffer: [sentence's words]

No edges

Final

Buffer: []



Dependency Parsing: *Algorithms*

Transition-based dependency parsing

Method

- 1. Process a sentence word by word from a buffer*
- 2. Temporarily place words on a stack*
- 3. Perform an Action*

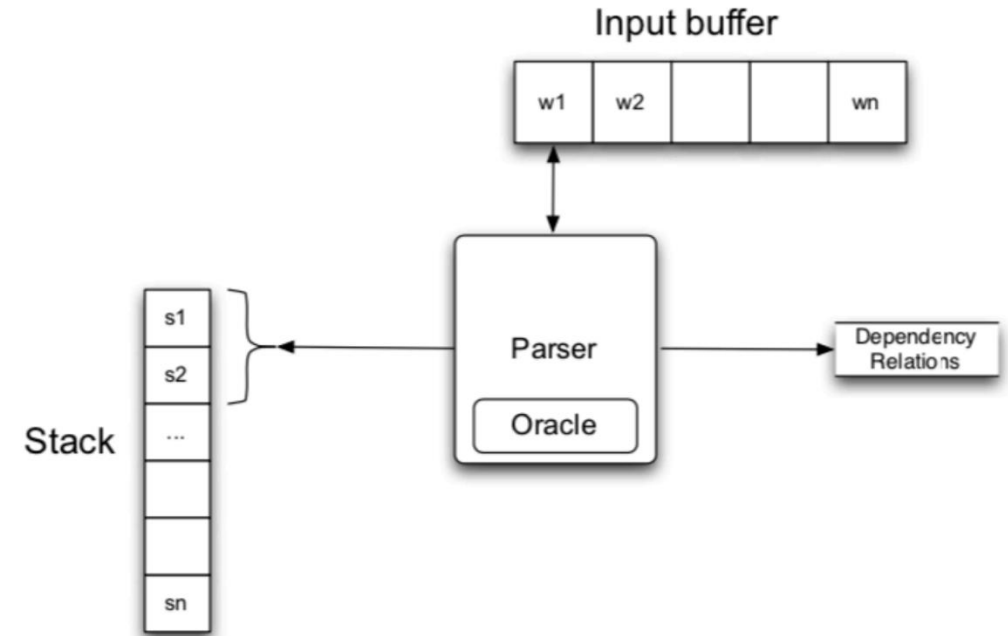
Dependency Parsing: *Algorithms*

Transition-based dependency parsing

Method

- 1. Process a sentence word by word from a buffer*
- 2. Temporarily place words on a stack*
- 3. Perform an Action:*

- *Left-arc (reduce):*
 - Assert an edge between the word at the top of the stack and the word directly beneath it: $edge(s_1, s_2)$
 - Remove the lower word from the stack.

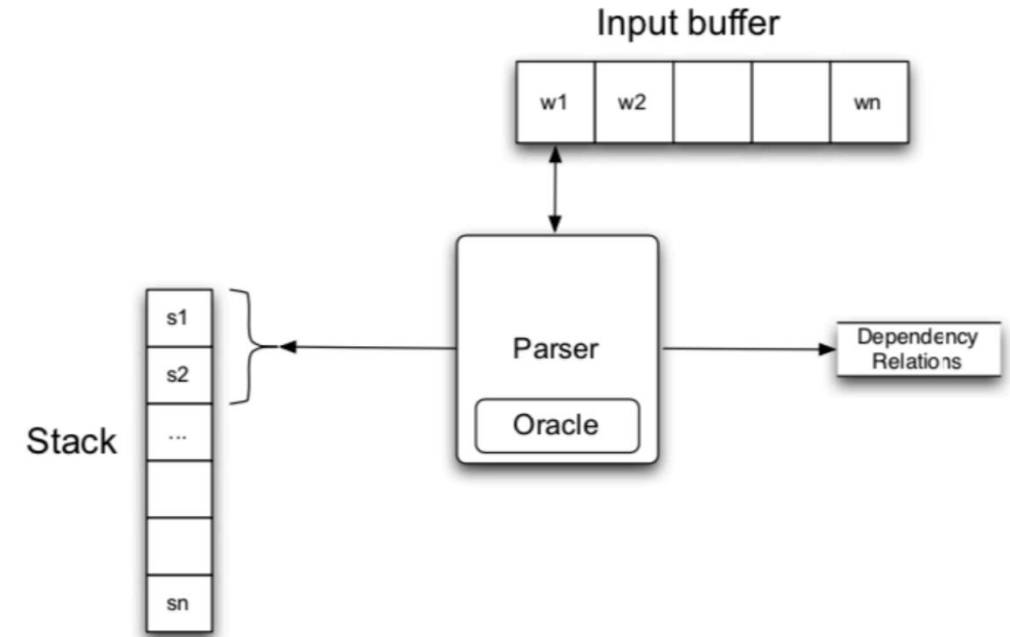


Dependency Parsing: *Algorithms*

Transition-based dependency parsing *Method*

Another way for Left-arc (reduce)

- Add *edge* (w_1, s_1) w_1 : First word of buffer
 s_1 : top of stack
- Pop the stack
- Conditions:
 - Stack and buffer must be non-empty
 - w_1 cannot be root



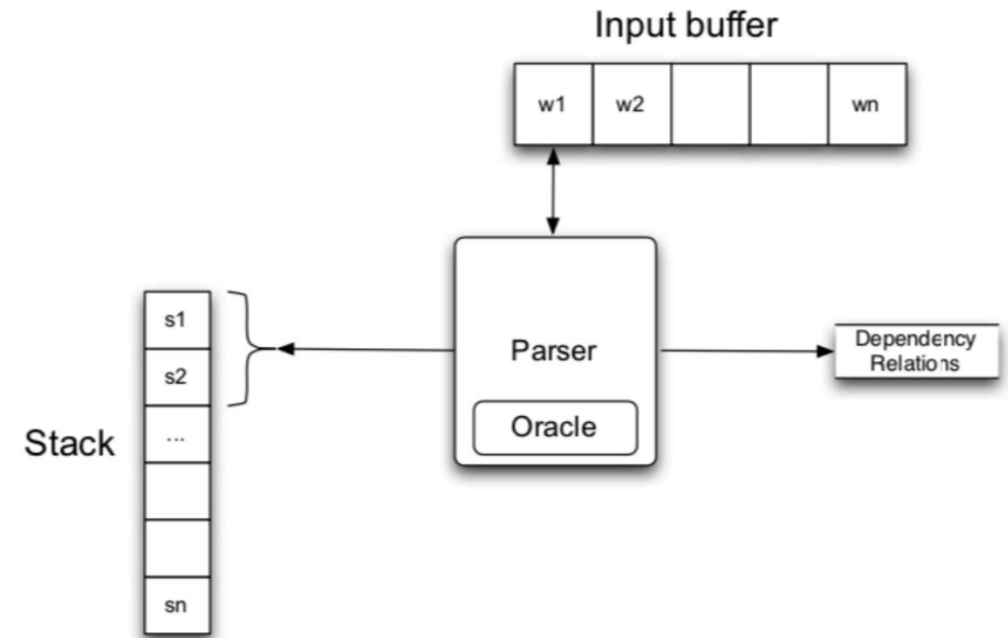
Dependency Parsing: *Algorithms*

Transition-based dependency parsing

Method

- 1. Process a sentence word by word from a buffer*
- 2. Temporarily place words on a stack*
- 3. Perform an Action:*

- *Right-arc (reduce):*
 - Assert an edge between the second word on the stack and the word at the top;
 $edge(s_2, s_1)$
 - remove the word at the top of the stack

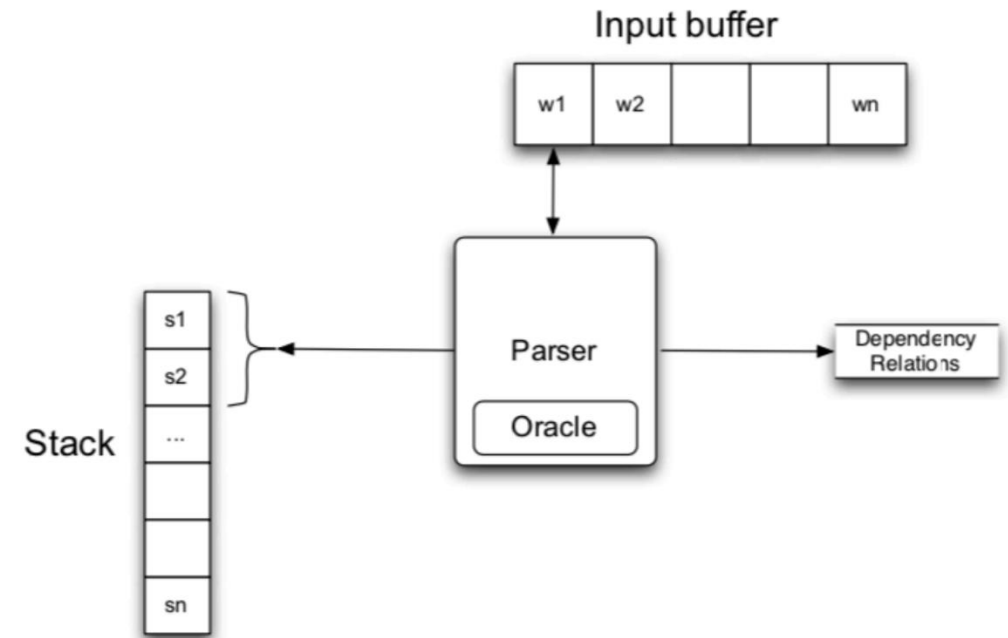


Dependency Parsing: *Algorithms*

Transition-based dependency parsing *Method*

Another way for Right-arc (reduce)

- Add *edge* (s_1, w_1) w_1 : First word of buffer
 s_1 : top of stack
- Pop the stack
- Replace w_1 with s_1 at the head of the buffer



Dependency Parsing: *Algorithms*

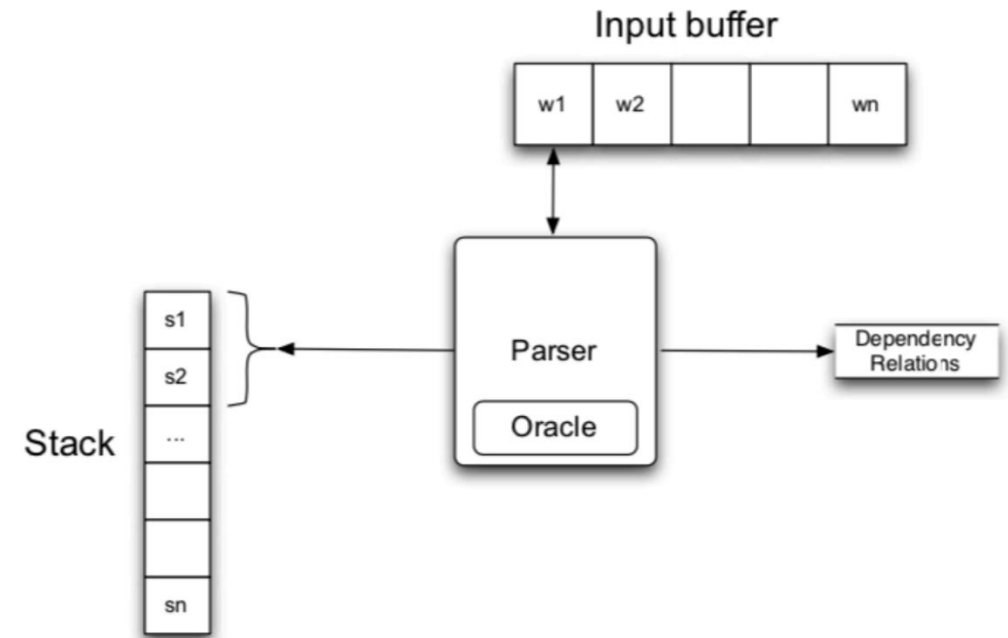
Transition-based dependency parsing

Method

- 1. Process a sentence word by word from a buffer*
- 2. Temporarily place words on a stack*
- 3. Perform an Action:*

Shift:

Remove the word (w_1) from the front of the input buffer and push it onto the top of the stack.



Dependency Parsing: *Algorithms*

Transition-based dependency parsing

Example *Economic news had little effect on financial markets .*

Stack

[root]

Buffer

[*Economic, news, had, little, effect, on, financial, markets, .*]

Action

Root Economic news had little effect on financial markets .

Dependency Parsing: *Algorithms*

Transition-based dependency parsing

Example *Economic news had little effect on financial markets .*

Stack

[root *Economic*]

Buffer

[*news, had, little, effect, on,*
financial, markets, .]

Action

shift

Root Economic news had little effect on financial markets .

Dependency Parsing: *Algorithms*

Transition-based dependency parsing

Example *Economic news had little effect on financial markets .*

Stack

[root *Economic*]

Buffer

[*news, had, little, effect, on, financial, markets, .*]

Action

Left-arc

Root Economic news had little effect on financial markets .

Dependency Parsing: *Algorithms*

Transition-based dependency parsing

Example *Economic news had little effect on financial markets .*

Stack

[root]

Buffer

[*news, had, little, effect, on, financial, markets, .*]

Action

Left-arc

Root *Economic* ← *news* *had* *little* *effect* *on* *financial* *markets* .

Dependency Parsing: *Algorithms*

Transition-based dependency parsing

Example *Economic news had little effect on financial markets .*

Stack

[root **news**]

Buffer

[*had, little, effect, on, financial, markets, .*]

Action

shift

Root Economic ← news had little effect on financial markets .

Dependency Parsing: *Algorithms*

Transition-based dependency parsing

Example *Economic news had little effect on financial markets .*

Stack

[root **news**]

Buffer

[*had, little, effect, on, financial, markets, .*]

Action

Left-arc

Root Economic ← news had little effect on financial markets .

Dependency Parsing: *Algorithms*

Transition-based dependency parsing

Example *Economic news had little effect on financial markets .*

Stack

[root]

Buffer

[*had, little, effect, on, financial, markets, .*]

Action

Left-arc

Root Economic ← news ← had little effect on financial markets .

Dependency Parsing: *Algorithms*

Transition-based dependency parsing

Example *Economic news had little effect on financial markets .*

Stack

[root *had*]

Buffer

[*little, effect, on, financial, markets, .*]

Action

shift

Root *Economic* ← *news* ← *had* *little* *effect* *on* *financial* *markets* .

Dependency Parsing: *Algorithms*

Transition-based dependency parsing

Example *Economic news had little effect on financial markets .*

Stack

[root *had little*]

Buffer

[*effect, on, financial, markets, .*]

Action

shift

Root Economic ← news ← had little effect on financial markets .

Dependency Parsing: *Algorithms*

Transition-based dependency parsing

Example *Economic news had little effect on financial markets .*

Stack

[root *had*]

Buffer

[*effect, on, financial, markets, .*]

Action

Left-arc

Root *Economic* ← *news* ← *had* *little* ← *effect* *on* *financial* *markets* .

Dependency Parsing: *Algorithms*

Transition-based dependency parsing

Example *Economic news had little effect on financial markets .*

Stack

[root *had effect*]

Buffer

[*on, financial, markets, .*]

Action

shift

Root *Economic* ← *news* ← *had* *little* ← *effect* *on* *financial* *markets* .

Dependency Parsing: *Algorithms*

Transition-based dependency parsing

Example *Economic news had little effect on financial markets .*

Stack

[root *had effect on*]

Buffer

[*financial, markets, .*]

Action

shift

Root Economic ← *news* ← *had* *little* ← *effect* *on* *financial* *markets* *.*

Dependency Parsing: *Algorithms*

Transition-based dependency parsing

Example *Economic news had little effect on financial markets .*

Stack

[root *had effect on financial*]

Buffer

[*markets, .*]

Action

shift

Root *Economic* ← *news* ← *had* *little* ← *effect* *on* *financial* *markets* .

Dependency Parsing: *Algorithms*

Transition-based dependency parsing

Example *Economic news had little effect on financial markets .*

Stack

[root *had effect on*]

Buffer

[*markets, .*]

Action

Left-arc

Root *Economic* ← *news* ← *had* *little* ← *effect* *on* *financial* ← *markets* .

Dependency Parsing: *Algorithms*

Transition-based dependency parsing

Example *Economic news had little effect on financial markets .*

Stack

[root *had effect*]

Buffer

[*on*, .]

Action

Right-arc

Root *Economic* ← *news* ← *had* *little* ← *effect* *on* *financial* ← *markets* .



Dependency Parsing: *Algorithms*

Transition-based dependency parsing

Example *Economic news had little effect on financial markets .*

Stack

[root *had*]

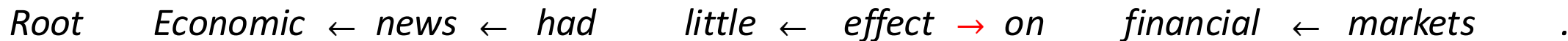
Buffer

[*effect*, .]

Action

Right-arc

Root *Economic* ← *news* ← *had* *little* ← *effect* → *on* *financial* ← *markets* .



Dependency Parsing: *Algorithms*

Transition-based dependency parsing

Example *Economic news had little effect on financial markets .*

Stack

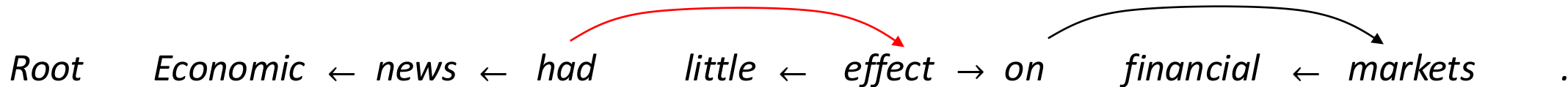
[root]

Buffer

[*had*, .]

Action

Right-arc



Dependency Parsing: *Algorithms*

Transition-based dependency parsing

Example *Economic news had little effect on financial markets .*

Stack

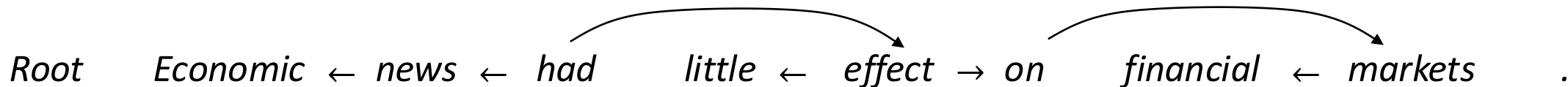
[root *had*]

Buffer

[*.*]

Action

shift



Dependency Parsing: *Algorithms*

Transition-based dependency parsing

Example *Economic news had little effect on financial markets .*

Stack

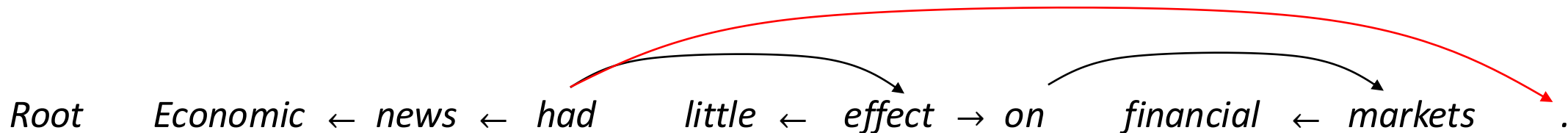
[root]

Buffer

[*had*]

Action

shift



Dependency Parsing: *Algorithms*

Transition-based dependency parsing

Example *Economic news had little effect on financial markets .*

Stack

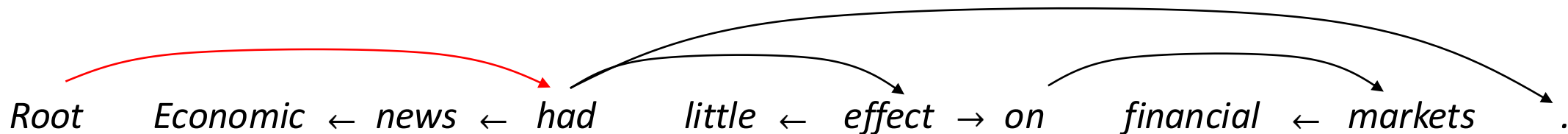
[root]

Buffer

[*had*]

Action

Right-arc



Dependency Parsing: *Algorithms*

Transition-based dependency parsing

Oracle

supervised classifier that learns a function from the configuration to the next operation

Labels

Shift

Left-arc

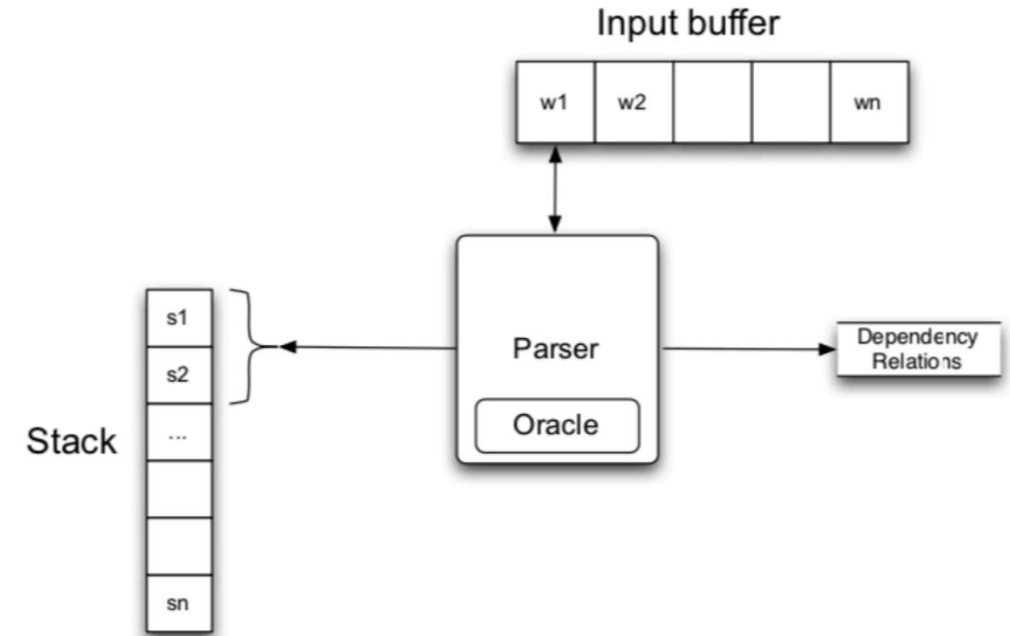
Right-arc

Features

Top of the stack

POS-tags

...



Dependency Parsing: *Algorithms*

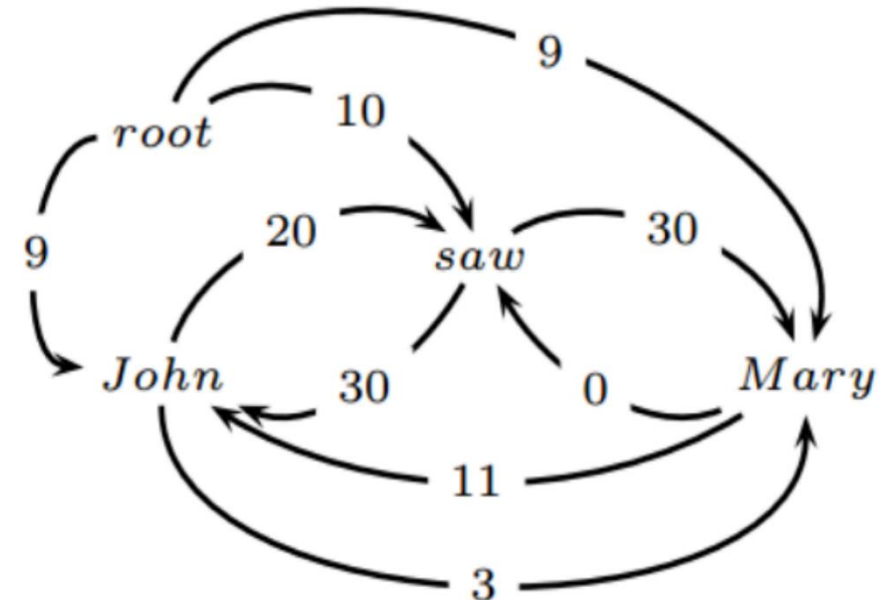
Graph-based dependency parsing

For a given sentence:

Nodes: Each word in the sentence is represented as a node in the graph.

Edges: Directed edges connect pairs of words (nodes), indicating the syntactic dependencies. The direction of the edge typically goes from a "head" (governor) to a "dependent" (modifier).

John saw Mary



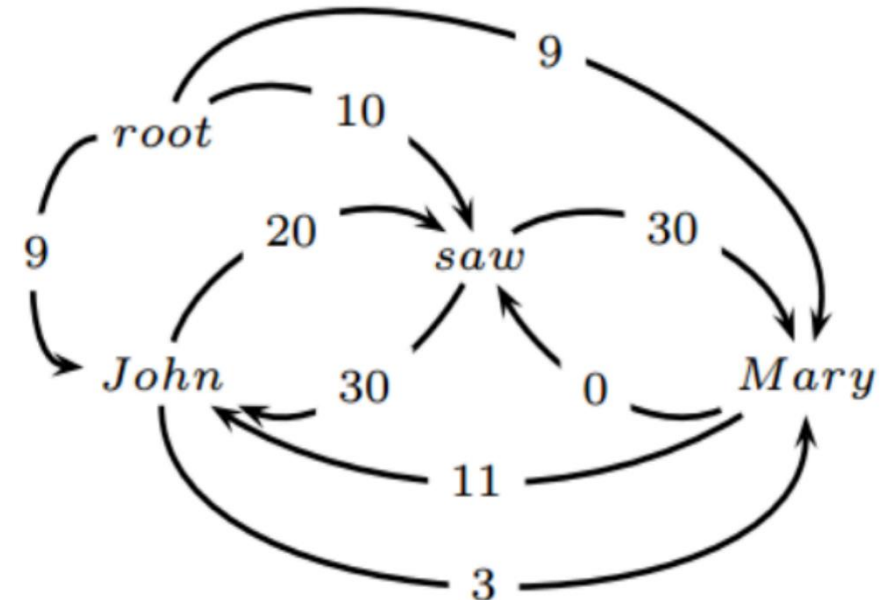
Dependency Parsing: *Algorithms*

Graph-based dependency parsing

Goal:

The goal of graph-based parsing is to find the most probable dependency tree (graph) for a given sentence. The graph must be a tree, meaning it is connected and acyclic, and there is exactly one node with no incoming edges, known as the root (usually the main verb or predicate of the sentence).

John saw Mary



Dependency Parsing: *Algorithms*

Graph-based dependency parsing

Algorithms:

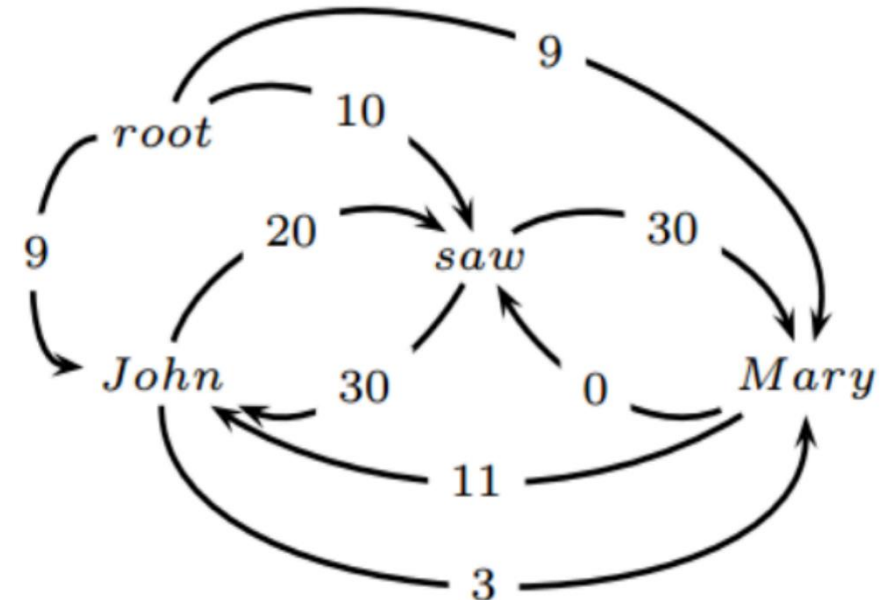
Eisner's Algorithm: An efficient algorithm for projective parsing (where edges do not cross when drawn above the sentence).

([link](#))

Chu-Liu/Edmonds' Algorithm: Finds the maximum spanning tree in a directed graph, used for non-projective parsing (where edges can cross).

([link](#))

John saw Mary



Dependency Parsing: *Algorithms*

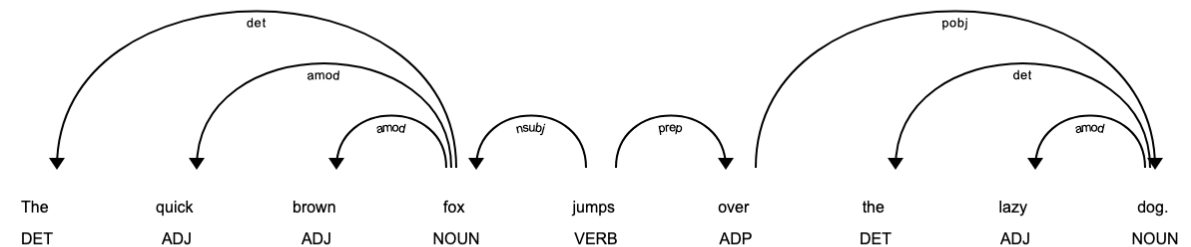
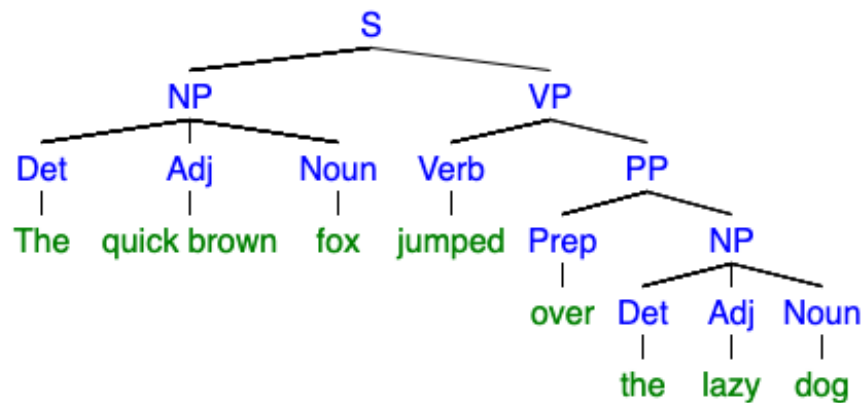
Parser		Test	
		UAS	LAS
(Chen and Manning, 2014)	T	91.8	89.6
(Dyer et al., 2015)		93.1	90.9
(Ballesteros et al., 2016)		93.56	92.41
(Weiss et al., 2015)		94.26	91.42
(Andor et al., 2016)		94.61	92.79
(Ma et al., 2018) §		95.87	94.19
(Kiperwasser and Goldberg, 2016a) §	G	93.0	90.9
(Kiperwasser and Goldberg, 2016b)		93.1	91.0
(Wang and Chang, 2016)		94.08	91.82
(Cheng et al., 2016)		94.10	91.49
(Kuncoro et al., 2016)		94.26	92.06
(Zheng, 2017) §		95.53	93.94
(Dozat and Manning, 2017)		95.74	94.08
Baseline	G	95.68	93.96
Our Model §		95.97	94.31

T: transition-based / G: graph-based

Constituency / Dependency Grammar

Constituency trees can (potentially) be converted to Dependency trees.

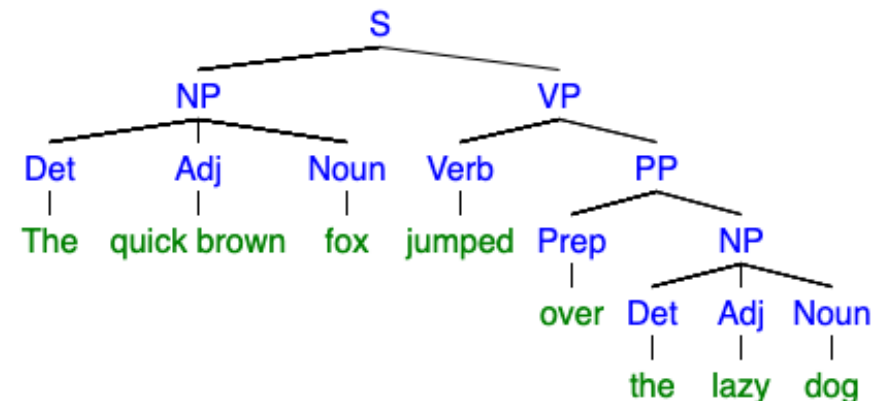
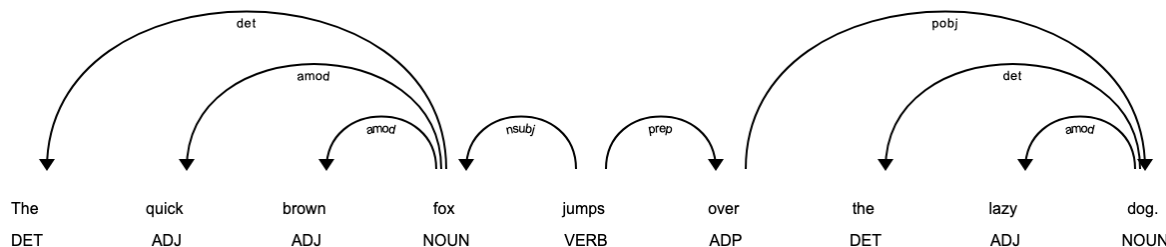
- "jumped" is the main verb and the root.
- "fox" (head of the first NP) is the subject of "jumps".
- "over" (head of the PP) is a dependent of "jumps", and "dog" (head of the second NP) is the object of "over".
- Adjectives "quick" and "brown" are dependents of "fox"; "lazy" is a dependent of "dog".
- Determiners "the" are dependents of their respective nouns ("fox" and "dog").



Constituency / Dependency Grammar

Dependency trees can (potentially) be converted to constituency trees.

- Identify the verb "jumps" as the main action. This suggests a VP.
- "fox" and its modifiers form an NP.
- "over the lazy dog" represents a prepositional phrase (PP), with "dog" and its modifiers forming another NP.
- The sentence structure follows $S \rightarrow NP VP$.



Syntactic Parsing: *Methods*

Rule-based Parsing

uses a set of hand-crafted rules to generate parse trees. These rules are based on the grammar of the language and are typically expressed in a formal grammar, such as a context-free grammar (CFG).

Statistical Parsing

Neural Parsing (data-driven)

Recursive Descent Parser

A recursive descent parser is a top-down parser that works by recursively calling itself to match the input symbols with the rules of the grammar. It follows a predetermined set of rules to break down the input sentence into its constituent parts. Recursive descent parsers are efficient and easy to implement, but they can be difficult to write for complex grammars.

Chart Parser

A chart parser is a type of bottom-up parser that builds a chart of all possible partial parses of the input sentence. The chart is a data structure that stores the possible ways to combine the symbols in the input sentence. The parser iteratively fills in the chart by applying the rules of the grammar to the symbols in the chart. Chart parsers are more complex than shift-reduce parsers, but they can handle complex grammars and ambiguous sentences.

Regex Parser

A regular expression parser is a type of parser that uses regular expressions to match patterns in text. Regular expressions are a powerful tool for pattern matching, but they are not as expressive as context-free grammars (CFGs), which are the type of grammar that is used most often in NLP parsing.

Syntactic Parsing: *Rule-based*

[optional]

Method	Algorithm	Strengths	Weaknesses
Recursive Descent Parser	Top-down	Efficient, easy to implement	Difficult for complex grammars
Chart Parser	Bottom-up	Can handle complex grammars and ambiguous sentences	More complex
Regex Parser	Pattern matching	Powerful for pattern matching	Not as expressive as CFGs

Syntactic Parsing: *Methods*

Rule-based Parsing

Statistical Parsing

uses statistical models to generate parse trees. These models are trained on a large corpus of text and learn to associate words with their grammatical roles based on their past occurrences.

Neural Parsing (data-driven)

Syntactic Parsing: *Methods*

Rule-based Parsing

Statistical Parsing

- Discriminative
- Generative

Neural Parsing (data-driven)

- **Hidden Markov Models (HMMs):** model the grammatical structure of a sentence by assuming that the part-of-speech (POS) are hidden states and the words are observed symbols.
- **Probabilistic Context-Free Grammars (PCFGs):** a type of context-free grammar (CFG) that assigns probabilities to each grammar rule.
- **Maximum Entropy Models (MEMs):** able to capture long-range dependencies between words in a sentence. They are often used in conjunction with PCFGs to improve parsing accuracy.

Syntactic Parsing: *Methods*

Rule-based Parsing

Statistical Parsing

Neural Parsing (data-driven)

uses neural networks to generate parse trees. Neural networks are trained on a large corpus of text and learn to parse sentences based on their input and output representations.

- **Recurrent Neural Networks (RNNs)**
- **Convolutional Neural Networks (CNNs)**
- **Attention Mechanisms and Transformers**
- **Graph-Based Neural Models**

Syntactic Parsing: *Methods*

Method	Description	Advantages	Disadvantages
Rule-based parsing	Uses a set of hand-crafted rules	Efficient and accurate for well-formed sentences	Can be brittle and fail to handle ungrammatical sentences
Statistical parsing	Uses statistical models to generate parse trees	Robust to ungrammatical sentences	Can be less accurate than rule-based parsing
Neural parsing	Uses neural networks to generate parse trees	Highly accurate and can handle complex sentences	Can be computationally expensive and require large amounts of training data

Syntactic Parsing: *in Python*

POS Tagging with spaCy

```
import spacy

from spacy import displacy

# Load the language model
nlp = spacy.load("en_core_web_sm")

sentence = 'There is a nodule in the left lung.'

# nlp function returns an object with individual token information,
# linguistic features and relationships
doc = nlp(sentence)

# Iterate over the tokens
for token in doc:
    # Print the token and its part-of-speech tag
    print(token.text, "-->", token.pos_)
```

```
There --> PRON
is --> VERB
a --> DET
nodule --> NOUN
in --> ADP
the --> DET
left --> ADJ
lung --> NOUN
. --> PUNCT
```

Syntactic Parsing: *in Python*

Dependency Parsing with spaCy

```
print("{:<15} | {:<8} | {:<15} | {:<20}".format('Token', 'Relation', 'Head', 'Children'))
print("-" * 70)

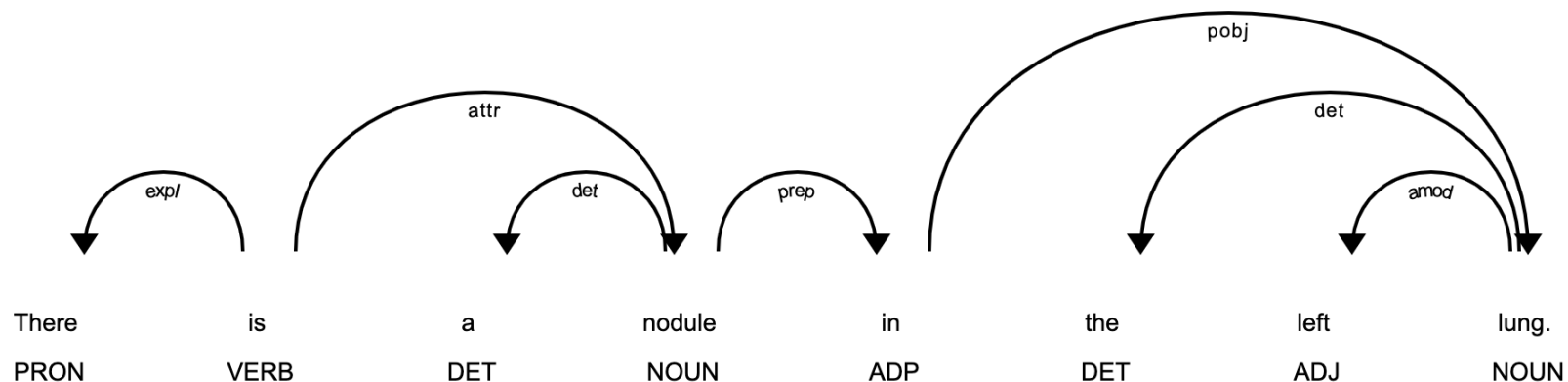
for token in doc:
    # Print the token, dependency nature, head and all dependents of the token
    print("{:<15} | {:<8} | {:<15} | {:<20}"
          .format(str(token.text), str(token.dep_), str(token.head.text), str([child for child in token.children])))

# Use displayCy to visualize the dependency
displacy.render(doc, style='dep', jupyter=True, options={'distance': 120})
```

Syntactic Parsing: *in Python*

Dependency Parsing with spaCy

Token	Relation	Head	Children
There	expl	is	[]
is	ROOT	is	[There, nodule, .]
a	det	nodule	[]
nodule	attr	is	[a, in]
in	prep	nodule	[lung]
the	det	lung	[]
left	amod	lung	[]
lung	pobj	in	[the, left]
.	punct	is	[]



Shallow Parsing / Chunking

Definition: Process of extracting short phrases or chunks from a sentence.

- Unlike full parsing, which breaks down a sentence into its syntactic components, chunking typically focuses on extracting specific pieces of information, such as noun phrases or verb phrases, without analyzing the entire sentence structure.
- sometimes referred to as shallow parsing

Example:

"The quick brown fox jumped over the lazy dog"

the noun phrases:

[The quick brown fox]NP

[the lazy dog]NP

Parsing

- **Types:**
 - Syntactic Parsing
 - Constituency Parsing
 - Dependency Parsing
 - **Semantic Parsing**

Semantic Parsing

Definition: task of translating natural language into a formal meaning representation on which a machine can act for some domain-specific application.

- go beyond syntactic parsing, which focuses on identifying the grammatical structure of a sentence, to capture the underlying intent and meaning.

Semantic Parsing

Example

“””

CHEST CT

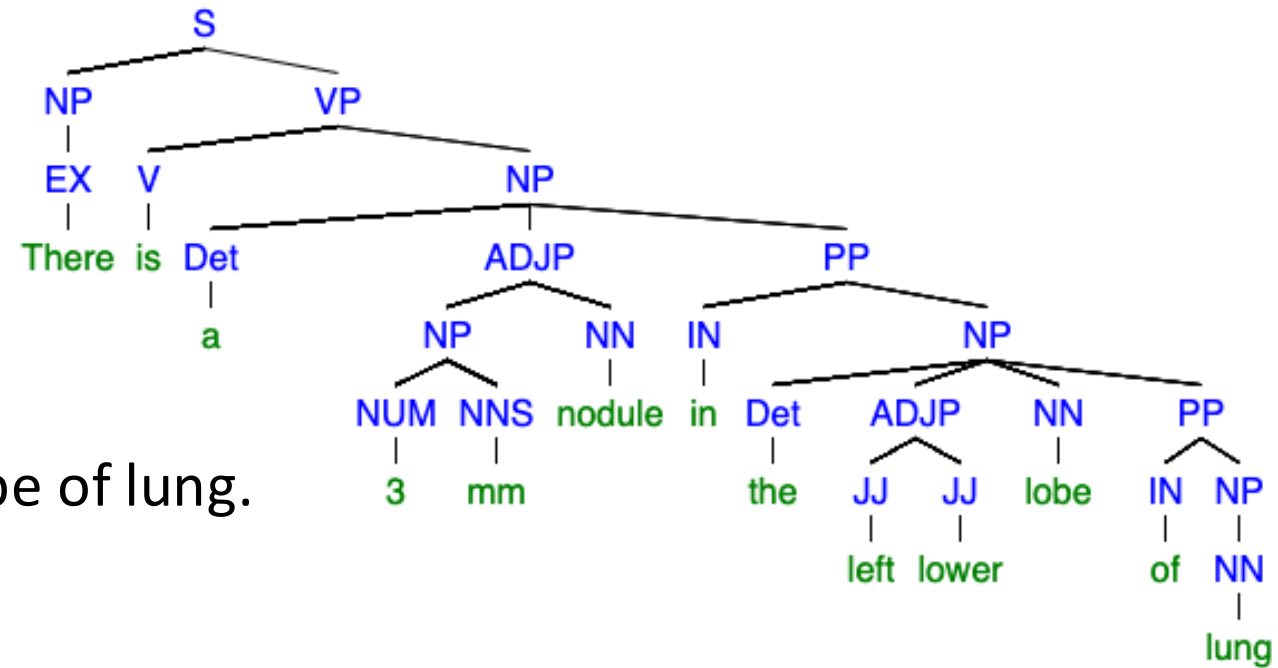
11/03/2021

...

There is a 3mm nodule in the left lower lobe of lung.

...

“””



Semantic Parsing

Example

““““

CHEST CT

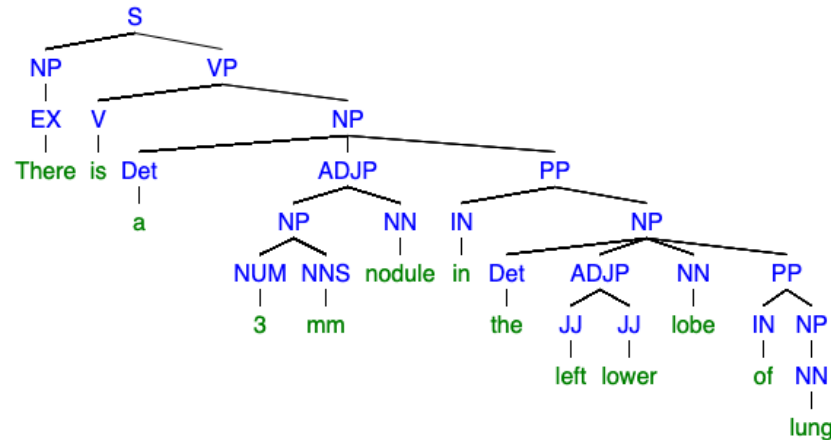
11/03/2021

...

There is a 3mm nodule in the left lower lobe of lung.

...

““““



```
{
  "event": ,
  "date": ,
  "finding": ,
  "attribute_type": ,
  "attribute_value":
}
```

Semantic Parsing

Example

“””

CHEST CT

11/03/2021

...

There is a 3mm nodule in the left lower lobe of lung.

...

“””

```
{  
  "event": "CHEST CT",  
  "date": "11/03/2021",  
  "finding": "nodule",  
  "attribute_type": "size",  
  "attribute_value": "3mm"  
}
```

Semantic Parsing: *Algorithms*

Rule-based

- “event”: keyword matching
- “date”: pattern matching
- “finding”: keyword matching
- “attribute_type”: pattern matching
- “attribute_value”: pattern matching

Example

“””

CHEST CT

11/03/2021

...

There is a 3mm nodule in the left lower lobe of lung.

...

“””

```
{  
  “event”: ,  
  “date”: ,  
  “finding”: ,  
  “attribute_type”: ,  
  “attribute_value”:  
}
```

Semantic Parsing: *Algorithms*

Statistical

OR

neural network-based

- **Discriminative algorithms:** directly predict the most likely semantic representation for a given sentence.
 - E.g., support vector machines, neural networks
- **Generative algorithms:** first generate a set of possible semantic representations for the sentence and then assign probabilities to each representation.
 - E.g., Hidden Markov Models (HMMs) or Probabilistic Context-Free Grammars (PCFGs).

Semantic Parsing: *Algorithms*

Statistical

OR

neural network-based

- Learn a representation

Example

"""

<EVENT>

<DATE>

...

There is a <ATTR_VAL> <ATTR_TYPE> in the left lower lobe of lung.

...

"""

Example

"""

CHEST CT

11/03/2021

...

There is a 3mm nodule in the left lower lobe of lung.

...

"""

{

 "event": ,

 "date": ,

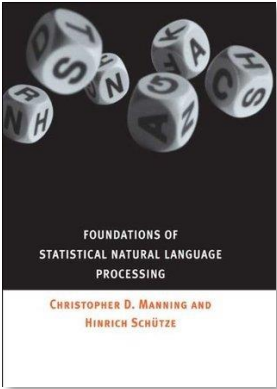
 "finding": ,

 "attribute_type": ,

 "attribute_value":

}

References



Foundations of Statistical
Natural Language Processing
([link](#))

Statistical Constituency Parsing ([link](#))

Jose Luis Verdu-Mas et al., “[A Comparison of PCFG Models](#)”, CoNLL-2000.

Speech and Language Processing, Daniel Jurafsky & James H. Martin

[Chapter 17](#): Context-Free Grammars and Constituency Parsing

[Chapter 18](#): Dependency Parsing

[Chapter 14](#): Statistical Constituency Parsing